

DLSystem – A PyTorch-lite implementation

Wei Zhang
weiz@us.ibm.com

May 27, 2026

Abstract

This document deals with deep learning system implementation. I wrote these materials by self-studying the CMU DLsys class 10714. [4] at <https://dlsyscourse.org/>.

1 Matrix Calculus Primer

This section gets most of the materials from <https://atmos.washington.edu/~dennis/MatrixCalculus.pdf>.

1.1 Gradients vs (Partial) Derivatives

(Partial) Derivatives are Jacobians (aka row vector when $f : \mathbb{R}^n \mapsto \mathbb{R}$). Gradients are vectors (Jacobian's transpose). We should stick with (Partial) Derivatives and ignore gradients. See http://michael.orlitzky.com/articles/the_derivative_of_a_quadratic_form.xhtml by <https://github.com/davidrosenberg/mlcourse/issues/2>

1.2 $\mathbf{x}^T \mathbf{y}$ partial derivative

This is the most useful partial derivative that I have learnt:

$$\begin{aligned}\alpha &= \mathbf{x}^T \mathbf{y} \\ \mathbf{x} &= x(z) : \mathbb{R}^n \mapsto \mathbb{R}^n \\ \mathbf{y} &= y(z) : \mathbb{R}^n \mapsto \mathbb{R}^n\end{aligned}\tag{1}$$

Similar to product rule in single variable calculus

$$(gf)' = g'f + f'g\tag{2}$$

$$\frac{\partial \mathbf{x}^T \mathbf{y}}{\partial \mathbf{z}} = \mathbf{y}^T \frac{\partial \mathbf{x}}{\partial \mathbf{z}} + \mathbf{x}^T \frac{\partial \mathbf{y}}{\partial \mathbf{z}}\tag{3}$$

- $\frac{\partial \mathbf{x}}{\partial \mathbf{z}}$ and $\frac{\partial \mathbf{y}}{\partial \mathbf{z}}$ are both $n \times n$ matrix.

- Equation (2) can be deducted by applying chain rule. I am able to deduct Equation (3) from a chain rule using Auto-Diff (See below). I am also able to deduct Equation (3) in a element-wise fashion.

$$\frac{\partial \alpha}{\partial y} = x^T \quad (4)$$

$$\frac{\partial \alpha}{\partial x} \stackrel{\alpha=y^T x}{=} y^T \quad (5)$$

$$\frac{\partial \alpha}{\partial z} \stackrel{\text{Auto-Diff adjoint is the sum of partial adjoints}}{=} \frac{\partial \alpha}{\partial x} \frac{\partial x}{\partial z} + \frac{\partial \alpha}{\partial y} \frac{\partial y}{\partial z} \quad (6)$$

$$= y^T \frac{\partial x}{\partial z} + x^T \frac{\partial y}{\partial z} \quad (7)$$

1.3 $\mathbf{x}^T \mathbf{A} \mathbf{x}$ partial derivative

$$\alpha = \mathbf{x}^T \mathbf{A} \mathbf{x}, \mathbf{x} \in \mathbb{R}^n, \mathbf{A} \in \mathbb{R}^{n \times n}$$

From equation (3), one can get

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = (\mathbf{A} \mathbf{x})^T \frac{\partial \mathbf{x}}{\partial \mathbf{x}} + \mathbf{x}^T \frac{\partial \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} \quad (8)$$

$$= \mathbf{x}^T \mathbf{A}^T \mathbf{I} + \mathbf{x}^T \mathbf{A} \quad (9)$$

$$= \mathbf{x}^T (\mathbf{A}^T + \mathbf{A}) \quad (10)$$

1.4 A list of useful equations

$$\alpha \in \mathbb{R}, \mathbf{a} \in \mathbb{R}^n, \mathbf{A} \in \mathbb{R}^{n \times n}$$

$$\mathbf{x} \in \mathbb{R}^n$$

$$\frac{\partial \alpha \mathbf{x}}{\partial \mathbf{x}} = \alpha \quad (11)$$

$$\frac{\partial \mathbf{a}^T \mathbf{x}}{\partial \mathbf{x}} = \mathbf{a}^T \quad (12)$$

$$\frac{\partial \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = \mathbf{A} \quad (13)$$

$$\frac{\partial \mathbf{x}^T \mathbf{x}}{\partial \mathbf{x}} = 2\mathbf{x}^T \quad (14)$$

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = \mathbf{x}^T (\mathbf{A}^T + \mathbf{A}) \quad (15)$$

Equation (13) shows matrix multiplication is a linear transformation, which can be modeled perfectly by a linear approximation (aka Jacobian)

Equation (14) and Equation (15) can be obtained by using Equation (3)

1.5 Some formulas by me

1.5.1 Hadamard Product, aka Element-wise Product.

Some useful rules: $A, B \in R^{m \times n}$, $x = (x_1, \dots, x_m)^T \in R^m$, $y = (y_1, \dots, y_n)^T \in R^n$

$$A \odot B = B \odot A \quad (16)$$

$$A^T \odot B^T = (A \odot B)^T \quad (17)$$

$$A \odot \begin{bmatrix} | \\ x \\ | \end{bmatrix} = A \odot \underbrace{\begin{bmatrix} | & & | \\ x & \dots & x \\ | & & | \end{bmatrix}}_{m \times n} = \underbrace{\begin{bmatrix} x_1 & & \\ & \ddots & \\ & & x_m \end{bmatrix}}_{m \times m} A \quad (18)$$

$$A \odot \begin{bmatrix} \text{---} & y^T & \text{---} \end{bmatrix} = A \odot \underbrace{\begin{bmatrix} \text{---} & y^T & \text{---} \\ & \dots & \\ \text{---} & y^T & \text{---} \end{bmatrix}}_{m \times n} = A \underbrace{\begin{bmatrix} y_1 & & \\ & \ddots & \\ & & y_n \end{bmatrix}}_{n \times n} \quad (19)$$

Equation (18) depicts matrix hadamard multiply a column vector, which scales A's rows. *weiz 2023-11-19 this is Numpy's broadcast semantics, I am not sure if hadamard product really allows two matrices of different shape.*

Equation (19) depicts matrix hadamard multiply a row vector, which scales A's columns. *weiz 2023-11-19 this is Numpy's broadcast semantics, I am not sure if hadamard product really allows two matrices of different shape.*

1.5.2 Why numpy array is a row vector, in the perspective of Hadamard product (update 2024-02-02 actually in the perspective of bcast) ?

`np.multiply()`

is a Hadamard product (aka element-wise product), which broadcasts a vector when it multiplies with a dimension-matching matrix.

Example:

```
>>> A=np.arange(6).reshape(2,3)
>>> A
array([[0, 1, 2],
       [3, 4, 5]])
>>> a=np.array([1,2,3])
>>> np.multiply(A,a)
array([[ 0,  2,  6],
       [ 3,  8, 15]])
>>> b=np.array([1,2])
>>> np.multiply(A,b)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
ValueError: operands could not be broadcast together with shapes
      ↪ (2,3) (2,)
```

Were numpy array a column vector, $\mathbf{b}$ would be broadcast to a 2x3 matrix and able to multiply with $\mathbf{A}$. So the conclusion is that numpy array is a row vector, from a Hadamard perspective. [weiz update 2024-02-02: This is actually more generally referring to the bcast rule, or See https://stackoverflow.com/questions/61593567/does-1-dimension-al-numpy-array-always-behave-like-a-row-vector](https://stackoverflow.com/questions/61593567/does-1-dimension-al-numpy-array-always-behave-like-a-row-vector)

1.5.3 A Special Case of Vector Function Jacobian

$\mathbf{x} \in \mathbb{R}^n, f : \mathbb{R}^n \mapsto \mathbb{R}^n$ In particular, f_i only depends on x_i

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \underbrace{\begin{bmatrix} \frac{\partial f_1(x_1)}{x_1} & & \\ & \ddots & \\ & & \frac{\partial f_n(x_n)}{x_n} \end{bmatrix}}_{n \times n} \quad (20)$$

When f is an identity function, its Jacobian is an identity matrix.

1.6 Chain Rule

Suppose $\mathbf{x} \in \mathbb{R}^n$

$\mathcal{L} : \mathbb{R}^n \mapsto \mathbb{R}^m$

$f_i : \mathbb{R}^{k_{i-1}} \mapsto \mathbb{R}^{k_i} (k_0 = n, k_l = m)$

$D_{f_l} \in \mathbb{R}^{k_l \times k_{l-1}}$

$D_{\mathcal{L}} \in \mathbb{R}^{m \times n}$

D is the partial differentiation operator, D_f denotes the Jacobian matrix of function f .

$$\mathcal{L}(\mathbf{x}) = f_l(f_{l-1}(\dots f_1(x)\dots)) \quad (21)$$

$$D_{\mathcal{L}} = D_{f_l} D_{f_{l-1}} \dots D_{f_1} \quad (22)$$

Equation (22) is the definition of Chain Rule.

2 Matrix Calculus used in Deep Learning

This section deals mainly with back propagation.[8] is the main source of dealing with matrix differential calculus. In addition, Stefan's teaching at [5, 6] are extremely helpful. Also I proved the paramount 4 BP equations described in [10].

2.1 Hadamard Product (two vectors)

$x, y \in R^n$

$$x \odot y = \begin{bmatrix} x_1 y_1 \\ \vdots \\ x_n y_n \end{bmatrix} = \underbrace{\begin{bmatrix} x_1 & & \\ & \ddots & \\ & & x_n \end{bmatrix}}_{n \times n} y \quad (23)$$

$$x^T \odot y^T = [x_1 y_1 \dots x_n y_n] = (x \odot y)^T = y^T \underbrace{\begin{bmatrix} x_1 & & \\ & \ddots & \\ & & x_n \end{bmatrix}}_{n \times n} \quad (24)$$

2.2 vec and de-vec_{m×n}

vec operator (operator is a linear mapping that maps from one subspace to the same subspace, see [1], so see next paragraph): $X \in R^{m \times n}$,

$$vec(X) = \begin{bmatrix} X_1 \\ \vdots \\ X_n \end{bmatrix} \quad (25)$$

X_j is the j -th column of X . **weiz:update 2024-06-01, this is quite intuitive, as this is to concatenate n vectors in the R^m space to become a vector in the R^{mn} vector space.**

Also, it is not difficult to prove

$$vec(A + B) = vec(A) + vec(B) \quad (26)$$

$$\alpha vec(A) = vec(\alpha A) \quad (27)$$

$$Null(vec) = \{\vec{0}\} \quad (28)$$

As a result, **vec is an invertible linear operator.**

de-vec_{m×n} operation (an invention by Wei): It is easy to see that vec operator is invertible as the $Null(vec) = \vec{0}$. We define the inverse operator of vec as **de-vec_{m×n}**, i.e.,

$$X = \text{de-vec}_{m \times n}(vec(X)) \quad (29)$$

de-vec operator first cuts the long vector into n pieces, then horizontally stack these n pieces as n columns to construct a m by n matrix.

Note **vec** is a build-in operator in Matlab, in numpy, we need to do this:

```
import numpy as np
def vec(X):
    return (X.transpose().flatten().reshape(1,-1)).transpose()
```

```
def vec(X):
    return X.reshape((-1, 1), order="F")
def devec(x, m, n):
    return (x.transpose().reshape(n,m)).transpose()
```

Alternative, one can do this <https://stackoverflow.com/questions/25248290/most-elegant-implementation-of-matlabs-vec-function-in-numpy> by using column formatting

```
import numpy as np
def vec(X):
    return X.reshape((-1, 1), order="F")
def devec(x, m, n):
    return x.reshape(m, n, order="F")
```

weiz 2024-06-07, note on transpose: Let $x \in R^{n \times 1}$, then

$$\begin{aligned} \text{vec}(x) &= \text{vec}(x^T) = x \quad [8] \text{ p34(11)} \\ \implies d(\text{vec}(x^T)) &= d(x) \\ \implies \text{The Jacobian matrix of transpose} &\text{ is Identity matrix} \end{aligned} \tag{30}$$

However,

$$d(x^T) = d(x)^T 5 \neq d(x) \tag{31}$$

Even though Jacobian is Identity, differential-wise $d(x^T) \neq d(x)$,
because Jacobian is really w.r.t to the vectorization of x^T

whereas differential needs to make sure it has the same shape as function output

Maybe I should write an email to ask Magnus!

2.3 The Kronecker Product

Let A be an $m \times n$ matrix and B a $p \times q$ matrix. The $mp \times nq$ matrix defined by

$$\begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix}$$

is called the *Kronecker product* of A and B and is written $A \otimes B$ [8] (p31).

Assume $a \in R^m, b \in R^n$, operator $'$ means transpose

$$\begin{aligned}
\text{vec}(ab') &= \text{vec} \begin{bmatrix} a_1 b' \\ \vdots \\ a_m b' \end{bmatrix} \\
&= \text{vec} \begin{bmatrix} a_1 b_1 & \cdots & a_1 b_n \\ \vdots & \ddots & \vdots \\ a_m b_1 & \cdots & a_m b_n \end{bmatrix} \\
&= \text{vec} \begin{bmatrix} b_1 a_1 & \cdots & b_n a_1 \\ \vdots & \ddots & \vdots \\ b_1 a_m & \cdots & b_n a_m \end{bmatrix} \\
&= \text{vec} \begin{bmatrix} | & & | \\ b_1 a & \cdots & b_n a \\ | & & | \end{bmatrix} \\
&= \begin{bmatrix} b_1 a \\ \vdots \\ b_n a \end{bmatrix} \\
&= b \otimes a
\end{aligned} \tag{32}$$

$\text{vecABC} = (\mathbf{C}' \otimes \mathbf{A}) \text{vecB}$ (aka **Theorem 2.2** in [8])

First assume $A \in R^{m \times n}, B \in R^{n \times q}, C \in R^{q \times k}$

Thus, B has q columns, and e_j is the j -th standard basis, then (by Strang's 4th MM)

$$B = \sum_{j=1}^q b_j e'_j \tag{33}$$

$$\begin{aligned}
vec(ABC) &\stackrel{\text{equation (33)}}{=} vec(A(\sum_{j=1}^q b_j e'_j)C) \\
&\stackrel{\text{MM is distributive } A(B+C)=AB+AC}{=} vec(\sum_{j=1}^q (Ab_j e'_j)C) \\
&\stackrel{\text{MM is associative}}{=} vec(\sum_{j=1}^q (Ab_j)(e'_j C)) \\
&= vec(\sum_{j=1}^q (Ab_j)(C' e_j)') \\
&\stackrel{\text{Equation (26)}}{=} \sum_{j=1}^q vec((Ab_j)(C' e_j)') \\
&\stackrel{\text{Equation (32)}}{=} \sum_{j=1}^q (C' e_j) \otimes (Ab_j) \\
&\stackrel{(A \otimes B)(C \otimes D) = AC \otimes BD, \text{ see [8] p32 (3)}}{=} \sum_{j=1}^q (C' \otimes A)(e_j \otimes b_j) \\
&\stackrel{C' \otimes A \text{ is const w.r.t } j}{=} (C' \otimes A) \sum_{j=1}^q (e_j \otimes b_j) \\
&\stackrel{\text{Equation (32)}}{=} (C' \otimes A) \sum_{j=1}^q vec(b_j e'_j) \\
&\stackrel{\text{Equation (26)}}{=} (C' \otimes A) vec(\sum_{j=1}^q (b_j e'_j)) \\
&\stackrel{\text{Equation (33)}}{=} (C' \otimes A) vec(B)
\end{aligned} \tag{34}$$

A special case of Equation (34) is

$$\begin{aligned}
vec(AB) &= vec(I_m AB) \\
&\stackrel{\text{Equation (34)}}{=} B' \otimes I_m vec(A)
\end{aligned} \tag{35}$$

2.4 $\frac{\partial Wx}{\partial W}$ See Page 203 [8]

Assume $y \in R^{m \times 1}, x \in R^{n \times 1}, W \in R^{m \times n}$

$$\begin{aligned}
f(W) &= Wx \\
d(f(W)) &= d(Wx) \\
&= d(W)x + Wd(x) \\
&\stackrel{x \text{ is const w.r.t } W}{=} d(W)x + W\vec{0} \\
&= d(W)x \\
d(W)x &\stackrel{\text{is a col vector}^3}{=} \text{vec}(d(W)x) \\
&\stackrel{\text{Equation (35)}}{=} x^T \otimes I_m \text{vec}(d(W)) \\
d\text{vec}U &\stackrel{= \text{vec}dU[8] \text{ p164}}{=} x^T \otimes I_m d(\text{vec}(W))
\end{aligned} \tag{36}$$

Recall in [8] p105, to find the derivatives of a function, we need to be in the format of

$$DF(C) = Df(\text{vec}C) \tag{37}$$

Equation (36) gives precisely this form, so that we know

$$\frac{\partial Wx}{\partial W} = \underbrace{x^T \otimes I_m}_{m \times nm} \tag{38}$$

Note, Equation (38) is the key equation used in Section 2.8.2

2.5 Scalar function of matrix derivative and gradient

Magnus's definition of derivative and gradient, a case study of scalar function of matrix. Suppose $f(X)$ is a mapping from $R^{m \times n} \rightarrow R$. According to Magnus [8],

$$df = D_f d(\text{vec}(X)) \tag{39}$$

⁴vec() on a col vector is the vector itself

then D_f , the derivative, is of the shape $1 \times mn$,

$$D_f = \left[\frac{\partial f}{\partial X_{11}} \cdots \frac{\partial f}{\partial X_{m1}}, \dots, \frac{\partial f}{\partial X_{1n}} \cdots \frac{\partial f}{\partial X_{mn}} \right] \quad (40)$$

$$\nabla_f = \begin{bmatrix} \frac{\partial f}{\partial X_{11}} \\ \vdots \\ \frac{\partial f}{\partial X_{m1}} \\ , \\ \vdots \\ , \\ \frac{\partial f}{\partial X_{1n}} \\ \vdots \\ \frac{\partial f}{\partial X_{mn}} \end{bmatrix} \quad (41)$$

∇_f , the gradient of f , is the transpose of derivative D_f .

Wiki definition of derivative and gradient, a case study of scalar function of matrix. Many (see https://en.wikipedia.org/wiki/Matrix_calculus#Numerator-layout_notation) define D_f as a $n \times m$ matrix, ∇f as a shape $m \times n$ matrix. $\nabla_f = D_f^T$

$$D_f = \begin{bmatrix} \frac{\partial f}{\partial X_{11}} & \cdots & \frac{\partial f}{\partial X_{m1}} \\ & \ddots & \\ \frac{\partial f}{\partial X_{1n}} & \cdots & \frac{\partial f}{\partial X_{mn}} \end{bmatrix} \quad (42)$$

$$\nabla_f = \begin{bmatrix} \frac{\partial f}{\partial X_{11}} & & \frac{\partial f}{\partial X_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial X_{m1}} & & \frac{\partial f}{\partial X_{mn}} \end{bmatrix} \quad (43)$$

Each row i of Equation (42) is $\frac{\partial f}{\partial X_i^T}$ (X_i is the i -th column of X). Each column i is the ∇_f w.r.t the i -th column of X . Basically, this notation treats X as a variable matrix whose each column i is the vector that f takes derivative against.

Wei argues that the two above-mentioned treatments (aka Wiki and Magnus) are consistent with each other In Equation (40), if we do $D_f.\text{reshape}(n,m)$, then we get the D_f defined in Equation (42). Note if we de-vec $_{m \times n}$ ∇_f in Equation (41), then we get ∇_f in Equation (43). In either case, we have

$$\nabla_f = D_f^T \quad (44)$$

In conclusion, we use Magnus rule to get the D_f in a row vector, then we can reshape it to a $n \times m$ D_f matrix used in Stefan's teaching. Also, in Stefan's

teaching, gradient w.r.t to weights always have the same shape as weights. In contrast, in Mangus's teaching, the differential will always have the same shape as the function (output).

2.6 The application of Trace

Most of the time, the loss function is a scalar over trainable parameter X , which is a matrix. Then one can refer to the treatment of trace in [8] page 35 (see Equation (45) below), page 196 and page 197 (Table 9.2 3rd row aka Equation (46) and Equation (47)).

$$\text{tr}(A^T B) = \text{vec}(A)^T \text{vec}(B) \quad (45)$$

$$d\phi(X) = \text{tr}(A^T dX)$$

$$\stackrel{\text{Equation (45)}}{=} (\text{vec}(A))^T d\text{vec}(X)$$

$$\implies D\phi(X) = (\text{vec}A)^T \quad (46)$$

$$\implies \nabla_X \phi_{\text{magnus}} = A \quad (47)$$

For an important application of the cost function is Frobenius norm of a matrix, see the treatment in Section 5.1

2.7 Matrix Differentiation (and some other stuff) by Randal J. Barnes

This section tries to prove [3] by (1) using [5] (thus [8]) and (2) 摆摆好

I only selected a few interesting Propostions to prove

2.7.1 Proposition 6 $y=Ax$

$y \in R^{m \times 1}, x \in R^n, A \in R^{m \times n}$ Then,

$$\frac{\partial y}{\partial x} = A \quad (48)$$

proof 1 by [5] This is a direct application of Eq (15) in [5]. Also, we can use the idea that first-order derivative is just a linear approximation of the function, Ax is a linear operator, as a result it is first-order derivative is precisely A

proof 2 by 摆摆好

$$\begin{aligned}
y_i &= \sum_{k=1}^n A_{i,k} x_k \\
\Rightarrow \frac{\partial y_i}{\partial x_j} &= A_{i,j} \\
\Rightarrow \frac{\partial y_i}{\partial x} &= A_{i,:} \text{ (aka, A's i-th row)} \\
\Rightarrow \frac{\partial y}{\partial x} &= \begin{bmatrix} \text{---} & A_{1,:} & \text{---} \\ & \ddots & \\ \text{---} & A_{m,:} & \text{---} \end{bmatrix} \\
&= A \quad \square
\end{aligned} \tag{49}$$

2.7.2 Proposition 8 $\alpha = x^T A x$

$x \in R^{n \times 1}$, $A \in R^{n \times n}$, and A does not depend on x .

proof 1 by [5]

$$\begin{aligned}
d(\alpha) &\stackrel{[5](4)}{=} d(x^T) A x + x^T d(A x) \\
&\stackrel{5}{=} d(x)^T A x + x^T d(A x) \\
&\stackrel{d(x)^T A x \text{ is a scalar and } [5]15}{=} x^T A^T d(x) + x^T A d(x) \\
&= x^T (A^T + A) d(x)
\end{aligned} \tag{50}$$

Thus,

$$\frac{\partial \alpha}{\partial x} = x^T (A^T + A) \tag{51}$$

Proof 2 by 摆摆好

$$\alpha = \sum_{j=1}^n \sum_{i=1}^n A_{ij} x_i x_j \quad (52)$$

$$\alpha = \underbrace{\sum_{j=1}^{j \neq k, n} \sum_{i=1}^n A_{ij} x_i x_j}_{\alpha 1} + \underbrace{\sum_{i=1}^n A_{ik} x_i x_k}_{\alpha 2} \quad (53)$$

$$\frac{\partial \alpha 1}{\partial x_k} = \sum_{j=1}^{j \neq k, n} A_{kj} x_j \quad (54)$$

$$\alpha 2 = \sum_{i=1}^{i \neq k, n} A_{ik} x_i x_k + A_{kk} x_k^2 \quad (55)$$

$$\frac{\partial \alpha 2}{\partial x_k} = \sum_{i=1}^{i \neq k, n} A_{ik} x_i + 2A_{kk} x_k \quad (56)$$

$$\frac{\partial \alpha}{\partial x_k} = \frac{\partial \alpha 1}{\partial x_k} + \frac{\partial \alpha 2}{\partial x_k} \quad (57)$$

$$= \underbrace{\sum_{j=1}^n A_{kj} x_j}_{\langle A \text{'s } k\text{-th row, } x \rangle} + \underbrace{\sum_{i=1}^n A_{ik} x_i}_{\langle A \text{'s } k\text{-th column, } x \rangle} \quad (58)$$

$$\Rightarrow \frac{\partial \alpha}{\partial x} = \underbrace{(Ax)^T + (A^T x)^T}_{\text{transpose is to make partial derivatives a row vector}} \quad (59)$$

$$= x^T (A^T + A) \quad (60)$$

I feel this proof is the hardest on among all the propositions in [3]

Proof 3 by Automatic-Diff

$$u = x$$

$$y = x$$

$$\mathbf{u}, \mathbf{y} \text{ will contain the partial adjoints w.r.t } \mathbf{x} \quad z = Au$$

$$\alpha = y^T z \quad (61)$$

$$\frac{\partial \alpha}{\partial y} = z^T = u^T A^T \stackrel{u=x}{=} x^T A^T \quad (62)$$

$$\frac{\partial \alpha}{\partial z} = y^T \stackrel{y=x}{=} x^T \quad (63)$$

$$\frac{\partial \alpha}{\partial u} = \frac{\partial \alpha}{\partial z} \frac{\partial z}{\partial u} = x^T A \quad (64)$$

$$\frac{\partial \alpha}{\partial x} \stackrel{\text{adjoint is sum of partial adjoints}}{=} \frac{\partial \alpha}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial \alpha}{\partial y} \frac{\partial y}{\partial x} \quad (65)$$

$$= x^T A + x^T A^T \quad (66)$$

$$= x^T (A + A^T) \quad \square \quad (67)$$

2.7.3 Proposition 10 $\alpha = y(z)^T x(z)$

I feel this Proposition is the most important one in [3]

Let the scalar α be defined by $\alpha = y^T x$, where $y \in R^{n \times 1}, x \in R^{n \times 1}$, and both y and x are functions of the vector $z \in R^{m \times 1}$. Then

$$\frac{\partial \alpha}{\partial z} = x^T \frac{\partial y}{\partial z} + y^T \frac{\partial x}{\partial z} \quad (68)$$

proof 1 by [5]

$$d(\alpha) \stackrel{[5](4)}{=} d(y^T)x + y^T d(x) \quad (69)$$

$$d(y^T)x \text{ is a scalar and } \stackrel{5}{=} x^T d(y) + y^T d(x) \quad (70)$$

$$\stackrel{\text{chain rule}}{=} x^T \frac{\partial y}{\partial z} + y^T \frac{\partial x}{\partial z} \quad \square \quad (71)$$

proof 2 by 摆摆好

$$\begin{aligned}
\alpha &= \sum_{j=1}^n x_j y_j & (72) \\
\frac{\partial \alpha}{\partial z_k} &\stackrel{\text{chain rule}}{=} \sum_{j=1}^n \left(\frac{\partial x_j}{\partial z_k} y_j + \frac{\partial y_j}{\partial z_k} x_j \right) \\
&= \sum_{j=1}^n \frac{\partial x_j}{\partial z_k} y_j + \sum_{j=1}^n \frac{\partial y_j}{\partial z_k} x_j \\
&\quad D_{x|z} \text{ is the Jacobian of } \mathbf{x}(z) \\
&\quad D_{y|z} \text{ is the Jacobian of } \mathbf{y}(z) \\
&\quad = \langle D_{x|z} \text{'s } j\text{-th col}, y \rangle + \langle D_{y|z} \text{'s } j\text{-th col}, x \rangle \\
&= y^T \frac{\partial x}{\partial z} \underset{:,j}{=} + x^T \frac{\partial y}{\partial z} \underset{:,j}{=} \\
&\implies \\
\frac{\partial \alpha}{\partial z} &= y^T \frac{\partial x}{\partial z} + x^T \frac{\partial y}{\partial z} \quad \square & (73)
\end{aligned}$$

Proof 3 by Auto-diff

$$u = y(z) \tag{74}$$

$$v = x(z) \tag{75}$$

u,v will contain the partial adjoints w.r.t z

$$\alpha = y(z)^T x(z) = u^T v \quad (76)$$

$$\frac{\partial \alpha}{\partial u} = v^T \quad (77)$$

$$\frac{\partial \alpha}{\partial v} = u^T \quad (78)$$

$$\frac{\partial \alpha}{z} \text{ adjoint is } \sum (\text{partial adjoints}) \frac{\partial \alpha}{\partial u} \frac{\partial u}{\partial z} + \frac{\partial \alpha}{\partial v} \frac{\partial v}{\partial z} \quad (79)$$

$$= v^T \frac{\partial u}{\partial z} + u^T \frac{\partial v}{\partial z} \quad (80)$$

$$= x(z)^T \frac{\partial y(z)}{\partial z} + y(z)^T \frac{\partial x(z)}{\partial z} \quad \square \quad (81)$$

2.8 Multi-Layer-Perceptron Building Blocks

This section takes most of the contents from Stefan’s online teaching[6], re-organized to follow an easier (for me) flow.

2.8.1 Square Loss Layer

$$E = \frac{1}{2}(y - t)^T(y - t) \quad (82)$$

Equation (82) defines the square loss, $y \in R^m$, $t \in R^m$, t is a target vector, which is a constant. As a syntactic sugar, we let $r = y - t$

$$dE = d\left(\frac{1}{2}(y - t)^T(y - t)\right) \quad (83)$$

$$\stackrel{d(UV)=dU\dot{V}+UdV}{=} \frac{1}{2}(d((y - t)^T)(y - t) + (y - t)^T d(y - t)) \quad (84)$$

$$\stackrel{d(U^T)=dU^T}{=} \frac{1}{2}((d(y - t))^T(y - t) + (y - t)^T d(y - t)) \quad (85)$$

$$\stackrel{(y-t)^T(y-t) \text{ is a scalar}}{=} \frac{1}{2}((y - t)^T d(y - t) + (y - t)^T d(y - t)) \quad (86)$$

$$= (y - t)^T d(y - t) \quad (87)$$

$$\stackrel{t \text{ is constant}}{=} (y - t)^T dy \quad (88)$$

$$\stackrel{r=y-t}{=} r^T dy \quad (89)$$

From Equation (89), we can get the following

$$D_E = r^T \quad (90)$$

$$\nabla_E = D_E^T = r \quad (91)$$

$$(92)$$

2.8.2 Linear Layer

$$y = Wx + b \quad (93)$$

$W \in R^{m \times n}$ is weight matrix, $x \in R^n$ is input, $b \in R^m$ is bias and $y \in R^m$ is output.

Derivatives w.r.t W

$$dy = d(Wx + b) \quad (94)$$

$$\Rightarrow dy|_W \stackrel{\text{b is constant w.r.t } W}{=} d(Wx) \quad (95)$$

$$\stackrel{\text{Equation (38)}}{=} x^T \bigotimes I_m dW \quad (96)$$

$$\Rightarrow \frac{\partial y}{\partial W} = x^T \bigotimes I_m \quad (97)$$

$$\Rightarrow \frac{\partial E}{\partial W} \stackrel{\text{chain rule}}{=} \frac{\partial E}{\partial y} \frac{\partial y}{\partial W} \quad (98)$$

$$\stackrel{\text{Equation (97)}}{=} \frac{\partial E}{\partial y} x^T \bigotimes I_m \quad (99)$$

$$= \frac{\partial E}{\partial y} [x_1 I_m, \dots, x_n I_m] \quad (100)$$

$$\stackrel{\frac{\partial E}{\partial y} \in \mathbb{R}^{1 \times m}}{=} \underbrace{\left[x_1 \frac{\partial E}{\partial y}, \dots, x_n \frac{\partial E}{\partial y} \right]}_{m \times n \text{ length row vector}} \quad (101)$$

Following the last paragraph discussion in Section 2.5, we can reshape Equation (101) to a $n \times m$ matrix, thus

$$\frac{\partial E}{\partial W} = \begin{bmatrix} x_1 \frac{\partial E}{\partial y} \\ \vdots \\ x_n \frac{\partial E}{\partial y} \end{bmatrix} \quad (102)$$

$$= \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \left[-\frac{\partial E}{\partial y} \right] \quad (103)$$

$$= x \frac{\partial E}{\partial y} \quad (104)$$

$$\nabla E|_W \stackrel{\text{Equation (44)}}{=} \left(\frac{\partial E}{\partial W} \right)^T \quad (105)$$

$$= \left(\frac{\partial E}{\partial y} \right)^T x^T \quad (106)$$

$$\stackrel{\text{Equation (44)}}{=} \nabla E|_y x^T \quad (107)$$

It is easy to see that cost function derivatives/gradients w.r.t W is a rank-one matrix, someone probably has taken advantage of this effect (i.e., to check [9]). To conclude,

$$\frac{\partial E}{\partial W} = x \frac{\partial E}{\partial y} \quad (108)$$

$$\nabla E|_W = \nabla E|_y x^T \quad (109)$$

I think Equation (109) is the single most important equation for DL

Next we show using 摆摆好to prove Equation (108) and Equation (109).

摆摆好method 1 We use W_i to represent the i -th row of W , in the following we treat W_i as a column vector.

$$\begin{aligned}
W_i^T x &= y_i \\
y_i \text{ is scalar} &\implies y_i = x^T W_i \\
&\implies \frac{\partial y_i}{\partial W_i} \stackrel{\text{Equation (48)}}{=} x^T \\
\frac{\partial E}{\partial W_i} &= \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial W_i} \\
&= \frac{\partial E}{\partial y_i} x^T
\end{aligned} \tag{110}$$

$$\text{The key step} \implies \begin{bmatrix} \frac{\partial E}{\partial W_1} \\ \vdots \\ \frac{\partial E}{\partial W_m} \end{bmatrix} \stackrel{\text{Equation (110)}}{=} \begin{bmatrix} \frac{\partial E}{\partial y_1} x^T \\ \vdots \\ \frac{\partial E}{\partial y_m} x^T \end{bmatrix} \tag{111}$$

The LHS of Equation (111) is $\nabla E|_W$, the RHS of Equation (111) is $\nabla E|_y x^T$.
As a result, $\nabla E|_W = \nabla E|_y x^T$ $\square$

摆摆好method 2 We can also use

$$\frac{\partial y_i}{\partial W_{i,j}} = x_j \tag{112}$$

$$\begin{aligned}
\frac{\partial E}{\partial W_{i,j}} &= \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial W_{i,j}} \\
&\stackrel{\text{Equation (112)}}{=} \frac{\partial E}{\partial y_i} x_j
\end{aligned} \tag{113}$$

$$\begin{bmatrix} \frac{\partial E}{\partial W_{1,1}} & \cdots & \frac{\partial E}{\partial W_{1,n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial W_{m,1}} & \cdots & \frac{\partial E}{\partial W_{m,n}} \end{bmatrix} \stackrel{\text{Equation (113)}}{=} \begin{bmatrix} \frac{\partial E}{\partial y_1} x_1 & \cdots & \frac{\partial E}{\partial y_1} x_n \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial y_m} x_1 & \cdots & \frac{\partial E}{\partial y_m} x_n \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial y_1} x \\ \vdots \\ \frac{\partial E}{\partial y_m} x \end{bmatrix} \tag{114}$$

The LHS of Equation (114) is $\nabla E|_W$, the RHS of Equation (114) is $\nabla E|_y x^T$.
As a result, $\nabla E|_W = \nabla E|_y x^T$ $\square$

Derivatives w.r.t b

$$dy = d(Wx + b) \quad (115)$$

$$\Rightarrow dy|_b \stackrel{Wx \text{ is const w.r.t } b}{=} d(b) \quad (116)$$

$$= I_m db \quad (117)$$

$$\Rightarrow \frac{\partial y}{\partial b} = I_m \quad (118)$$

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial y} I_m \quad (119)$$

$$= \frac{\partial E}{\partial y} \quad (120)$$

To conclude,

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial y} \quad (121)$$

$$\nabla_b E = \nabla_y E \quad (122)$$

Derivatives w.r.t x

$$dy = d(Wx + b) \quad (123)$$

$$\Rightarrow dy|_x \stackrel{b \text{ is const w.r.t } x}{=} d(Wx) \quad (124)$$

$$= W dx \quad (125)$$

$$\Rightarrow \frac{\partial y}{\partial x} = W \quad (126)$$

$$\Rightarrow \frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} W \quad (127)$$

To conclude,

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} W \quad (128)$$

$$\nabla_x E = W^T \nabla_y E \quad (129)$$

2.8.3 Component-wise Activation Layer (e.g., sigmoid, ReLU, Tahn)

Suppose, $a, z \in R^n$, σ is a component-wise activation function(e.g., sigmoid, tahn). Such functions are component-wise, i.e., the output variable only depends on the input variable, but not the entire input vector as in the softmax case. Note we use $\sigma'(z)$ to represent the vector whose i -th component is $\frac{\partial \sigma(z_i)}{\partial z_i}$ (see

Equation (130))

$$\sigma'(z) \equiv \begin{bmatrix} \frac{\partial \sigma(z_1)}{\partial z_1} \\ \vdots \\ \frac{\partial \sigma(z_1)}{\partial z_n} \end{bmatrix} \quad (130)$$

$$a = \sigma(z) \quad (131)$$

$$\frac{\partial a}{\partial z} = \begin{bmatrix} \frac{\partial \sigma}{\partial z_1} & & \\ & \ddots & \\ & & \frac{\partial \sigma}{\partial z_n} \end{bmatrix} \quad (132)$$

$$\frac{\partial E}{\partial z} = \frac{\partial E}{\partial a} \begin{bmatrix} \frac{\partial \sigma}{\partial z_1} & & \\ & \ddots & \\ & & \frac{\partial \sigma}{\partial z_n} \end{bmatrix} \quad (133)$$

$$\stackrel{\text{Equation (24)}}{=} \frac{\partial E}{\partial a} \odot (\sigma'(z))^T \quad (134)$$

To conclude,⁵,

$$\frac{\partial E}{\partial z} = \frac{\partial E}{\partial a} \odot (\sigma'(z))^T \quad (135)$$

TODO: add tahn, sigmoid derivative derivation. sigmoid should have the same form $a(1-a)$ as the softmax $\frac{\partial a_i}{\partial z_i}$

2.9 Activation function and Loss function

2.9.1 Softmax activation function

Suppose $z \in R^m, a \in R^m$

$$a_i = \frac{e^{z_i}}{\sum_{k=1}^m e^{z_k}} \quad (136)$$

$$\frac{\partial a_i}{\partial z_i} = \frac{e^{z_i} \sum_{k=1}^{k=m} e^{z_k} - (e^{z_i})^2}{(\sum_{k=1}^{k=m} e^{z_k})^2} = \frac{e^{z_i}}{\sum_{k=1}^{k=m} e^{z_k}} - \left(\frac{e^{z_i}}{\sum_{k=1}^{k=m} e^{z_k}} \right)^2 = a_i - (a_i)^2 = a_i(1 - a_i) \quad (137)$$

$$\frac{\partial a_i}{\partial z_j} = \frac{-e^{z_i} e^{z_j}}{(\sum_{k=1}^{k=m} e^{z_k})^2} = -a_i a_j \quad (138)$$

⁵ means take derivative component-wise for this column vector

As a result, we can derive the Jacobian of softmax activation function as

$$\frac{\partial a}{\partial z} = \begin{bmatrix} a_1(1-a_1) & -a_1a_2 & \dots & -a_1a_m \\ -a_1a_2 & a_2(1-a_2) & \dots & -a_2a_m \\ \vdots & \vdots & \ddots & \vdots \\ -a_1a_m & \dots & \dots & a_m(1-a_m) \end{bmatrix} \quad (139)$$

$$= \begin{bmatrix} a_1 & a_1 & \dots & a_1 \\ a_2 & a_2 & \dots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_m & a_m & a_m & a_m \end{bmatrix} \odot \begin{bmatrix} 1-a_1 & -a_2 & \dots & -a_m \\ -a_1 & (1-a_2) & \dots & -a_m \\ \dots & \dots & \dots & \dots \\ -a_1 & -a_2 & \dots & (1-a_m) \end{bmatrix} \quad (140)$$

$$= \begin{bmatrix} a_1 & a_1 & \dots & a_1 \\ a_2 & a_2 & \dots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_m & a_m & a_m & a_m \end{bmatrix} \odot (I_m - \begin{bmatrix} a_1 & a_2 & \dots & a_m \\ a_1 & a_2 & \dots & a_m \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \dots & a_m \end{bmatrix}) \quad (141)$$

$$\begin{aligned} \odot \text{ is distributive} \\ \equiv \begin{bmatrix} a_1 & a_1 & \dots & a_1 \\ a_2 & a_2 & \dots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_m & a_m & a_m & a_m \end{bmatrix} \odot I_m - \begin{bmatrix} a_1 & a_1 & \dots & a_1 \\ a_2 & a_2 & \dots & a_2 \\ \vdots & \vdots & \ddots & \vdots \\ a_m & a_m & a_m & a_m \end{bmatrix} \odot \begin{bmatrix} a_1 & a_2 & \dots & a_m \\ a_1 & a_2 & \dots & a_m \\ \vdots & \vdots & \ddots & \vdots \\ a_1 & a_2 & \dots & a_m \end{bmatrix} \end{aligned} \quad (142)$$

$$= \text{Diag}(a) - aa^T \quad \# \text{Diag}(a) \text{ is to create a diagonal matrix whose diagonal is } a \quad (143)$$

To conclude,

$$\frac{\partial a}{\partial z} = \text{Diag}(a) - aa^T \quad (144)$$

In numpy jargon,

```
def jacobian_softmax(a, z):
'''
a: mx1 vector, the output of softmax function
z: mx1 vector, the input to softmax function (aka, logits)
return the Jacobian of softmax
'''
    return np.diag(a.flatten())-a@a.transpose() % alternaviely,
        ↳ one can also use a*a.transpose(), as it is a vector
        ↳ outer product
```

2.9.2 Cross Entropy Loss

Suppose $a \in R^m, y \in R^m$, further, y is a one-hot encoding. Let assume k is the target class, i.e.,

$$y = \begin{bmatrix} 0 \\ \vdots \\ y_k = 1 \\ \vdots \\ 0 \end{bmatrix} \quad (145)$$

Kolter uses e_k (aka, the standard basis with the k-th dimension is selected) to represent the same y in Equation (145), in lecture 2 of [4].

$$E = -y^T \log^a \quad (146)$$

$$dE = d(-y^T \log^a) \quad (147)$$

$$= -y^T d(\log^a) \quad (148)$$

$$= -y^T \begin{bmatrix} \frac{1}{a_1} & & & \\ & \ddots & & \\ & & \frac{1}{a_m} & \\ & & & \end{bmatrix} da \quad (149)$$

$$= \underbrace{[0 \dots, -\frac{1}{a_k}, \dots 0]}_m da \quad (150)$$

To conclude,

$$\frac{\partial E}{\partial a} = \underbrace{[0 \dots, -\frac{1}{a_k}, \dots 0]}_m \quad (151)$$

2.9.3 Cross Entropy Loss w.r.t logits

$$E = \text{cross_entropy}(\text{softmax}(z)) \quad (152)$$

$$\frac{\partial E}{\partial z} = \frac{\partial E}{\partial a} \frac{\partial a}{\partial z} \quad (153)$$

$$= \underbrace{[0 \dots, -\frac{1}{a_k}, \dots 0]}_m (\text{Diag}(a) - aa^T) \quad (154)$$

Use row space concept,
only the k-th row of
 $\text{Diag}(a) - aa^T$ is selected $\frac{-1}{a_k}(-a_k) [a_1, a_2, \dots a_k - 1 \dots a_m]$ (155)

$$= [a_1, a_2, \dots a_k - 1 \dots a_m] \quad (156)$$

$$= (a - y)^T \quad (157)$$

To conclude,

$$\frac{\partial E}{\partial z} = (a - y)^T \quad (158)$$

2.9.4 Cross Entropy vs Square Loss

Cross Entropy

$$a = \text{soft_max}(z) \quad (159)$$

$$E = \text{cross_entropy}(a) \quad (160)$$

$$y \text{ is the one-hot encoding, aka target} \quad (161)$$

$$\frac{\partial E}{\partial z} \stackrel{\text{Equation (158)}}{=} (a - y)^T \quad (162)$$

Square Loss

$$a = \text{ID}(z) \quad (163)$$

$$E = \text{square_loss}(a) \quad (164)$$

$$y \text{ is the target} \quad (165)$$

$$\frac{\partial E}{\partial z} \stackrel{\text{ID is the activation}}{=} \frac{\partial E}{\partial a} \stackrel{\text{Equation (90)}}{=} (a - y)^T \quad (166)$$

If we use ID as the activation function in square loss and soft max as the activation function in cross entropy, then square loss and cross entropy loss have the same formula for the $\frac{\partial E}{\partial z}$, where E is the loss and z is the logits. (TODO, maybe i should draw the block diagram)

2.10 Logistic Regression (aka multi-class classification)

This is a use case of the combining several building blocks.

2.10.1 Fundamentals (One sample at a time)

Figure 1 illustrates logistic regression. It consists of a linear layer, followed by softmax and cross-entropy layer. $x \in R^n, y \text{ (target)} \in R^m, b \in R^m, W \in R^{m \times n}, z \in R^m, a \in R^m, E \in R$

$$z = Wx + b \quad (167)$$

$$a = \text{softmax}(z) \quad (168)$$

$$E = \text{cross_entropy}(a, y) \quad (169)$$

$$E = \text{cross_entropy}(\text{soft_max}(Wx + b)) \quad (170)$$

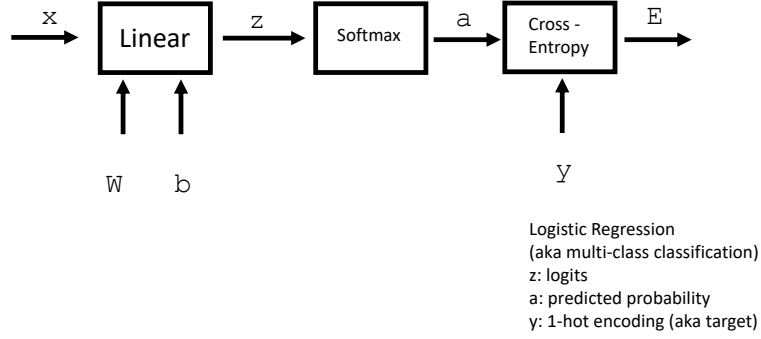


Figure 1: Logistic Regression (aka multi-class classification). z is the logits, a is the model predicted probability vector. In cross-entropy, one needs to take negative log of a before inner producing with the one-hot encoding y

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial z} \frac{\partial z}{\partial b} \quad (171)$$

$$\stackrel{\text{Equation (121)}}{=} \frac{\partial E}{\partial z} \quad (172)$$

$$\stackrel{\text{Equation (158)}}{=} (a - y)^T \quad (173)$$

$$\frac{\partial E}{\partial W} \stackrel{\text{Equation (108)}}{=} x \frac{\partial E}{\partial z} \quad (174)$$

$$\stackrel{\text{Equation (108)}}{=} x(a - y)^T \quad (175)$$

Equation (175) checks with Lecture 2 of [4], and I am using a much more principled method here.

2.10.2 Multiple samples at a time (mini-batch, aka vectorization)

$X \in R^{n \times mb}$, mb is the mini-batch size, Y (target) $\in R^{m \times mb}$, $b \in R^m$, $W \in R^{m \times n}$, $Z \in R^{m \times mb}$, $A \in R^{m \times mb}$, $E \in R$

$$Z = WX + b \underbrace{[1, \dots, 1]}_{mb} \quad (176)$$

$$A = \text{soft_max}(Z, \text{dim} = 0) \quad (177)$$

$$E = \text{cross_entropy}(A, Y) \quad (178)$$

Note the *soft_max* here treats every column as logits. In PyTorch jargon, it is *soft_max*($Z, \text{dim}=0$). Similarly, *cross_entropy* also treats each column of A as predicted probability and corresponding column of Y as target 1-hot encoding – it is not going to be compatible with PyTorch’s *CrossEntropyLoss*, as PyTorch would require the inputs in a row-major format. But our math derivation has nothing to do with this format, as it based purely on basis, linear algebra and matrix differential calculus.

Recall in the above section,

$$\frac{\partial E}{\partial b} = (a - y)^T \quad (179)$$

When using mini-batch, we would like to get

$$\frac{\partial E}{\partial b} = \sum_{i=1}^{mb} (A_i - Y_i)^T \quad (180)$$

$$\text{Strang 3rd way of MatMul} \quad \underline{\underline{=}} \quad (1_m)^T (A - Y)^T \quad (181)$$

Similarly, by using Strang’s 4-th way of calculating matrix multiplication, and that would be a mb rank-one matrices addition into the derivative.

$$\frac{\partial E}{\partial W} = X(A - Y)^T \quad (182)$$

TODO: Maybe I should try use Magnus’s method to derive these, going to be long and fun.

2.10.3 Use row vectors to represent data

Subscript $_r$ stands for row, it is the transpose of all the counterparts in Section 2.10.2, and they represent the exactly same data, model parameters and optimization mechanism, the only difference is the layout of the data. $X_r \in R^{mb \times n}$, mb is the mini-batch size, Y (target) $\in R^{mb \times m}$, $b_r \in R^{1 \times m}$, $W_r \in R^{n \times m}$, $Z_r \in R^{mb \times m}$, $A_r \in R^{mb \times m}$, $E \in R$

$$Z_r = X_r W_r + \underbrace{\begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}}_{mb \times 1} b_r \quad (183)$$

$$A_r = \text{soft_max}(Z_r, \text{dim} = 1) \quad (184)$$

$$E = \text{cross_entropy}(A_r, Y_r) \quad (185)$$

Since it is the same data, model and optimization, the only thing changed was the shape. We leverage the fact that we proved in Section 2.5 that scalar function w.r.t matrix gradient can have the same shape as the matrix and still be consistent with Magnus book's differential theory. E.g., $\nabla_W E_{i,j} = \frac{\partial E}{\partial W_{i,j}}$. In another word, the derivatives in the column-major representation are exactly the gradients in the row-major representation, and we only need to make sure we use the data in the right format by using equivalence such as $X_r = X^T$. So we have

$$\nabla_{b_r} E = \left(\frac{\partial E}{\partial b_r} \right)^T \quad (186)$$

$$\stackrel{\text{re-arrangement}}{=} \frac{\partial E}{\partial (b_r)^T} \quad (187)$$

$$= \frac{\partial E}{\partial b} \quad (188)$$

$$\stackrel{\text{Equation (181)}}{=} (1_m)^T (A - Y)^T \quad (189)$$

$$= (1_m)^T (A^T - Y^T) \quad (190)$$

$$= (1_m)^T (A_r - Y_r) \quad (191)$$

Similary, we have

$$\nabla_{W_r} E = \left(\frac{\partial E}{\partial W_r} \right)^T \quad (192)$$

$$\stackrel{\text{rearrangement}}{=} \frac{\partial E}{\partial (W_r)^T} \quad (193)$$

$$= \frac{\partial E}{\partial W} \quad (194)$$

$$\stackrel{\text{Equation (182)}}{=} X(A - Y)^T \quad (195)$$

$$= X_r^T (A^T - Y^T) \quad (196)$$

$$= X_r^T (A_r - Y_r) \quad (197)$$

Equation (197) checks with [4] lecture 2 page 19 $\square$.

Note, this section deals with logistic regression in the row-major format at a rather intuitive level (but sufficiently rigorous), see Section 2.11.3 for the more formal treatment on this matter: the notation difference is that in Section 2.11.3, we use math bold font to represent $_r$ data/parameter matrix of this section.

2.11 Derivation of Nielson's 4 BP equations, vectorization and row-major

2.11.1 One sample at a time (the original 4 BP equations)

In Chapter 2 of [10], there are 4 BP equations:

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \quad (\text{BP1})$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad (\text{BP2})$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \quad (\text{BP4})$$

- L : number of layers in NN, including input layer.
- l : layer index, $l \in 2, \dots, L$. The first layer is always input layer.
- $\nabla_a C$: a vector whose componets are the partial derivatives $\frac{\partial C}{\partial a_j^L}$. C is the cost function.
- w_{jk}^l : weight for the connection from the k -th neuron in the $(l-1)$ -th layer to the j -th neuron in the l -th layer. $l \in 2, \dots, L$. Note w^1 is the input layer weights, which is nothing but an identity matrix, thus being ignored. Figure 2 depicts an example of weight matrix.
- z^l : $z^l \equiv w^l a^{l-1} + b^l$, so-called "weighted input to the neurons in layer l ". z^l is a column vector
- a^l : $\sigma(z^l)$, activation layer, the output of layer l , and the input to the layer $l+1$. a^1 is the input layer input and a^L is the output layer output. σ is the activation function of layer l . a^l is column vector.
- δ^l : $\delta_j^l \equiv \frac{\partial C}{\partial z_j^l}$ δ^l is a column vector. $\delta^l = (\frac{\partial C}{\partial z^l})^T k$

Figure 3 depicts the relationship between all the above-mentioned definitions, δ^l (aka $(\frac{\partial C}{\partial z^l})^T$) is the key in BP.

Derivation of BP1

$$\frac{\partial C}{\partial z^L} = \frac{\partial C}{\partial a^L} \frac{\partial a^L}{\partial z^L} \quad (198)$$

$$\stackrel{\text{assume } \sigma \text{ is component-wise}}{=} \frac{\partial C}{\partial a^L} \text{Diag}(\sigma'(z^L)) \quad (199)$$

$$\implies \left(\frac{\partial C}{\partial z^L}\right)^T = \text{Diag}(\sigma'(z^L)) \left(\frac{\partial C}{\partial a^L}\right)^T \quad (200)$$

$$\implies \delta^L \stackrel{\text{Equation (23)}}{=} \sigma'(z^L) \odot \nabla_{a^L} C \quad (201)$$

$$\stackrel{\substack{\nabla_a C \equiv \nabla_{a^L} C \\ \odot \text{ is commutative}}}{=} \nabla_a C \odot \sigma'(z^L) \quad \square \quad (202)$$

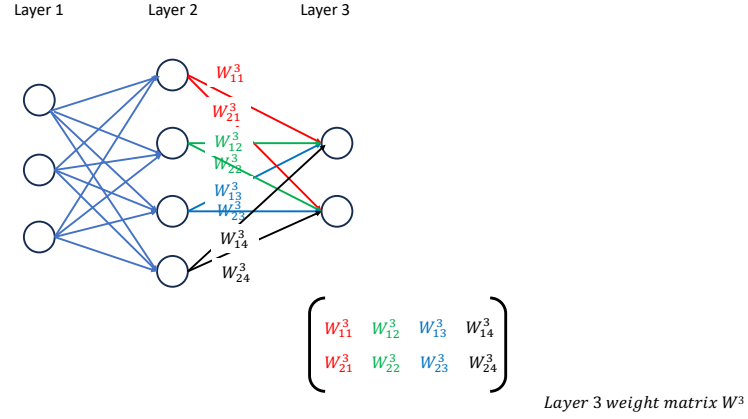


Figure 2: Weight matrix and NN connections

Note this derivation assumes that the last layer activation function (before cost function) is a component-wise function, which means each j -th neuron output a_j^L only depends on weighted input z_j^L . This is the case of sigmoid, but not softmax. But as we can see, this equation is not difficult even for special case where the last layer activation function is softmax, and the cost function is cross entropy, as we have done exactly so in Equation (158)

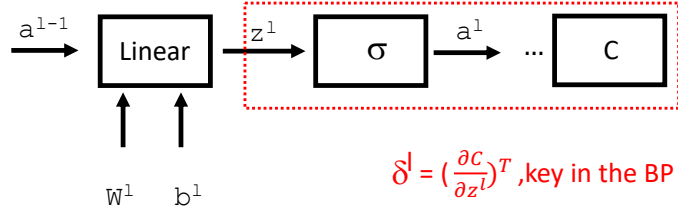


Figure 3: Nielson book One Layer, δ^l (aka $(\frac{\partial C}{\partial z^l})^T$) is the key in BP

Derviation of BP2

$$\frac{\partial C}{\partial z^l} = \frac{\partial C}{\partial z^{l+1}} \frac{\partial z^{l+1}}{\partial a^l} \frac{\partial a^l}{\partial z^l} \quad (203)$$

$$\stackrel{\text{Equation (126)}}{=} \frac{\partial C}{\partial z^{l+1}} W^{l+1} \frac{\partial a^l}{\partial z^l} \quad (204)$$

$$\stackrel{\text{Equation (132)}}{=} \frac{\partial C}{\partial z^{l+1}} W^{l+1} \begin{bmatrix} \frac{\partial \sigma}{\partial z_1^l} \\ \vdots \\ \frac{\partial \sigma}{\partial z_n^l} \end{bmatrix} \quad (205)$$

$$\stackrel{\text{Equation (24)}}{=} \frac{\partial C}{\partial z^{l+1}} W^{l+1} \odot (\sigma'(z^l))^T \quad (206)$$

$$\Rightarrow \left(\frac{\partial C}{\partial z^l}\right)^T = \begin{bmatrix} \frac{\partial \sigma}{\partial z_1^l} \\ \vdots \\ \frac{\partial \sigma}{\partial z_n^l} \end{bmatrix} (W^{l+1})^T \left(\frac{\partial C}{\partial z^{l+1}}\right)^T \quad (207)$$

$$\stackrel{\text{Equation (23)}}{=} \sigma'(z^l) \odot ((W^{l+1})^T \delta^{l+1}) \quad (208)$$

$$\stackrel{\odot \text{ is commutative}}{=} ((W^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad (209)$$

$$\Rightarrow \delta^l = ((W^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad \square \quad (210)$$

Derivation of BP3

$$\frac{\partial C}{\partial b^l} = \frac{\partial C}{\partial z^l} \frac{\partial z^l}{\partial b^l} \quad (211)$$

$$\stackrel{\text{Equation (121)}}{=} \frac{\partial C}{\partial z^l} \quad (212)$$

$$\Rightarrow \nabla_{b^l} C = \delta^l \quad \square \quad (213)$$

Derivation of BP4

$$\frac{\partial C}{\partial w^l} \stackrel{\text{Equation (108)}}{=} a^{l-1} \frac{\partial C}{\partial z^l} \quad (214)$$

$$\Rightarrow \nabla_{w^l} C \stackrel{\text{Or directly by Equation (109)}}{=} \delta^l (a^{l-1})^T \quad (215)$$

$$\Rightarrow \frac{\partial C}{\partial w_{jk}^l} = \delta_j^l a_k^{l-1} \quad \square \quad (216)$$

Note, follow the discussion in Section 2.5, gradients w.r.t weights (in Stefan's teaching) could have the same shape as weights. That is how I get the per-element derivative expression the same as Nielson – even though he achieved his conclusion in the reverse order (aka get the per-element expression first, then get the vectorization format). I feel my approach is much cleaner and faster.

2.11.2 Vectorization (multiple samples at a time)

Weapons at disposal:

- Strang's 4 ways of matrix multiplication, especially 3rd way (rows multiply matrix) and 4th way (columns multiply rows)
- Equation (23) and Equation (24) for hadamard product and diagonal matrix
- knowledge summarized in Section 2.5

Note in this section, we deliberately use derivatives not gradients as written in [10] as we are going to write in the gradients form in the next section (aka row-major) which turns out to be the derivative in the column-major form.

We use mb to represent mini-batch size. In the multi-example setup, we use C_k to represent the cost w.r.t the k -th example. Then the total cost is

$$C = \sum_{k=1}^{mb} C_k \quad (217)$$

Since differentiation is a linear transformation, we have

BP1-vec $\frac{\partial C}{\partial Z^L}$ For the ease of discussion, we assume the loss function to be always softmax followed by cross-entropy. Thus we have

$$\frac{\partial C_k}{\partial Z_k^L} \stackrel{\text{Equation (158)}}{=} (A_k^L - Y_k)^T \quad (218)$$

Furthermore, since C_k only depends on Z_k^L among all the elements in Z^L , we have

$$\frac{\partial C_k}{\partial Z^L} = \begin{bmatrix} \dots 0 \dots \\ \vdots \\ \frac{\partial C_k}{\partial Z_k^L} \\ \vdots \\ \dots 0 \dots \end{bmatrix} \quad (219)$$

$$\frac{\partial C}{\partial Z^L} \stackrel{\text{differentiation is linear}}{=} \sum_{k=1}^{mb} \frac{\partial C_k}{\partial Z^L} \quad (220)$$

$$\stackrel{\text{Equation (219)}}{=} \begin{bmatrix} \frac{\partial C_1}{\partial Z_1^L} \\ \vdots \\ \dots 0 \dots \end{bmatrix} + \dots + \begin{bmatrix} \dots 0 \dots \\ \frac{\partial C_k}{\partial Z_k^L} \\ \dots 0 \dots \end{bmatrix} + \dots + \begin{bmatrix} \dots 0 \dots \\ \vdots \\ \frac{\partial C_{mb}}{\partial Z_{mb}^L} \end{bmatrix} \quad (221)$$

$$= \begin{bmatrix} \frac{\partial C_1}{\partial Z_1^L} \\ \vdots \\ \frac{\partial C_{mb}}{\partial Z_{mb}^L} \end{bmatrix} \quad (222)$$

$$= \begin{bmatrix} \frac{\partial C_1}{\partial Z_1^L} (A_1^L - Y_1)^T \\ \vdots \\ \frac{\partial C_{mb}}{\partial Z_{mb}^L} (A_{mb}^L - Y_{mb})^T \end{bmatrix} \quad (223)$$

$$= (A^L - Y)^T \quad (224)$$

BP2-vec $\frac{\partial C}{\partial Z^l}$ Recall

$$\frac{\partial C_k}{\partial Z_k^l} \stackrel{\text{Equation (206)}}{=} \frac{\partial C_k}{\partial Z_k^{l+1}} W^{l+1} \odot (\sigma'(Z_k^l))^T \quad (225)$$

Then,

$$\frac{\partial C}{\partial Z^l} \stackrel{\text{similar to Equation (222)}}{=} \begin{bmatrix} -\frac{\partial C_1}{\partial Z_1^l} \\ \vdots \\ -\frac{\partial C_{mb}}{\partial Z_{mb}^l} \end{bmatrix} \quad \begin{array}{l} \text{This is a very important definition} \\ \text{will be used extensively} \\ \text{in the following BP equations} \end{array} \quad (226)$$

$$\stackrel{\text{Equation (225)}}{=} \begin{bmatrix} -\frac{\partial C_1}{Z_1^{l+1}} W^{l+1} \odot (\sigma'(Z_1^l))^T \\ \vdots \\ -\frac{\partial C_{mb}}{Z_{mb}^{l+1}} W^{l+1} \odot (\sigma'(Z_{mb}^l))^T \end{bmatrix} \quad (227)$$

$$\stackrel{\text{Strang's 3rd MM}}{=} \begin{bmatrix} -\frac{\partial C_1}{\partial Z_1^{l+1}} \\ \vdots \\ -\frac{\partial C_{mb}}{\partial Z_{mb}^{l+1}} \end{bmatrix} W^{l+1} \odot (\sigma'(Z^l))^T \quad (228)$$

$$\stackrel{\text{Equation (226)}}{=} \frac{\partial C}{\partial Z^{l+1}} W^{l+1} \odot (\sigma'(Z^l))^T \quad (229)$$

BP3-vec $\frac{\partial C}{\partial b^l}$ Recall

$$\frac{\partial C_k}{\partial b^l} \stackrel{\text{Equation (212)}}{=} \frac{\partial C_k}{\partial Z_k^l} \quad (230)$$

for a single example.

$$\frac{\partial C}{\partial b^l} = \sum_{k=1}^{mb} \frac{\partial C_k}{\partial b^l} \quad (231)$$

$$\stackrel{\text{Equation (230)}}{=} \sum_{k=1}^{mb} \frac{\partial C_k}{\partial Z_k^l} \quad (232)$$

$$\stackrel{\text{Strang's 3rd way of MM}}{=} \underbrace{\begin{bmatrix} 1, \dots, 1 \end{bmatrix}}_{mb} \begin{bmatrix} -\frac{\partial C_1}{\partial Z_1^l} \\ \vdots \\ -\frac{\partial C_{mb}}{\partial Z_{mb}^l} \end{bmatrix} \quad (233)$$

$$\stackrel{\text{Equation (226)}}{=} \underbrace{\begin{bmatrix} 1, \dots, 1 \end{bmatrix}}_{mb} \frac{\partial C}{\partial Z^l} \quad (234)$$

$$= \text{numpy.sum}(\frac{\partial C}{\partial Z^l}, \text{axis} = 0) \quad (235)$$

BP4-vec $\frac{\partial C}{\partial W^l}$

$$\text{Recall } \frac{\partial C_k}{\partial W^l} \stackrel{\text{Equation (214)}}{=} A_k^{l-1} \frac{\partial C_k}{\partial Z_k^l}$$

$$\frac{\partial C}{\partial W^l} = \sum_{k=1}^{mb} A_k^{l-1} \frac{\partial C_k}{\partial Z_k^l} \quad (236)$$

$$\text{Strang's 4th} \quad \left[\begin{array}{c|c|c} \vdots & A_k^{l-1} & \vdots \\ \hline \vdots & & \vdots \end{array} \right] \left[\begin{array}{c|c|c} \text{---} & \ddots & \text{---} \\ \text{---} & \frac{\partial C_k}{\partial Z_k^l} & \text{---} \\ \text{---} & \dots & \text{---} \end{array} \right] \quad (237)$$

$$\stackrel{\text{Equation (226)}}{=} A^{l-1} \frac{\partial C}{\partial Z^l} \quad (238)$$

2.11.3 Row-major

In the previous section, we deliberately left the 4 BP vectorized equations in the derivative format (instead of the gradient format as what Nielson book originally used) because they are the exactly the gradients in the row-major that we are going to use here (see the discussion in Section 2.10.3) Note we also use math bond font to represent the data/parameter matrix in row-major. E.g. $\mathbf{X} \in R^{n \times m}$, and $\mathbf{X} = X^T$, for the input data matrix. Another example: $\mathbf{W}^L = (W^L)^T$, for the last layer's weight matrix

BP1 $\nabla_{\mathbf{Z}^L} C$

$$\nabla_{\mathbf{Z}^L} C = \frac{\partial C}{\partial Z^L} \quad (239)$$

$$= (A^L - Y)^T \quad (240)$$

$$= (A^L)^T - Y^T \quad (241)$$

$$= \mathbf{A}^L - \mathbf{Y} \quad (242)$$

BP2 $\nabla_{\mathbf{Z}^l} C$

$$\begin{aligned} \nabla_{\mathbf{Z}^l} C &= \frac{\partial C}{\partial Z^l} \\ &= \frac{\partial C}{\partial Z^{l+1}} W^{l+1} \odot (\sigma'(Z^l))^T \\ &= (\nabla_{\mathbf{Z}^{l+1}} C) (\mathbf{W}^{l+1})^T \odot \sigma'(\mathbf{Z}^l) \end{aligned} \quad (243)$$

BP3 $\nabla_{\mathbf{b}^l} C$

$$\begin{aligned} \nabla_{\mathbf{b}^l} C &= \frac{\partial C}{\partial b^l} \\ &\stackrel{\text{Equation (234)}}{=} \underbrace{\left[1, \dots, 1 \right]}_{mb} \frac{\partial C}{\partial Z^l} \\ &= \underbrace{\left[1, \dots, 1 \right]}_{mb} \nabla_{\mathbf{Z}^l} C \end{aligned} \quad (244)$$

BP4 $\nabla_{\mathbf{W}^l} C$

$$\begin{aligned}
\nabla_{\mathbf{W}^l} C &= \frac{\partial C}{\partial W^l} \\
&\stackrel{\text{Equation (238)}}{=} A^{l-1} \frac{\partial C}{\partial Z^l} \\
&= A^{l-1} \nabla_{\mathbf{Z}^l} C \\
&= (\mathbf{A}^{l-1})^T \nabla_{\mathbf{Z}^l} C
\end{aligned} \tag{245}$$

2.12 2-layer NN as in CMU DLsys

Note, although I am using most of the notations from Nielson book [10] in this section, to make the superscripts consistent with Chen and Zolter [4]. We use $L = 2$ for this two-layer NN, instead of 3 in [10]. In this case, A^0 is the input.

$$\nabla_{\mathbf{Z}^2} C \stackrel{\text{Equation (242)}}{=} \mathbf{A}^2 - \mathbf{Y} \tag{246}$$

$$\begin{aligned}
\nabla_{\mathbf{b}^2} C &\stackrel{\text{Equation (244)}}{=} (1_{mb})^T \nabla_{\mathbf{Z}^2} C \\
&= \text{numpy.sum}(\mathbf{A}^2 - \mathbf{Y}, \text{axis}=0) \text{ To Check w/ PyT!}
\end{aligned} \tag{247}$$

$$\begin{aligned}
\nabla_{\mathbf{W}^2} C &\stackrel{\text{Equation (245)}}{=} (\mathbf{A}^1)^T \nabla_{\mathbf{Z}^2} C \\
&= (\mathbf{A}^1)^T (\mathbf{A}^2 - \mathbf{Y}) \text{ Checked with [4]!}
\end{aligned} \tag{248}$$

$$\begin{aligned}
\nabla_{\mathbf{Z}^1} C &\stackrel{\text{Equation (243)}}{=} (\nabla_{\mathbf{Z}^2} C)(\mathbf{W}^2)^T \odot \sigma'(\mathbf{Z}^1) \\
&= (\mathbf{A}^2 - \mathbf{Y})(\mathbf{W}^2)^T \odot \sigma'(\mathbf{Z}^1)
\end{aligned} \tag{249}$$

$$\begin{aligned}
\nabla_{\mathbf{b}^1} C &\stackrel{\text{Equation (244)}}{=} (1_{mb})^T \nabla_{\mathbf{Z}^1} C \\
&= \text{numpy.sum}((\mathbf{A}^2 - \mathbf{Y})(\mathbf{W}^2)^T \odot \sigma'(\mathbf{Z}^1), \text{axis}=0) \text{ To Check w/ PyT!}
\end{aligned} \tag{250}$$

$$\begin{aligned}
\nabla_{\mathbf{W}^1} C &\stackrel{\text{Equation (245)}}{=} (\mathbf{A}^0)^T \nabla_{\mathbf{Z}^1} C \\
&= (\mathbf{X})^T ((\mathbf{A}^2 - \mathbf{Y})(\mathbf{W}^2)^T \odot \sigma'(\mathbf{Z}^1)) \text{ Checked with :) [4]!}
\end{aligned} \tag{251}$$

2.13 Double check with PyTorch implementation

TODO

2.14 Summary

The studying flow is the following:

1. Single most important equation: Equation (38)
2. Get Nielson's 4 BP equations
3. Vectorization (use of Strang's 3rd and 4th way of doing Matrix Multiplication)

4. Row-major (what derived as derivatives in the column -major is precisely the gradients used in the row-major, just need to make sure the data (aka constants) layout correctly – i.e., $X_{row} = X_{col}^T$)

All in all, Section 2.11 is the easily the highlight of this chapter and well-captured the above-mentioned steps.

3 HW0 Manual Backprop

The mathematical part is dealt with in Section 2.12

Some takeaways:

1. mnist dataset is big-endian, one would need to use

```
struct.unpack
```

to first read in some meta data and then use

```
np.frombuffer()
```

to load data. Also, use `gzip.open` to handle mnist zip files

2. recall the cross entropy loss is the inner product of the ground truth and the **negative log** of the model predicted probabilities. When we get the model predicted probability vector a , don't forget to apply $\log$ and then negate before dot product with 1-hot encoding.
3. one-hot encoding can be done via `np.eye(num_classes)[y]`, $[y]$ is indexing (or slicing)
4. In this lecture, the loss is average loss over minibatch samples, so the gradients need to divide by batch size.
5. I use `numpy.maximum(x,0)` for the relu function impl and use fancy indexing to do the relu derivative.
6. I find laying out each intermediate term (e.g., δ^l) explicitly is error-free for coding, because sometimes you might miss a `()` somewhere (see $\nabla_{\mathbf{W}^2} C$ in my code and make mistakes.
7. Some links useful for developing C++ module with py11 binding in pycharm : <https://stackoverflow.com/questions/66081432/how-to-use-created-c-module-by-pybind11-in-pycharm> and let pycharm see the python module <https://stackoverflow.com/questions/28705029/pycharm-error-no-module-when-trying-to-import-own-module-python-script>
8. One data pointer bug I had is:

```
X_batch = vec_to_mat((float *)(X+start_idx*n), X_batch,
    ↪ batch, n); // weiz 2023-12-18, the bug that I had:
    ↪ i had X+start_idx, instead of X+start_idx*n
```

4 HW1 Needle: NEcessary Elements of Deep LEarning, an Automatic Differentiation Engine

4.1 Backward Formulae

4.1.1 PowerScalar

Raise input to an integer (scalar) power Let $y, x \in R^n; \alpha, l \in R$

$$y = x^\alpha \quad (252)$$

$$\Rightarrow \frac{\partial y}{\partial x} = \begin{bmatrix} \alpha x_1^{\alpha-1} & & \\ & \ddots & \\ & & \alpha x_n^{\alpha-1} \end{bmatrix} \quad (253)$$

$$\begin{aligned} \Rightarrow \nabla_x y &= \left(\frac{\partial y}{\partial x} \right)^T \\ &= \begin{bmatrix} \alpha x_1^{\alpha-1} & & \\ & \ddots & \\ & & \alpha x_n^{\alpha-1} \end{bmatrix} \end{aligned} \quad (254)$$

$$l = L(y) \quad (255)$$

$$\begin{aligned} \Rightarrow \frac{\partial l}{\partial x} &= \frac{\partial l}{\partial y} \frac{\partial y}{\partial x} \\ \Rightarrow \nabla_x l &= \left(\frac{\partial l}{\partial x} \right)^T \\ &= \left(\frac{\partial y}{\partial x} \right)^T \left(\frac{\partial l}{\partial y} \right)^T \\ &= \nabla_x y \nabla_y l \\ &= \begin{bmatrix} \alpha x_1^{\alpha-1} & & \\ & \ddots & \\ & & \alpha x_n^{\alpha-1} \end{bmatrix} \nabla_y l \\ &= \alpha * x^{\alpha-1} \odot \nabla_y l \quad \square \end{aligned} \quad (256)$$

```
def compute(self, a: NDArray) -> NDArray:
    ### BEGIN YOUR SOLUTION
    return array_api.power(a, self.scalar)
    ### END YOUR SOLUTION
```

```

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    return self.scalar * array_api.power(node.inputs[0], self.
        ↳ scalar-1) * out_grad # weiz 2024-01-16 bug fixed , only
        ↳ manifested during hw2
    ### END YOUR SOLUTION

```

4.1.2 EwiseDiv

True division of the inputs, element-wise (2 inputs) Let $y, a, b \in R^n, l \in R$.

$$y = a/b \quad (257)$$

$$\frac{\partial y}{\partial a} = \begin{bmatrix} 1/b_1 & & \\ & \ddots & \\ & & 1/b_n \end{bmatrix} \quad (258)$$

$$\begin{aligned} \Rightarrow \nabla_a y &= \left(\frac{\partial y}{\partial a} \right)^T \\ &= \begin{bmatrix} 1/b_1 & & \\ & \ddots & \\ & & 1/b_n \end{bmatrix} \end{aligned} \quad (259)$$

$$\frac{\partial y}{\partial b} = \begin{bmatrix} -a_1(b_1)^{-2} & & \\ & \ddots & \\ & & -a_n(b_n)^{-2} \end{bmatrix} \quad (260)$$

$$\begin{aligned} \Rightarrow \nabla_b y &= \left(\frac{\partial y}{\partial b} \right)^T \\ &= \begin{bmatrix} -a_1(b_1)^{-2} & & \\ & \ddots & \\ & & -a_n(b_n)^{-2} \end{bmatrix} \end{aligned} \quad (261)$$

$$l = L(y) \quad (262)$$

$$\begin{aligned} \nabla_a l &= \left(\frac{\partial l}{\partial a} \right)^T \\ &= \nabla_a y \nabla_y l \\ &= (1/b) \odot \nabla_y l \quad \square \end{aligned} \quad (263)$$

$$\begin{aligned} \nabla_b l &= \nabla_b y \nabla_y l \\ &= -a \odot b^{-2} \odot \nabla_y l \quad \square \end{aligned} \quad (264)$$

```

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    a,b = node.inputs[0], node.inputs[1]
    grad_a = (b**(-1)) * out_grad

```

```

grad_b = (-1) * a * (b**(-2)) * out_grad
return grad_a, grad_b
### END YOUR SOLUTION

```

4.1.3 DivScalar

: True division of the input by a scalar, element-wise (1 input, scalar - number)
 Let $x, y \in R^n, \alpha, l \in R$

$$y = x/\alpha \quad (265)$$

$$\frac{\partial y}{\partial x} = \begin{bmatrix} \frac{1}{\alpha} & & \\ & \ddots & \\ & & \frac{1}{\alpha} \end{bmatrix} \quad (266)$$

$$l = L(y) \quad (267)$$

$$\begin{aligned} \nabla_x l &= \nabla_x y \nabla_y l \\ &= \frac{1}{\alpha} \nabla_y l \quad \square \end{aligned} \quad (268)$$

```

def gradient(self, out_grad, node):
### BEGIN YOUR SOLUTION
    result = ndl.Tensor(1/self.scalar) * (out_grad) # weiz
    ↪ 2023-12-25, for bwd pass, result seems have to be ndl
    ↪ tensor type ?, whereas fwd has no such type constraint
    ↪ ?
    return result

```

4.1.4 MatMul

: matrix multiplication of the inputs (2 inputs) Let $A \in R^{m \times n}, B \in R^{n \times k}, Y \in R^{m \times k}, l \in R$

$$Y = AB \quad (269)$$

$$l = L(Y) \quad (270)$$

Method 1 Strang's MM, Equation (109) and Equation (129)

$$\nabla_B l \stackrel{\text{Equation (43)}}{=} \begin{bmatrix} | & & | \\ \nabla_{B_1} l & \vdots & \nabla_{B_k} l \\ | & & | \end{bmatrix} \quad (271)$$

$$\begin{aligned} Y_i &\stackrel{\text{Strang's MM2nd}}{=} AB_i \\ \implies Y_i &\text{ only depends on } B_i \\ \implies \frac{\partial Y_{j \neq i}}{\partial B_i} &= \mathbf{0}_{m \times n} \text{ (aka } m \times n \text{ zero-matrix)} \end{aligned} \quad (272)$$

$$\begin{aligned} \frac{\partial l}{\partial B_i} &\stackrel{\text{chain rule}}{=} \sum_{j=1}^{j=k} \frac{\partial l}{\partial Y_j} \frac{\partial Y_j}{\partial B_i} \\ &\stackrel{\text{Equation (272)}}{=} \frac{\partial l}{\partial Y_i} \frac{\partial Y_i}{\partial B_i} \\ &= \frac{\partial l}{\partial Y_i} A \\ \implies \nabla_{B_i} l &= A^T \nabla_{Y_i} l \\ &\stackrel{\text{Equation (271)}}{\implies} \nabla_B l \stackrel{\text{Strang's MM2nd}}{=} A^T \nabla_Y l \end{aligned} \quad (273)$$

To conclude,

$$\nabla_B l = A^T \nabla_Y l \quad \square \quad (274)$$

Note by using (i) Strang's MM 2nd and (ii) the fact that Y_i only depends on B_i (and A) thus Equation (273), we are able to generalize Equation (129) (w.r.t a vector of x) to Equation (274) (w.r.t a matrix format of x , aka B)

Now let's look at $\nabla_A l$,

$$\begin{aligned} \frac{\partial l}{\partial A} &\stackrel{\text{chain rule}}{=} \sum_{i=1}^k \frac{\partial l}{\partial Y_i} \frac{\partial Y_i}{\partial A} \\ \implies \frac{\partial l}{\partial A} &\stackrel{\text{Equation (108)}}{=} \sum_{i=1}^k B_i \frac{\partial l}{\partial Y_i} \\ &\stackrel{\text{Strang's MM4th}}{=} B \frac{\partial l}{\partial Y} \\ \implies \nabla_A l &= \nabla_Y l B^T \end{aligned}$$

To conclude,

$$\nabla_A l = \nabla_Y l B^T \quad \square \quad (275)$$

Note by using (i) Strang's MM 4th and (ii) Equation (108), we are able to generalize Equation (109) to Equation (275)

Method 2: Use Magnus method with kronecker product and matrix block multiplication

$$\begin{aligned}
Y &= I_m AB \\
&\Rightarrow \frac{\partial Y}{\partial A} \stackrel{p.197[8]}{=} B^T \otimes I_m \\
&\Rightarrow \frac{\partial l}{\partial A} = \frac{\partial l}{\partial Y} \frac{\partial Y}{\partial A} \\
&= \underbrace{\left[\frac{\partial l}{\partial Y_1}, \dots, \frac{\partial l}{\partial Y_k} \right]}_{1 \times mk} \underbrace{\begin{bmatrix} \begin{bmatrix} B_{11} & & \\ & \ddots & \\ & & B_{11} \end{bmatrix} & & \begin{bmatrix} B_{n1} & & \\ & \ddots & \\ & & B_{n1} \end{bmatrix} \\ & \ddots & \\ \begin{bmatrix} B_{1k} & & \\ & \ddots & \\ & & B_{1k} \end{bmatrix} & & \begin{bmatrix} B_{nk} & & \\ & \ddots & \\ & & B_{nk} \end{bmatrix} \end{bmatrix}}_{mk \times mn} \\
&\stackrel{\text{Block MatMul}}{=} \underbrace{\left[\sum_{j=1}^k B_{1j} \frac{\partial l}{\partial Y_j}, \dots, \sum_{j=1}^k B_{nj} \frac{\partial l}{\partial Y_j} \right]}_{m \times n} \quad (276)
\end{aligned}$$

By using the partial derivative definition used in Equation (42), we can reshape Equation (276) to the following

$$\begin{aligned}
\frac{\partial l}{\partial A} &= \underbrace{\begin{bmatrix} \sum_{j=1}^k B_{1j} \frac{\partial l}{\partial Y_j} \\ \dots \\ \sum_{j=1}^k B_{nj} \frac{\partial l}{\partial Y_j} \end{bmatrix}}_{n \times m} \\
&\stackrel{\text{Strang's MM3rd}}{=} \begin{bmatrix} B_{1\text{st row}} \frac{\partial l}{\partial Y} & \\ \dots & \\ B_{n\text{th row}} \frac{\partial l}{\partial Y} & \end{bmatrix} \quad (277)
\end{aligned}$$

$$\stackrel{\text{Strang's MM3rd}}{=} B \frac{\partial l}{\partial Y} \quad (\text{note: } \frac{\partial l}{\partial Y} \text{ is per def in Equation (42)}) \quad (278)$$

$$(279)$$

To conclude,

$$\nabla_A l \stackrel{\text{Equation (278)}}{=} \nabla_Y l B^T \quad \square \quad (280)$$

As we can see, Equation (280) and Equation (275) are the same.

$$Y = ABI_k \quad (281)$$

$$\begin{aligned}
\frac{\partial Y}{\partial B} &\stackrel{p.197[8]}{=} I_k^T \otimes A \\
&= I_k \otimes A \\
&= \underbrace{\begin{bmatrix} A & & \\ & \ddots & \\ & & A \end{bmatrix}}_{mk \times nk} \\
\frac{\partial l}{\partial B} &= \frac{\partial l}{\partial Y} \frac{\partial Y}{\partial B} \\
&= \underbrace{\begin{bmatrix} \frac{\partial l}{\partial Y_1} & \dots & \frac{\partial l}{\partial Y_k} \end{bmatrix}}_{1 \times mk} \underbrace{\begin{bmatrix} A & & \\ & \ddots & \\ & & A \end{bmatrix}}_{mk \times nk} \\
&\stackrel{\text{Blk MatMul}}{=} \underbrace{\begin{bmatrix} \frac{\partial l}{\partial Y_1} A & \dots & \frac{\partial l}{\partial Y_k} A \end{bmatrix}}_{1 \times nk} \quad (282)
\end{aligned}$$

By using the wiki derivative def in Equation (42), we can reshape Equation (282) in the following way:

$$\begin{aligned}
\frac{\partial l}{\partial B} &= \begin{bmatrix} \frac{\partial l}{\partial Y_1} A \\ \vdots \\ \frac{\partial l}{\partial Y_k} A \end{bmatrix} \\
&\stackrel{\text{Strang's MM3rd}}{=} \frac{\partial l}{\partial Y} A \text{ (note } \frac{\partial l}{\partial Y} \text{ here is per def in Equation (42))} \quad (283)
\end{aligned}$$

To conclude,

$$\nabla_B l \stackrel{\text{Equation (283)}}{=} A^T \nabla_Y l \quad (284)$$

As we can see, Equation (284) and Equation (274) are the same. Also in this paragraph, I have shown to use the die-hard Magnus approach (i.e., differential matrix calculus) to derive the same results as using Strang's matrix multiplication method (especially 3rd row-based and 4th col multiply row), so I venture to say that I should be also able to use Magnus approach to derive the batched Neural-Network backprop formulae as in Section 2.11.2, which were derived by using Strang's methods.

```

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    A = node.inputs[0]
    B = node.inputs[1]
    grad_A = matmul(out_grad, transpose(B, [-2,-1]))
    grad_B = matmul(transpose(A, [-2,-1]), out_grad)
    if(len(grad_A.shape) > len(A.shape)): # e.g., shape(A)
        ↪ =(5,4), shape(B)=(6,6,4,3), need to sum over the
        ↪ first two axes for grad_A (6,6,5,4) to (5,4)
        diff_len_A = len(grad_A.shape) - len(A.shape)
        grad_A = summation(grad_A, tuple(range(diff_len_A)))
    if (len(grad_B.shape) > len(B.shape)): # e.g., shape(A) =
        ↪ (6,6,5,4) , shape(B) = (4,3), need to sum over the
        ↪ first two axes for grad_B (6,6,4,3) to (4,3)
        diff_len_B = len(grad_B.shape) - len(B.shape)
        grad_B = summation(grad_B, tuple(range(diff_len_B)))
    assert(grad_A.shape == A.shape)
    assert(grad_B.shape == B.shape)
    return grad_A, grad_B

```

Note that when there is a batch matrix multiplication involved, we need to sum over the "bcast"-ed extra axes so the gradients are correctly accounted for. Also for the interwinte of broadcast and summation, see the paragraphs that deal with **BroadcastTo** and **Summation**.

Method 3: Using per element derivative and make educated guess about what the matrix format is, see Lecture 4 slides page 22. Here I only deal with

$$\frac{\partial l}{\partial A} \quad (285)$$

Note, when we take derivative of l w.r.t $A_{i,j}$, the intermediate value that we use for chain rule is $Y_{i,p}$, $p \in 1 \dots k$, as only the i -th row in Y depend on $A_{i,j}$, i.e.,

$$\frac{\partial l}{\partial A_{i,j}} = \sum_{p=1}^k \frac{\partial l}{\partial Y_{i,p}} \frac{\partial Y_{i,p}}{\partial A_{i,j}}, i \in 1 \dots m, j \in 1 \dots n \quad (286)$$

$$Y_{i,p} = \sum_{q=1}^n A_{i,q} B_{q,p} = \sum_{p=1}^k \frac{\partial l}{\partial Y_{i,p}} B_{j,p} \quad (287)$$

$$= \langle (\nabla_Y l \text{ i-th row})^T, (B \text{ j-th row})^T \rangle \quad (288)$$

$$= \langle (\nabla_Y l \text{ i-th row})^T, B^T \text{ j-th column} \rangle \quad (289)$$

Equation (289) indicates $\nabla_A l_{i,j}$ is the inner product of $\nabla_Y l$'s i -th row (as a column vector) and B^T 's j -th column, which by Strang's MM 1st, mean the following

$$\nabla_A l = \nabla_Y l B^T \quad (290)$$

As we can see, Equation (280), Equation (275) and Equation (290) are the same.

4.1.5 Summation

: sum of array elements over given axes (1 input, axes - tuple) **sum** will annihilate the dimension of axes it sums over. E.g., it will reduce the dimension of the same data by `len(axis)`. For example

```
>>> np.sum(np.arange(6),axis=0).shape
()
>>> np.sum(np.arange(6).reshape(2,3),axis=0).shape
(3,)
>>> np.sum(np.arange(6).reshape(1,2,3),axis=0).shape
(2, 3)
```

update: weiz-2024-01-23: If we want to keep the annihilated axis (by making its corresponding dimension to be 1) so that we can do broadcast more easily (i.e., w/o I have to bookeep which axes were annihilated and reshape it with 1), we can use the option `keepdims=True` when doing `numpy.sum` E.g.

```
>>> np.sum(np.arange(6),axis=0, keepdims=True).shape
(1,)
>>> np.sum(np.arange(6).reshape(2,3),axis=0, keepdims=True).shape
(1, 3)
>>> np.sum(np.arange(6).reshape(1,2,3),axis=0, keepdims=True).
    ↪ shape
(1, 2, 3)
```

Example Sum $A(2 \times 3)$:

$+_{1,1}$	$+_{1,2}$	$+_{1,3}$
$-_{2,1}$	$-_{2,2}$	$-_{2,3}$

⁶ over axis=1 to get vector

⁶I use "+" to represent the first row and "-" the 2nd row, similarly done in the **BroadcastTo** paragraph

$a(2 \times 1)$: $\begin{bmatrix} + \\ - \end{bmatrix}$. Let a' the wiki gradients : $\begin{bmatrix} +' \\ -' \end{bmatrix}$

$$\begin{aligned}
\frac{\partial l}{\partial a} &= \nabla_a l^T = \begin{bmatrix} +' & -' \end{bmatrix} \\
\frac{\partial a}{\partial A} &= \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \\
\frac{\partial l}{\partial A} &= \frac{\partial l}{\partial a} \frac{\partial a}{\partial A} \\
&= \begin{bmatrix} +' & -' & +' & -' & +' & -' \end{bmatrix} \\
\nabla_A l \text{ (Magnus)} &= \left(\frac{\partial l}{\partial A} \right)^T \\
&= \begin{bmatrix} +' \\ -' \\ +' \\ -' \\ +' \\ -' \end{bmatrix} \tag{291}
\end{aligned}$$

$$\begin{aligned}
\nabla_A l \text{ (wiki)} &\stackrel{\text{Equation (291)}}{=} \begin{bmatrix} +' & +' & +' \\ -' & -' & -' \end{bmatrix} \\
&= \text{np.broadcast_to}(a', A.\text{shape}) \tag{292}
\end{aligned}$$

Conceptually, a small change of the one element in original A will lead to the exact same change of l as one corresponding element in a (the definition of derivative) because all the rests are constants w.r.t this particular element that we calculate derivative against⁷. So the conceptual implementation will be just

`broadcast_to(grad_out, orig_shape)`

Implementation

⁷Originally, I mistakenly though the derivative takes a proportional change accounting other elements that contribute to the sum in the corresponding position of a . But soon I realized that all the other elements, albeit summed with, are just constants when calculating derivative, thus the derivative is 1, which is the same case as in **add** case.

```

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    if(self.axes is None): # weiz 2023-12-30, when dealing
        ↪ with AD impl, realized self.axes could be None
        # self.axes = np.arange(len(node.inputs[0].shape)) #
        ↪ alternatively, one can do this, but i am not
        ↪ sure if this node's inputs share are going to
        ↪ change
        orig_shape = node.inputs[0].shape # when it is none,
        ↪ the out_grad is a 0-dimensional, just bcst it to
        ↪ the original shape
        return broadcast_to(out_grad, orig_shape)

    orig_shape = node.inputs[0].shape
    # step 1 get real shape of out_grad
    # as out_grad could be a 1-dim as a result of sum
    # but we want the explicit shape of out_grad
    l=[]
    for s in orig_shape:
        l.append(s)
    for s in self.axes:
        l[s] = 1
    real_shape = tuple(l)
    out_grad = out_grad.reshape(real_shape)

    # step 2 broadcast to the orig_shape
    return broadcast_to(out_grad, orig_shape)
    ### END YOUR SOLUTION

```

Note one corner case, is that when `self.axes` is `None`, that means the summation goes on all axes, so the `out_grad` is a 0-dimensional scalar and we just need to broadcast it to the original shape. So `self.axes` being `None` really means `self.axes= np.arange(len(input.shape))`, aka sum across **all** axes.

Update: weiz 2024-01-24 If I use the `keepdims=True` to keep track of the proper output shape then `squeeze` it to get what professors want for the summ-ed shape. The implementation would look like this:

```

def compute(self, a):
    ### BEGIN YOUR SOLUTION
    keepdims_result = array_api.sum(a, axis=self.axes,
    ↪ keepdims=True) # weiz 2024-01-24 keepdims so that
    ↪ we know the right results shape
    self.result_orig_shape = keepdims_result.shape
    return array_api.squeeze(keepdims_result) # use squeeze
    ↪ trick so that it drops the dimension=1 axes to
    ↪ conform with what the professors want
    ### END YOUR SOLUTION

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    if(self.axes is None): # weiz 2023-12-30, when dealing
    ↪ with AD impl, realized self.axes could be None
    # self.axes = np.arange(len(node.inputs[0].shape)) #
    ↪ alternatively, one can do this, but i am not
    ↪ sure if this node's inputs share are going to
    ↪ change
    orig_shape = node.inputs[0].shape # when it is none,
    ↪ the out_grad is a 0-dimensional, just bcst it to
    ↪ the original shape
    return broadcast_to(out_grad, orig_shape)
    orig_shape = node.inputs[0].shape
    # step 1 get real shape of out_grad
    out_grad = out_grad.reshape(self.result_orig_shape)
    # step 2 broadcast to the orig_shape
    return broadcast_to(out_grad, orig_shape)
    ### END YOUR SOLUTION

```

4.1.6 BroadcastTo

: broadcast an array to a new shape (1 input, shape - tuple). Per <https://numpy.org/doc/stable/user/basics.broadcasting.html>, When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and works its way left. Two dimensions are compatible when

1. they are equal, or
2. one of them is 1.

It made me think that 1-dimensional array in numpy is a row-vector, that is I can broadcast a (3,) array to (1,3) but not (3,1). E.g.,

```

>>> np.broadcast_to(np.arange(3), (1,3))
array([[0, 1, 2]])
>>> np.broadcast_to(np.arange(3), (3,1))
Traceback (most recent call last):
...
ValueError: operands could not be broadcast together with
  ↳ remapped shapes [original->remapped]: (3,) and requested
  ↳ shape (3,1)

```

Example, Broadcast a 2x1 vector a :

+
-

 to a 2x3 matrix A :

+	+	+
-	-	-

Let A' be the A 's gradients (wiki def) be

$+_{1,1}'$	$+_{1,2}'$	$+_{1,3}'$
$-_{2,1}'$	$-_{2,2}'$	$-_{2,3}'$

Per Magnus def (aka $\text{vec}(A)$), the

$$\begin{aligned}
 \frac{\partial l}{\partial A} &= \begin{bmatrix} +_{1,1}' & -_{2,1}' & +_{1,2}' & -_{2,2}' & +_{1,3}' & -_{2,3}' \end{bmatrix} \\
 D_{\text{broadcast}} = \frac{\partial A}{\partial a} &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \\
 \frac{\partial l}{\partial a} &= \frac{\partial l}{\partial A} \frac{\partial A}{\partial a} \\
 &= \left[\sum_{k=1}^3 +_{1,k}', \sum_{k=1}^3 -_{2,k}' \right] \\
 \nabla_a l = \left(\frac{\partial l}{\partial a} \right)^T &= \begin{bmatrix} \sum_{k=1}^3 +_{1,k}' \\ \sum_{k=1}^3 -_{2,k}' \end{bmatrix} = \text{np.sum}(A', \text{axis} = 1) \quad (293)
 \end{aligned}$$

Conceptually, broadcast expands a core tensor to added dimensions, and when calculate gradients against this core tensor, we simply need to sum gradients (w.r.t expanded tensor) over the expanded dimensions to drop the expanded dimensions and get the result in the original core tensor shape. See the code below

Implementation

```
def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    orig_shape = node.inputs[0].shape
    new_shape = self.shape
    # step 1 construct orig shape (bcast from)
    # and new shape (bcast to), expand orig shape
    # to the same dimension as the new shape
    orig_shape_lst = list(orig_shape)
    new_shape_lst = list(new_shape)
    orig_shape_lst = [1]*(len(new_shape_lst)-len(
        ↪ orig_shape_lst))+orig_shape_lst

    # step 2 find the axes indices where bcast happened
    axis_indices = [index for index, (elem1, elem2) in
        ↪ enumerate(zip(new_shape_lst, orig_shape_lst)) if
        ↪ elem1 > elem2]

    # step 3 sum over the axes where bcast happend, reshape
    # is needed because summation can endup with 1-dimension
    # numpy array, and we need to make
    # it explicitly the same shape as the original shape (be
    # it row vector or column vector)
    # e.g., gradient_check(ndl.broadcast_to, ndl.Tensor
    (np.random.randn(3, 1)), shape=(3, 3))
    # gradient_check(ndl.broadcast_to, ndl.Tensor(np.random.
        ↪ randn(1, 3)), shape=(3, 3))
    return summation(out_grad, tuple(axis_indices)).reshape(
        ↪ orig_shape)
    ### END YOUR SOLUTION
```

Note in both bcast and sum, we need some reshape, this is due to the fact `numpy.sum()` drop the axes it sums over. Had it merely kept the axes and left them the size 1, we wouldn't need to do this reshape.

4.1.7 Reshape

: gives a new shape to an array without changing its data (1 input, shape - tuple). We simply need to 摆摆好gradients w.r.t the original shape, i.e., we just need to reshape gradients to the original shape.

```
def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    orig_shape = node.inputs[0].shape
    return reshape(out_grad, orig_shape)
```



```
### END YOUR SOLUTION
```

4.1.8 Negate

Numerical negative, element-wise (1 input) We just need to negate the gradients.

```
def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    return -1 * out_grad
    ### END YOUR SOLUTION
```

4.1.9 Transpose

: reverses the order of two axes (axis1, axis2), defaults to the last two axes (1 input, axes - tuple) By transpose, it really means swap axes. We simply need to swap the axes back.

```
class Transpose(TensorOp):
    def __init__(self, axes: Optional[tuple] = None):
        if(axes is None): # these two lines are added by weiz
            ↪ 2023-12-22
            axes=[-2,-1]
        self.axes = axes

    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        return array_api.swapaxes(a, self.axes[0], self.axes[1])
        # weiz: 2023-12-22, this function is really about swap
        ↪ axes
        ### END YOUR SOLUTION

    def gradient(self, out_grad, node):
        ### BEGIN YOUR SOLUTION
        return transpose(out_grad, self.axes)
        ### END YOUR SOLUTION

def transpose(a, axes=None):
    return Transpose(axes)(a)
```

Exp $l \in R, x, y \in R^n$

$$\begin{aligned}
l &= L(y) \\
y &= \exp(x) \\
\frac{\partial l}{\partial x} &= \frac{\partial l}{\partial y} \frac{\partial y}{\partial x} \\
&= \frac{\partial l}{\partial y} \text{Diag}(\exp(x))
\end{aligned} \tag{294}$$

$$\begin{aligned}
\implies \nabla_x l &= \text{Diag}(\exp(x)) \nabla_y l \\
&= \exp(x) \odot \nabla_y l \quad \square
\end{aligned} \tag{295}$$

Conceptually, for higher dimensional tensors, we get similar results: we need to take an element-wise multiply $\frac{\partial l}{\partial y_{ij}}$ with $\frac{\partial y_{ij}}{\partial x_{ij}} = \exp(x_{ij})$ to get $\frac{\partial l}{\partial x_{ij}}$

```

class Exp(TensorOp):
    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        return array_api.exp(a)
        ### END YOUR SOLUTION

    def gradient(self, out_grad, node):
        ### BEGIN YOUR SOLUTION
        assert(out_grad.shape == node.inputs[0].shape)
        return out_grad * exp(node.inputs[0])
        ### END YOUR SOLUTION

```

4.1.10 Log

$l \in R, x, y \in R^n$

$$\begin{aligned}
l &= L(y) \\
y &= \log(x) \\
\frac{\partial l}{\partial x} &= \frac{\partial l}{\partial y} \frac{\partial y}{\partial x} \\
&= \frac{\partial l}{\partial y} \text{Diag}(1/x)
\end{aligned}$$

$$\implies \nabla_x l = (1/x) \odot \nabla_y l \quad \square \tag{296}$$

Conceptually, for higher dimensional tensors, we get similar results: we need to take an element-wise multiply $\frac{\partial l}{\partial y_{ij}}$ with $\frac{\partial y_{ij}}{\partial x_{ij}} = 1/(x_{ij})$ to get $\frac{\partial l}{\partial x_{ij}}$

```

class Log(TensorOp):
    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        return array_api.log(a)
        ### END YOUR SOLUTION

    def gradient(self, out_grad, node):
        ### BEGIN YOUR SOLUTION
        assert(out_grad.shape == node.inputs[0].shape)
        return out_grad / node.inputs[0]
        ### END YOUR SOLUTION

```

4.1.11 ReLU

Below is the ReLU implementation.

```

class ReLU(TensorOp):
    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        return array_api.maximum(0, a)
        ### END YOUR SOLUTION

    def gradient(self, out_grad, node):
        ### BEGIN YOUR SOLUTION
        input_grad = node.inputs[0].realize_cached_data().copy()
        input_grad[input_grad<=0] = 0
        input_grad[input_grad>0] = 1 # at this point input_grad is
            ↳ NDAarray, but the * in the next line make it a ndl.
            ↳ Tensory type ?
        return out_grad * input_grad # weiz 2024-01-02 don't
            ↳ forget to multiply out_grad, otherwise we didn't
            ↳ get the gradients w.r.t loss function!!
        ### END YOUR SOLUTION

```

Comments:

- Don't forget multiply out_grad (w/ the input_grad), that was the most severe mistake that I made!!!
- I use smart indexing for input_grad, as it is still NDAarray type; then the * in the next line seems to cast it to ndl. Tensor type still keep the result in NDAarray, weiz 2024-07-04, so when is this cast to Tensor ?

4.2 Automatic Differentiation (AD)

DFS, pre-order, post-order, topological order DFS can have be ordered in two different ways: pre-order and post-order ⁸. **Topological-order** is the reverse of DFS **post-order**. **It was a mistake to say topological-order is the reverse of DFS, because DFS can have different orders. It has to be the reverse of DFS post-order.**

Listing 1: DFS pre-order, post-order and in-order (in-order exists only when traversing on a binary tree)

```
preorder(node):
    if node is not null:
        visit(node)
        preorder(node.left)
        preorder(node.right)
postorder(node):
    if node is not null:
        postorder(node.left)
        postorder(node.right)
        visit(node)
inorder(node): # only exists when it traverses a binary tree
    if node is not null:
        inorder(node.left)
        visit(node)
        inorder(node.right)
```

So why topological sort is important ? Topological sort can be used to convert a directed acyclic graph, or more commonly a dependency tree into a linear order such that if any event B requires that A be completed before B is initiated then A will occur before B in the ordering⁹. In another word, **we can make sure only when one op's dependencies are calculated, we will calculate op's gradients, if we follow the topological order. That is the key point of running AD engine following a topological order in the gradient calculation graph.**

⁸DFS also has in-order, if it traverses a binary tree

⁹<https://www.quora.com/What-is-the-topological-order-in-the-context-of-algorithms>

Adjoint in computation graph, and the proof of adjoint The key question is does chain rule apply to the vector form in this ? Obviously, the if $x, y, z \in R$, it will hold, but what if they are vectors ?

$$\begin{aligned}
x &\in R^n, y \in R^m, z \in R^k, E \in R \\
y &= f(x) \\
z &= g(x) \\
E &= l(y, z) \\
\frac{\partial E}{\partial x} &= \frac{\partial E}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial E}{\partial z} \frac{\partial z}{\partial x}
\end{aligned} \tag{297}$$

proof

$$\begin{aligned}
x &\in R^n, y \in R^m, z \in R^k, E \in R \\
y &= f(x) \\
z &= g(x) \\
E &= l(y, z)
\end{aligned}$$

Let

$$\begin{aligned}
Y &= \text{concat}(y, z) \in R^{m+k} \\
&= \text{concat}(f(x), g(x)) \\
E &= L(Y) = l(y, z)
\end{aligned}$$

Then

$$\frac{\partial E}{\partial x} = \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial x}$$

Note

$$\begin{aligned}
\frac{\partial E}{\partial Y} &= \left[\frac{\partial l(y, z)}{\partial y}, \frac{\partial l(y, z)}{\partial z} \right] \\
\frac{\partial Y}{\partial x} &= \begin{bmatrix} \frac{\partial f(x)}{\partial x} \\ \frac{\partial g(x)}{\partial x} \end{bmatrix}
\end{aligned}$$

As a result

$$\begin{aligned}
\frac{\partial E}{\partial x} &= \frac{\partial E}{\partial Y} \frac{\partial Y}{\partial x} = \left[\frac{\partial l(y, z)}{\partial y}, \frac{\partial l(y, z)}{\partial z} \right] \begin{bmatrix} \frac{\partial f(x)}{\partial x} \\ \frac{\partial g(x)}{\partial x} \end{bmatrix} \\
&= \frac{\partial l(y, z)}{\partial y} \frac{\partial f(x)}{\partial x} + \frac{\partial l(y, z)}{\partial z} \frac{\partial g(x)}{\partial x} \\
&= \frac{\partial E}{\partial y} \frac{\partial y}{\partial x} + \frac{\partial E}{\partial z} \frac{\partial z}{\partial x} \quad \square
\end{aligned}$$

It then is quite straightforward (via induction proof) to see that

$$\bar{v}_i = \frac{\partial L}{\partial v_i} \text{ def of adjoint} \quad (298)$$

$$\overline{v_{i \rightarrow j}} = \frac{\partial L}{\partial v_j} \frac{\partial v_j}{\partial v_i} = \bar{v}_j \frac{\partial v_j}{\partial v_i} \quad \begin{array}{l} \text{def of partial adjoint} \\ v_j \text{'s contribution to } \bar{v}_i \end{array} \quad (299)$$

$$\bar{v}_i = \sum_{j \in \text{next}(i)} \overline{v_{i \rightarrow j}} \quad \begin{array}{l} \text{Sum up all } v_i \text{'s immediate dependencies} \\ \text{, in gradient calculation graph,} \\ \text{contributions to } \bar{v}_i \end{array} \quad (300)$$

from Lecture 4. Note the actual definition of adjoint (and partial adjoint) is in terms of **gradient** not **jacobian**. But we can simply take a transpose here see the discussion of Equation (44).

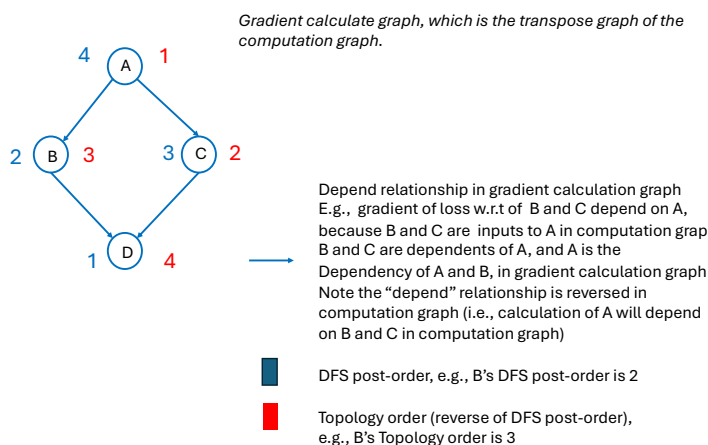


Figure 4: Automatic Differentiation Gradients Calculation Dependency Graph. DFS post-order and Topological Order

Why I use gradients calculation graph, the edge inverted graph (aka the transposed graph of computation graph) of calculation graph?

Figure 4 is the gradients calculation dependency graph. Note that one can get the topological order by reversing the DFS post-order (<https://www.youtube.com/watch?v=AfSk24UTFS8&t=584s>, MIT algo class on DFS and topological order). Note, in this figure, B and C are dependents of A and A is the dependency of B and C.

In the CMU 10714 lecture, the instructors insist on the forward calculation graph, which is the edge-inverted graph (aka transpose graph) of the depicted graph. In their treatment, one would need to traverse over the reversed topological

order, which is the topological order of the dependency graph that I depict in Figure 4. So in my treatment, **I only need to traverse the topological order of this gradient calculation graph as depicted in Figure 4.** *I feel using the gradients calculation dependency graph is much more intuitive for the purpose of calculating gradients, so I am sticking with it.*

Update 2024-01-08 CMU profs' point is that the added nodes during gradient calculation again format a computation graph, which is a cute idea. I.e., when we calculate gradients in each **ops** class, we return new tensors, which are the inputs of the gradients, this input relationship forms a new edge in the computation graph.

```

def find_topo_sort(node_list: List[Value]) -> List[Value]:
    """Given a list of nodes, return a topological sort list of
        ↪ nodes ending in them.

    A simple algorithm is to do a post-order DFS traversal on the
        ↪ given nodes,
    going backwards based on input edges. Since a node is added to
        ↪ the ordering
    after all its predecessors are traversed due to post-order DFS
        ↪ , we get a topological
    sort.
    """
    ### BEGIN YOUR SOLUTION
    #assert(list(node_list) == 1) # weiz 2023-12-30 there could be
        ↪ multiple root nodes in DAG
    visited = set()
    topo_order = [] # weiz 2023-12-30, really this is the dfs
        ↪ order
    for node in node_list:
        topo_sort_dfs(node, visited, topo_order)
    return topo_order
    ### END YOUR SOLUTION

def topo_sort_dfs(node, visited, topo_order):
    """Post-order DFS"""
    ### BEGIN YOUR SOLUTION
    for n in node.inputs:
        if(n not in visited): # technically, i don't need this if,
            ↪ as line 411 will guard it, at the cost of extra
            ↪ function calls
            topo_sort_dfs(n, visited, topo_order)
        if(node not in visited): # this branch is to avoid adding
            nodes multiple times, in DAG (richer than tree), there #could
            ↪ be multiple paths that lead to the same node
            visited.add(node)
            topo_order.append(node)

    ### END YOUR SOLUTION

```

Figure 5 depicts one key-point that I was missing: Gradient calculation $\frac{\partial \text{output}}{\partial \text{input}}$ happens in the output node. All in all, the AD algorithm is in the following code 2

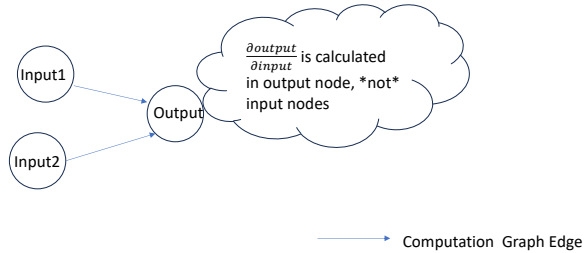


Figure 5: Gradient calculation $\frac{\partial \text{output}}{\partial \text{input}}$ happens in the output node

Listing 2: AD algo sketching

```
def compute_gradient_of_variables(output_tensor, out_grad):
    """Take gradient of output node with respect to each node in
    ↳ node_list.

    Store the computed result in the grad field of each Variable.
    """
    # a map from node to a list of gradient contributions from
    ↳ each output node
    node_to_output_grads_list: Dict[Tensor, List[Tensor]] = {}
    # Special note on initializing gradient of
    # We are really taking a derivative of the scalar reduce_sum(
    ↳ output_node)
    # instead of the vector output_node. But this is the common
    ↳ case for loss function.
    node_to_output_grads_list[output_tensor] = [out_grad]

    # Traverse graph in reverse topological order given the
    ↳ output_node that we are taking gradient wrt.
    reverse_topo_order = list(reversed(find_topo_sort([
    ↳ output_tensor])))
    for node in reverse_topo_order:
        if node.requires_grad:
            node.grad = sum_node_list(node_to_output_grads_list[
            ↳ node])
            if node.op is None: # if true inputs node, no ops are
            ↳ defined
                continue
            partial_adjoints = node.op.gradient_as_tuple(node.grad,
            ↳ node) # weiz 2023-12-30, note, we need to use
            ↳ node.op not input_node.op, this is most
            ↳ IMPORTANT!!!
            for input_node, input_node_partial_adjoint in zip(node.
            ↳ inputs, partial_adjoints):
                if (input_node not in node_to_output_grads_list):
```

Also, note that `gradient_as_tuple()` is paired with the inputs to provide the gradients of the output node w.r.t its each input.

4.3 Some takeaways

1. The forward computation part, I take the comments from jupyter note book (e.g., reverses the order of two axes (axis1, axis2), defaults to the last two axes (1 input, axes - tuple)) and then send it to chatgpt, chatgpt has been quiteful helping me find the right numpy function.
2. While the forward pass seems to be totally fine with the result in **numpy** type, the backward pass seems to be strict with that the result must be in the **needle.Tensor** type. See the following code (**DivScalar**)

```
def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    result = ndl.Tensor(1/self.scalar) * (out_grad) # weiz
        ↪ 2023-12-25, for bwd pass, result seems have to be
        ↪ ndl tensor type ?, whereas fwd has no such type
        ↪ constraint ?
    return result
```

Update: seems my current coding is fine now.

3. Needle library doesn't support implicit bcst and to construct a tuple in python when there is only one element a, we must use (a,) instead of (a). See these two points in the following code. Update: I have changed Summation coding, so now needle library also supports axes=a grammar just like numpy.

```
# make a column vector, element component is the
# sum of each row in Z_exp
# note that , I have to use axes=(1,) not axes=(1), as (1)
# is not a tuple
# in contrast, numpy could take axis=1,
# here we can only do axes=(1,)
Z_exp_sum_col = (ndl.summation(Z_exp, axes=(1,))).reshape
    ↪ ((-1,1))

# bcst (explicitly in needl) and normalize Z_exp
# to get Activation A
A = Z_exp / ndl.broadcast_to(Z_exp_sum_col, Z_exp.shape)
```

4. I used in-place update for the parameters updates to avoid building additional computation graph nodes (todo: double check w/ where did Tianqi talk about it in his lecture).

4.4 TODO

- ~~To understand `gradient_check()` numerically routine~~ (weiz 2024-07-04, See Listing 27 in Section 7.5.)
- To understand the gradient of gradient calculation (e.g., `test_autograd_hw.py:878`)!!!
I feel this is the most missing piece of my understanding of this lecture.
weiz 2024-07-04
- Draw the computation node to illustrate the inputs node adjoints can be only calculated using the `node.op.grad` (aka the error that I made when coding the `calculate_gradients` method)
- Explain the code comments

```
def gradient_as_tuple(self, out_grad: "Value",
    ↪ node: "Value") -> Tuple["Value"]:
```

"""Convenience method to always return a tuple from
↪ gradient call"""

5 HW 2 Build an NN library in Needle

5.1 Weight Decay

Weight Decay is just a regularizer for weights.

$$\min_{\theta} f(\theta) = \frac{1}{2} \|\theta\|_{Frob}^2 \quad (301)$$

$$\|\theta\|_F^2 \stackrel{[8] \text{ p.32 } \text{tr}(A^T B) = \text{vec}(A)^T \text{vec}(B)}{=} \text{vec}(\theta)^T \text{vec}(\theta) \quad (302)$$

$$d\|\theta\|_F^2 = d(\text{vec}(\theta)^T \text{vec}(\theta)) \quad (303)$$

$$\begin{aligned} &= \underbrace{d(\text{vec}(\theta)^T) \text{vec}(\theta)}_{\text{scalar}} + \underbrace{\text{vec}(\theta)^T d(\text{vec}(\theta))}_{\text{scalar}} \\ &= (d(\text{vec}(\theta)^T) \text{vec}(\theta))^T + \text{vec}(\theta)^T d(\text{vec}(\theta)) \\ &= 2\text{vec}(\theta)^T d(\text{vec}(\theta)) \end{aligned}$$

$$\implies d\left(\frac{1}{2} \|\theta\|_{Frob}^2\right) = \text{vec}(\theta)^T d(\text{vec}(\theta)) \quad (304)$$

$$\implies \frac{\partial f}{\partial \theta} = \text{vec}(\theta)^T \quad (305)$$

$$\begin{aligned} \implies \nabla_{\theta} f_{magnus} &= \text{vec}(\theta) \\ \implies \nabla_{\theta} f_{wiki} &= \theta \end{aligned} \quad (306)$$

5.2 Log-Sum-Exp

max operator TODO: `numpy.max` `numpy.amax` `numpy.argmax`

Log-Sum-Exp trick Let $z \in R^n$, max function is a function that maps from $R^n \rightarrow R$. Assuming t is the index of the largest value of vector $z \in R^n$.

$$\max(z) = z_t \quad (307)$$

$$\frac{\partial \max}{\partial z} = \begin{bmatrix} 0 & \dots & \underbrace{1}_{t-th} & \dots & 0 \end{bmatrix} \quad (308)$$

In RHS of Equation (308), the t -th element is 1 and all other elements are 0.

Let $x \in R^n$, then Equation (309) is the Log-Sum-Exp (LSE) trick.

$$\log\left(\sum_{i=1}^n e^{x_i}\right) = \log\left(\sum_{i=1}^n e^{(x_i - \max(x))}\right) + \max(x) \quad (309)$$

In particular, the LHS of Equation (309) is called Log-Sum-Exp

Proof:

$$\begin{aligned} \log\left(\sum_{i=1}^n e^{x_i}\right) &= \log\left(\sum_{i=1}^n \frac{e^{x_i}}{e^{\max(x)}} e^{\max(x)}\right) \\ &= \log\left(e^{\max(x)} \sum_{i=1}^n \frac{e^{x_i}}{e^{\max(x)}}\right) \\ &= \log(e^{\max(x)}) + \log\left(\sum_{i=1}^n \frac{e^{x_i}}{e^{\max(x)}}\right) \\ &= \max(x) + \log\left(\sum_{i=1}^n e^{(x_i - \max(x))}\right) \\ &= \log\left(\sum_{i=1}^n e^{(x_i - \max(x))}\right) + \max(x) \quad \square \end{aligned}$$

LSE forward and backward First, I define *mbcast* function, which is the bcast version of max function maps from $R^n \rightarrow R^n$.

$$mbcast(z) = \begin{bmatrix} z_t \\ \vdots \\ z_t \end{bmatrix} \quad (310)$$

$$\frac{\partial mbcast}{\partial z} = \begin{bmatrix} 0 & \dots & 1 & \dots & 0 \\ & & \vdots & & \\ 0 & \dots & 1 & \dots & 0 \end{bmatrix} \quad (311)$$

$$\begin{aligned} f(z) &= z - mbcast(z) \\ \frac{\partial f}{\partial z} &= eye(n) - \frac{\partial mbcast}{\partial z} \\ &= \begin{bmatrix} 1 & 0 & 0 & -1 & \dots \\ 0 & 1 & 0 & -1 & \dots \\ & & \ddots & & \\ 0 & 0 & 0 & 0 & \dots \\ 0 & & . & -1 & 1 \end{bmatrix} \end{aligned} \quad (312)$$

$$\begin{aligned} g(f) &= exp(f) \\ \frac{\partial g}{\partial f} &= \begin{bmatrix} e^{f_1} & & \\ & \ddots & \\ & & e^{f_n} \end{bmatrix} \end{aligned} \quad (313)$$

$$s = sum(g) \quad (314)$$

$$\frac{\partial s}{\partial g} = (1_n)^T \text{ones row vector} \quad (315)$$

$$l = log(s) \quad (316)$$

$$\frac{\partial l}{\partial s} = \frac{1}{s} \quad (317)$$

In Equation (311) is $n \times n$ matrix with all zeros except the t -th column is a all 1 vector. In Equation (312) is identity matrix except the t -th column is -1s (with the exception that (t, t) -th element is 0

$$\frac{\partial l}{\partial z} = \frac{\partial l}{\partial s} \frac{\partial s}{\partial g} \frac{\partial g}{\partial f} \frac{\partial f}{\partial z} \quad (318)$$

$$= \frac{1}{s} (1_n)^T \begin{bmatrix} e^{f_1} & & \\ & \ddots & \\ & & e^{f_n} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -1 & \dots \\ 0 & 1 & 0 & -1 & \dots \\ & & \ddots & & \\ 0 & 0 & 0 & 0 & \dots \\ 0 & & . & -1 & 1 \end{bmatrix} \quad (319)$$

$$= \begin{bmatrix} \frac{e^{f_1}}{s} & \dots & \frac{e^{f_n}}{s} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -1 & \dots \\ 0 & 1 & 0 & -1 & \dots \\ & & \ddots & & \\ 0 & 0 & 0 & 0 & \dots \\ 0 & & . & -1 & 1 \end{bmatrix} \quad (320)$$

$$= \begin{bmatrix} \frac{e^{f_1}}{s} & \dots & -1 \underbrace{\frac{(\sum_{i=1}^n e^{f_i}) - e^{f_t}}{s}}_{t-th} & \frac{e^{f_n}}{s} \end{bmatrix} \quad (321)$$

$$LSE(z) = l(z) + max(z) \quad (322)$$

$$\frac{\partial LSE}{\partial z} = \frac{\partial l}{\partial z} + \frac{\partial max}{\partial z}$$

$$\begin{matrix} \text{Equation (321)} \\ \text{Equation (308)} \end{matrix} = \begin{bmatrix} \frac{e^{f_1}}{s} & \dots & -1 \underbrace{\frac{(\sum_{i=1}^n e^{f_i}) - e^{f_t}}{s}}_{t-th} & \dots & \frac{e^{f_n}}{s} \end{bmatrix} \quad (323)$$

$$+ \begin{bmatrix} 0 & \dots & \underbrace{1}_{t-th} & \dots & 0 \end{bmatrix} \\ = \begin{bmatrix} \frac{e^{f_1}}{s} & \dots & \frac{e^{f_t}}{s} & \dots & \frac{e^{f_n}}{s} \end{bmatrix} \quad (324)$$

One key finding is that, such along axes operation (e.g., logsumexp, sum) only operate over the region that is defined by the axes and won't have interference with any other outputs' operated region. In another word, there is no datarace. So Equation (324) can be applied to any arbitrary region or axes. In another word, s is $sum(e^{f_i}, D)$, e^{f_i} is $e^{(x_i - max(X, D))}$ where D is the region aka axes. Note that renaming axes won't change anything – even xz plane is still a plane, axes is over (0,2) a discontinuous tuple, but the region is continuous. It took me over a week to come to this finding!!! Equipped with this finding, the logsumexp impl is the following:

```
class LogSumExp(TensorOp):
```

```

def __init__(self, axes: Optional[tuple] = None):
    if (axes is not None):
        if (type(axes) is not tuple):
            assert (type(axes) is int)
            self.axes = (axes,) # make it like sum axes
            ↪ semantic in weiz needle (aka sum semantic in
            ↪ numpy) weiz 2023-01-23
        else:
            self.axes = axes
    else:
        self.axes = None

def compute(self, Z):
    ### BEGIN YOUR SOLUTION
    self.fwd_input_orig_shape = Z.shape
    max_z = array_api.max(Z, axis=self.axes, keepdims=True) #
    ↪ keep the annihilated axes as dimension 1, that is
    ↪ long time what I want
    z_minus_z_max = Z - max_z # because I keep the annihilated
    ↪ dimension as 1, so here max_z can be bcasted to Z
    ↪ properly
    f = array_api.exp(z_minus_z_max)
    sum_exp_z = array_api.sum(f, axis=self.axes, keepdims=True
    ↪ ) # similar to max, keep the annihilated axes as
    ↪ dimension 1
    assert(sum_exp_z.shape == max_z.shape)
    self.grad_intermediate = f / sum_exp_z
    assert(self.grad_intermediate.shape == self.
    ↪ fwd_input_orig_shape)
    self.fwd_output_orig_shape = max_z.shape # bookeeping the
    ↪ fwd output right shape
    lse = array_api.squeeze(array_api.log(sum_exp_z) + max_z)
    ↪ # in order to make the semantic correspond to
    ↪ keepdims=False, the result need a reduction of axes,
    ↪ but the actual number stay the same
    return lse
    ### END YOUR SOLUTION

def gradient(self, out_grad, node):
    ### BEGIN YOUR SOLUTION
    # step 1 reshape out_grad to original output shape
    out_grad = out_grad.reshape(self.fwd_output_orig_shape)
    # step 2 bcast out_grad to original input shape
    out_grad = broadcast_to(out_grad, self.
    ↪ fwd_input_orig_shape)
    # step 3 element-wise multiply out_grad with

```

```

    ↪ grad_intermediate
    return out_grad * self.grad_intermediate
    ### END YOUR SOLUTION

```

The key thing is to keep a `grad_intermediate` during fwd computation, which exactly the Equation (324) generalized to any region D (e.g., axes). Also `grad_intermediate` shape should be the same as the input shape of `logsumexp`. During backward, we thus need to do three steps:

1. Reshape the `out_grad` to the proper shape , fwd output shape, (e.g., fill in the missing 1-dimension). See
`out_grad = out_grad.reshape(self.fwd_output_orig_shape)`
2. Bcast `out_grad` to proper shape, fwd input shape. See
`out_grad = broadcast_to(out_grad, self.fwd_input_orig_shape)`
3. Element-wise product: `out_grad * self.grad_intermediate`

I suspect this procedure will be the same for any "reduction-like" function (actually `logsumexp` is more complicated than a reduction function) that generate a scalar value along a region (aka axes).

Crux of the problem: Why is `logsumexp` needed in deep learning ? Answer is we need it for `cross-entropy-loss`.

Recall that P is the ground-truth distribution, Q is the model prediction (aka, the softmax of logits), let one-hot $\in R^k$ and logits $\in R^k$, where k is the number of classes, then we have

$$\begin{aligned}
L(\theta) &= - \langle P, \log(Q) \rangle \\
&= - \langle \text{one-hot}, \log(\text{softmax}(\text{logits})) \rangle \\
&= - \langle \text{one-hot}, \log\left(\frac{e^{\text{logits}}}{\sum e^{\text{logits}}}\right) \rangle \\
&= - \langle \text{one-hot}, \log(e^{\text{logits}}) - \log \sum (e^{\text{logits}}) \rangle \\
&= - \langle \text{one-hot}, \text{logits} - LSE(\text{logits}) \rangle \\
&= \langle \text{one-hot}, LSE(\text{logits}) - \text{logits} \rangle \\
&\stackrel{LSE(\text{logits}) \text{ is a scalar}}{=} LSE(\text{logits}) - \langle \text{one-hot}, \text{logits} \rangle \quad (325)
\end{aligned}$$

We now extend Equation (325) to the batch case, then one-hot $\in R^{m \times k}$ and logits $\in R^{m \times k}$, where m is the number of samples, k is the number of classes. We introduce an auxiliary variable $l(\theta) \in R^m$, which represents the cross-entropy loss for each sample in a batch. Then the final loss is just the sum of $l(\theta)$.

$$\begin{aligned}
l(\theta) &= LSE(\text{logits}, \text{axis} = 1) - \text{sum}(\text{one-hot} \odot \text{logits}, \text{axis} = 1) \\
L(\theta) &= \text{sum}(l(\theta)) \quad (326)
\end{aligned}$$

Summary

- The purpose of log-sum-exp trick to have a numerically stable implementation of $\log \sum_{i=1}^n e^{x_i}$, by using the trick in the proof of Equation (309).
- The reason that we need $\log \sum_{i=1}^n e^{x_i}$ at all is because it is used in the cross entropy loss calculation. See the proof of Equation (325)

TODO :

- To check how does pytorch handle multi loss functions This link could be useful <https://stackoverflow.com/questions/44264443/why-should-be-the-function-backward-be-called-only-on-1-element-tensor-or-with-g>
- Maybe use my own operator AD construction to decompose it log , sum and exp operators and treat it as a Module instead of ops to test the gradients. Note Max is also a function that has derivative.

5.3 LayerNorm

LayerNorm overview There are two sets of learnable parameters in LayerNorm: gain and bias. Also per abstract in [2]: *Like batch normalization, we also give each neuron its own adaptive bias and gain which are applied after the normalization but before the non-linearity.* But really how much gain and bias are useful ? Authors in [12] argue not all all, so really shouldn't use bias. Plus, the way Kolter described in [4] that the LN motivation is to normalize the Activation output norm, which suggests the LayerNorm should be applied *after* Activation layer, is against the literature and common practice as in [2] and [12]. So who knows exactly what is going on. I am just abstracting the computational essence here (as in what is described in the homework)

Figure 6 illustrates the LayerNorm layer, typically sandwiched between Linear layer and activation layer

LayerNorm Math Let $x \in R^n$ be the output of linear layer L (aka x is the input to the LayerNorm layer), $w \in R^n$ is the gain parameter, $b \in R^n$ is the bias parameter.

$$E[x] = \frac{\text{sum}(x)}{n} \quad (E[x] \in R) \quad (327)$$

$$\text{Var}[x] = \frac{(x - E[x])^T (x - E[x])}{n} \quad (\text{Var}[x] \in R) \quad (328)$$

$$y = w \odot \frac{x - E[x]}{\sqrt{\text{Var}(x) + \epsilon}} + b \quad (329)$$

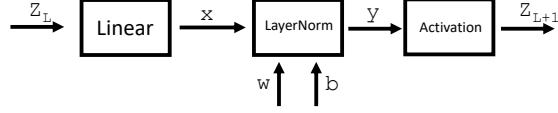


Figure 6: LayerNorm. The trainable parameters of this layer is the "gain" parameter W and "bias" parameter b , both parameters are the of the same shape of input x (assuming one training sample) As mentioned in [2] and [12], LayerNorm is typically used between Linear layer and Activation layer.

In homework, $X \in R^{m \times n}$, where m is batch size and n is the feature size. I now set $w \in R^{1 \times n}$ and $b \in R^{1 \times n}$.

$$E[X] = \frac{\text{sum}(X, \text{axis} = 1)}{n}, E[X] \in R^{m \times 1} \quad (330)$$

$$C[X] = X - \text{bcast}(E[X], X.\text{shape}), C[X] \in R^{m \times n} \text{ (centered X)} \quad (331)$$

$$\text{Var}[X] = \frac{\text{sum}(C[X] \odot C[X], \text{axis} = 1)}{n}, \text{Var}[X] \in R^{m \times 1} \quad (332)$$

$$\text{Std}[X] = \text{power_scalar}((\text{Var}[X] + \epsilon), 0.5), \text{Std}[X] \in R^{m \times 1} \quad (333)$$

$$\text{Norm}[X] = \text{divide}(C[X], \text{bcast}(\text{Std}[X], X.\text{shape})), \text{Norm}[X] \in R^{m \times n} \quad (334)$$

$$Y = \text{bcast}(w, X.\text{shape}) \odot \text{Norm}[X] + \text{bcast}(b, X.\text{shape}) \quad (335)$$

5.3.1 BatchNorm

Similar to LayerNorm, BatchNorm is between Linear and Activation Layers.

Training

Similar to LayerNorm, there are two set of learnable parameters, gain parameter $w \in R^n$, bias parameter $b \in R^n$. Since BatchNorm can only work in batches, there is no math formula for one-sample case as in Equation (327), i.e., the input X has to be at least 2D. Now, let $X \in R^{m \times n}$, where m is batch size

and n is the feature size. I now set $w \in R^{1 \times n}$ and $b \in R^{1 \times n}$.

$$E[X] = \frac{\text{sum}(X, \text{axis} = 0)}{m}, E[X] \in R^{1 \times n} \quad (336)$$

$$C[X] = X - \text{bcast}(E[X], X.\text{shape}), C[X] \in R^{m \times n} \text{ (centered X)} \quad (337)$$

$$\text{Var}[X] = \frac{\text{sum}(C[X] \odot C[X], \text{axis} = 0)}{m}, \text{Var}[X] \in R^{1 \times n} \quad (338)$$

$$\text{Std}[X] = \text{power_scalar}((\text{Var}[X] + \epsilon), 0.5), \text{Std}[X] \in R^{1 \times n} \quad (339)$$

$$\text{Norm}[X] = \text{divide}(C[X], \text{bcast}(\text{Std}[X], X.\text{shape})), \text{Norm}[X] \in R^{m \times n} \quad (340)$$

$$Y = \text{bcast}(w, X.\text{shape}) \odot \text{Norm}[X] + \text{bcast}(b, X.\text{shape}) \quad (341)$$

In addition, BatchNorm needs to keep running average of mean and variance to be use it in testing – recall there could be only one sample during testing, so cannot use mean over batch. Let $\hat{\mu} \in R^{1 \times n}$ represent the estimated mean of X and $\hat{\beta} \in R^{1 \times n}$ represent the estimated variance of X . In each training iteration, we update $\hat{\mu}$ and $\hat{\beta}$ in the following way:

$$\hat{\mu} = (1 - m)\hat{\mu} + mE[x] \quad (342)$$

$$\hat{\beta} = (1 - m)\hat{\beta} + m\text{Var}[x] \quad (343)$$

m is the momentum and per homework, $\hat{\mu}$ is initialized as 0_n^T and $\hat{\sigma}$ is initialized as 1_n^T

Inference. Assuming testing data $X \in R^{m \times n}$

$$\text{Norm}[X] = \frac{X - \text{bcast}(\hat{\mu}, X.\text{shape})}{\text{bcast}(\text{power_scalar}(\hat{\beta} + \epsilon, 0.5), X.\text{shape})} \quad (344)$$

$$Y = \text{bcast}(w, X.\text{shape}) \odot \text{Norm}[X] + \text{bcast}(b, X.\text{shape}) \quad (345)$$

Note:

- In BatchNorm, we also need to apply gain and bias in inference (I forgot about it in hw at the beginning.)
- For the BatchNorm test cases, it requires running mean and running var to be arrays instead of vectors so I had to force it by using `reshape(-1)`. It is unfortunate that those professors have this kind an assumption. Luckily, the array behaves exactly like a row vector when working with `bcast`, so I could in theory change my way of defining those learnable parameters to be just array as opposed to row vectors as of now.

5.4 Dropout Layer

Per homework, drop out layer is between activation layer and linear layer.

Dropout Math

$$\hat{z}_{i+1} = \sigma_i(W_i^T z_i + b_i)(z_{i+1})_j = \begin{cases} (\hat{z}_{i+1})_j / (1 - p) & \text{with probability } 1 - p \\ 0 & \text{with probability } p \end{cases} \quad (346)$$

p the probability of an element to be zeroed.

Implementation Note

- $*$ is the unpack operator that can convert a tuple of numbers to a sequence numbers (aka take off the parenthesis)
- note p is the probability to be zeroed, and the randb impl the parameter p is the probability to be one-ed. Thus I need to do $p = 1 - self.p$

```
def forward(self, x: Tensor) -> Tensor:
    if(self.training):
        # * is unpacking operator that converts tuple to sequence
        #   ↳ of numbers
        probs = randb(*x.shape, p=1-self.p) # p is the probability
        #   ↳ to be zeroed
        # and the randb impl is random_choice<=p,
        # thus randb(1-self.p) is the right percentage to be one-
        #   ↳ ed
        # e.g., p=0.2 means 20% will be zeroed,
        # thus 1/(1-0.2) ratio need be scaled
        return x * probs * (1/(1-self.p))
    else:
        return x
```

5.5 Residual Module

The module takes another module $\mathcal{F}$ and input Tensor x , returning $\mathcal{F}(x) + x$. So in AD framework, the ResNet literally is one-liner!!

```
class Residual(Module):
    def __init__(self, fn: Module):
        super().__init__()
        self.fn = fn

    def forward(self, x: Tensor) -> Tensor:
        ### BEGIN YOUR SOLUTION
        return x + self.fn(x)
        ### END YOUR SOLUTION
```

Do note that

- Residual connection requires $\mathcal{F}$ to have the same shape of input and output, so $\mathcal{F}$ outoupt can be added to x
- Residual connection has no trainable parameter. It is merely a way to nudge $\mathcal{F}$'s weights to all zeros if it is trying to get $\mathcal{F}(x) + x$ to be an identity function.
- `x[::-1]` means reverse tuple `x`. That is used when I was looking at the test case for Residual (see below) and double check if the $\mathcal{F}$ input and output shape are the same. `*shape[::-1]` simply means reverse shape and unpacked it to a sequence of numbers. In this case, it simply constructs two linear layers connected: `input_dim × output_dim` and then `output_dim × input_dim`, both input and output dimensions are 5 in this test case.

```
def residual_forward(shape=(5, 5)):
    np.random.seed(42)
    f = nn.Residual(
        nn.Sequential(nn.Linear(*shape), nn.ReLU(), nn.Linear(*
            ↪ shape[::-1]))
    )
    x = get_tensor(*shape[::-1])
    return f(x).cached_data
```

5.6 Flatten

Flatten is just reshape

```
class Flatten(Module):
    def forward(self, X):
        ### BEGIN YOUR SOLUTION
        shape = X.shape # weiz I am assuming X is the type of
            ↪ Tensor
        assert(len(shape) >=2)
        batch_size = shape[0]
        unit_size = reduce(lambda x, y: x * y, shape[1:], 1) # get
            ↪ the unit size, i.e, the product of all the
            ↪ elements in shape except the first one(the batch
            ↪ size dimension)
        return X.reshape((batch_size, unit_size))
```

I use `chatgpt` to get the code that computes the product of the last `n-1` elements of a tuple

5.7 SGD w/ weight decay and momentum

SGD w/ weight decay and momentum.

Step1 Apply Weight Decay to loss function: Rewrite Loss function to include the weight decay term as in Equation (347) (also see Section 5.1) and the corresponding gradient calculation in Equation (348). Note l is the original NN loss function.

$$L(\theta) = l(\theta) + \frac{1}{2}\lambda\|\theta\|_{Frob}^2 \quad (347)$$

$$\nabla_{\theta} L \stackrel{\text{Equation (306)}}{=} \nabla_{\theta} l + \lambda\theta \quad (348)$$

Step2 Apply momentum to gradients: u is the gradients history buffer, β is the momentum.

$$u = \beta u + (1 - \beta)\nabla_{\theta} L(\theta) \quad (349)$$

$$\theta = \theta - \alpha u \quad (350)$$

Note,

- This momentum implementation is a exponential smoothing of gradients, as we apply β to history and $1 - \beta$ to gradients. In the original definition of momentum, $1 - \beta$ was not applied to gradients – so it was like $u = \beta u + \nabla_{\theta} L(\theta)$ See Cornell optimization webpage at <https://optimization.cbe.cornell.edu/index.php?title=Momentum> and PyTorch implementation at <https://pytorch.org/docs/stable/generated/torch.optim.SGD.html#torch.optim.SGD>.
- Also note that PyT Nesterov implementation seems doesn't calculate gradients at the look-ahead position but rather just added (almost) gradients twice.
- And Goodfellow's Deep learning book on momentum seems get momentum sign wrong !? –Not really, in their book, they keep the running average of **negative** gradients.

One "bug" that delayed me a few hours is

```
class DivScalar(TensorOp):
    def __init__(self, scalar):
        self.scalar = scalar

    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        if(isinstance(self.scalar, int)): # weiz 2024-02-03, ugly
            ↪ hack to make the setter type check pass TODO!!
            self.scalar = a.dtype.type(self.scalar)
        return array_api.divide(a, self.scalar)
        ### END YOUR SOLUTION
```

Apparently when a float32 divided by an int in numpy, the result will be cast as float64. And the setter method in numpy requires type checking (essentially a float32 == float64 assertion failure would occur).

5.8 Adam

$$u_{t+1} = \beta_1 u_t + (1 - \beta_1) \nabla_{\theta} f(\theta_t) \quad (351)$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) (\nabla_{\theta} f(\theta_t))^2 \quad (352)$$

$$\hat{u}_{t+1} = u_{t+1} / (1 - \beta_1^t) \quad (\text{bias correction}) \quad (353)$$

$$\hat{v}_{t+1} = v_{t+1} / (1 - \beta_2^t) \quad (\text{bias correction}) \quad (354)$$

$$\theta_{t+1} = \theta_t - \alpha \hat{u}_{t+1} / (\hat{v}_{t+1}^{1/2} + \epsilon) \quad (355)$$

- θ : iterable of parameters of type `needle.nn.Parameter` to optimize
- t : t -th iteration (one bug i had was i mistakenly incremented this counter for each param in params)
- α : learning rate
- β_1 : coefficient used for computing running average of gradient
- β_2 : coefficient used for computing running average of square of gradient
- ϵ : term added to the denominator to improve numerical stability
- u : running average of gradients
- v : running average of gradients square
- $\hat{u}$: unbiased running average of gradients
- $\hat{v}$: unbiased running average of gradients square

Note I am treating weight decay exactly the same way as in SGD, and the implementation is very straightforward. I only failed the last optional memory check testing, but oh well, may come to this later when I have time.

5.9 Dataset

Random Flip Horizontal : Flip Horizontal is just make the pixels reversed along the horizontal axis. The input image is $H \times W \times C$, so we just need to reverse along W . Note each element in 3D matrix *img* will be changed in the following way:

$$img[i][j][k] = img[i][W - 1 - j][k] \quad (356)$$

That is keeping the 1st and 3rd dimension unchanged and flip the 2nd dimension. In numpy, the above-mentioned operation can be done via slicing. In particular `::-1` means reverse a sequence. So to reverse (aka flip) the 2nd dimension of *img* is just `img[:,::-1,:]`

Random Crop Two steps:

1. Apply padding p – the padded border is p pixels each side, so $H \times W \times C$ images become $(H + 2p) \times (W + 2p) \times C$.
2. Random crop $H \times W \times C$ out of $(H + 2p) \times (W + 2p) \times C$.

Dataset Just two methods need be implemented: `__getitem__`, which overloads indexing operator `[]`; and `__len__`, which allows `len()` method; therefore they are *dunder* methods.. One difference from hw0 is that I would need to reshape the image as $H \times W \times C$ (i.e., rows $\times$ cols $\times$ 1) instead of hard-coded 784, to use the image transform procedure.

The `__getitem__` returns a tuple of (data, label). data and label are both NDArrays.

TODO: Maybe refer to <https://huggingface.co/datasets/mnist> and see how HF implements it. Apparently for datasets, it returns NDArray.

Dataloader Take Dataset, which returns a tuple of NDArray arrays, and batch them. Each returned batch is a tuple (or list seems to be fine too) of data Tensors and label Tensors. One can use `np.stack()` to stack a list of NDArrays to become one big NDArray and then call `Tensor()` constructor to construt over this big NDArray. The test case also includes the case where Dataset only returns data (aka no label).

5.10 ResidualBlock

Recap:

- Norm Layer (e.g., LayerNorm or BatchNorm) input and output are of the same shape
- ReLu layer input and output are of the same shape
- Dropout Layer input and output are of the same shape
- Residual layer $x + \mathbf{F}(x)$ means $\mathbf{F}$ input and output are of the same shape. In Figure 7, first linear layer has `in_features=dim` and `out_features=hidden_dim`, and the last linear layer has `out_features=dim`.

Caveats

- BatchNorm and LayerNorm modules take input dimension as a parameter in constructor and have two trainable parameters W and b
- Dropout module takes `drop_prob` as a parameter in constructor and has no trainable parameters.
- ReLu has no constructor parameter and no trainable parameters.

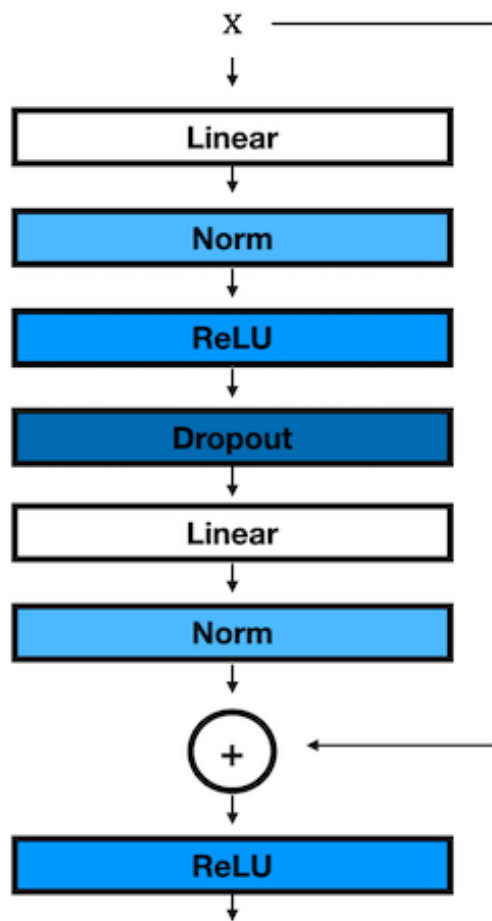


Figure 7: ResidualBlock (from CMU 10714)

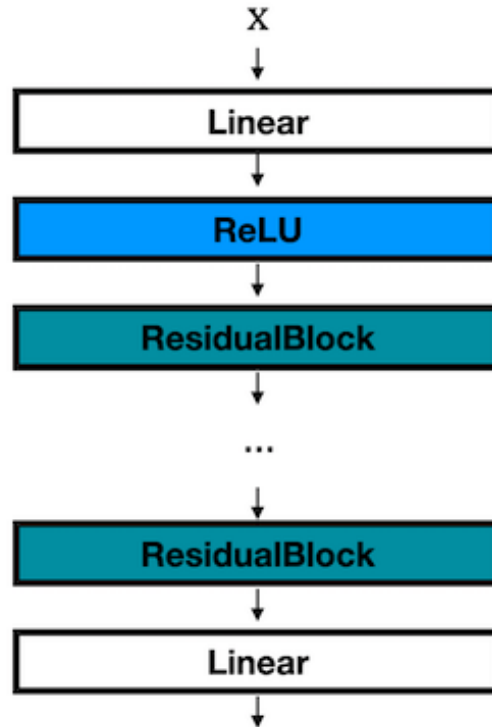


Figure 8: MLP ResNet (from CMU 10714)

Summary : The input and output to this ResidualBlock are of the same shape, because of the very last Residual Layer.

5.11 MLPResNet

Figure 8 is the full architecture.

Summary

- Use `Module` class's `parameters()` method to connect with `Optimizer` class so that optimizer can update the gradients. Also it is a good time to consider that `Parameter` class is really useful that defines the trainable parameters, not all Modules has parameters (e.g., `ReLU`, `SoftMaxLoss`, `Dropout` and `Residual`)
- Before `backward()` method, call `optimizer.reset_grad()` method to clear

out gradients. To think: Can we actually just set `grad.data` to 0 instead of setting `grad` to None as what is being done in `reset_grad()`

- Even Dataset tests require return images as 28x28x1, in the MLPResNet test, we need to flatten it to 784x1 so that it can work with linear layer.

Finally, I have finished hw2 on Feb 18, 2024!

Note: The only failed test (optional though) in hw2 is the Adam memory check test (aka `test_optim_adam_z_memory_check_1`).

6 HW3: Building an NDAarray library

6.1 Some (Given) Fundamental Functions

compact_strides `strides` is how many elements are jumped over when the corresponding dimension increases by 1. Given a `shape`, Listing 3 calculates the strides. E.g., when `shape` is (2,3), the `strides` is (3,1). When `shape` is (2,3,4), the `strides` is (12,4,1). My way: Given dimension `i`, the corresponding is `prod(shape[i+1:])`. Note that here stride is in the units of element, not in bytes (as used in numpy). I think our design is a better design than numpy in this regard. See

Listing 3: `compact_strides`

```
def compact_strides(shape):
    """Utility function to compute compact strides"""
    stride = 1
    res = []
    for i in range(1, len(shape) + 1):
        res.append(stride)
        stride *= shape[-i]
    return tuple(res[::-1])
```

Listing 4: `compact_strides`, Wei Way, note product of empty list is 1 (i.e., `prod(iter, start=1)`) and return strides as a tuple to make compatibility

```
def compact_strides(shape):
    strides=tuple([prod(shape[i+1:]) for i in range(len(shape))])
    return strides
```

make `make` allocates memory for a given `shape` and `strides`, `handle` is just the pointer.

1. `device` - An object of type `BackendDevice`, which is a simple wrapper that contains a link to the underlying device backend (e.g., CPU or CUDA).

2. **handle** - A class object that stores the underlying memory of the array. This is allocated as a class of type `device.Array()`, though this allocation all happens in the provided code (specifically the `NDArray.make` function), and you don't need to worry about calling it yourself.
3. **shape** - A tuple specifying the size of each dimension in the array.
4. **strides** - A tuple specifying the strides of each dimension in the array.
5. **offset** - An integer indicating where in the underlying `device.Array` memory the array actually starts (it's convenient to store this so we can more easily manage pointing back to existing memory, without having to track allocations).

Listing 5: The 'make' function for creating a new NDArray

```
def make(shape, strides=None, device=None, handle=None, offset=0):
    """Create a new NDArray with the given properties. This will
    allocate the
    memory if handle=None, otherwise it will use the handle of an
    existing
    array."""
    array = NDArray.__new__(NDArray)
    array._shape = tuple(shape)
    array._strides = NDArray.compact_strides(shape) if strides is
        None else strides
    array._offset = offset
    array._device = device if device is not None else
        default_device()
    if handle is None:
        array._handle = array.device.Array(prod(shape))
    else:
        array._handle = handle
    return array
```

`np.lib.stride_tricks.as_strided` Zolter mentioned his favorite numpy method `np.lib.stride_tricks.as_strided`¹⁰

To tile a matrix A of a shape 6×6

$$\begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 & 16 & 17 \\ 18 & 19 & 20 & 21 & 22 & 23 \\ 24 & 25 & 26 & 27 & 28 & 29 \\ 30 & 31 & 32 & 33 & 34 & 35 \end{bmatrix}$$

¹⁰<https://www.youtube.com/watch?v=7kclgMlcMq0&t=2866s> at 38:00–58:00

to a $3 \times 3 \times 2 \times 2$ tensor B

$$\begin{bmatrix} \begin{bmatrix} 0 & 1 \\ 6 & 7 \end{bmatrix} & \begin{bmatrix} 2 & 3 \\ 8 & 9 \end{bmatrix} & \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix} \\ \begin{bmatrix} 12 & 13 \\ 18 & 19 \\ 24 & 25 \\ 30 & 31 \end{bmatrix} & \begin{bmatrix} 14 & 15 \\ 20 & 21 \\ 26 & 27 \\ 32 & 33 \end{bmatrix} & \begin{bmatrix} 16 & 17 \\ 22 & 23 \\ 28 & 29 \\ 34 & 35 \end{bmatrix} \end{bmatrix}$$

One only need to do:

`B=np.lib.stride_tricks.as_strided(A, shape=(3,3,2,2), strides=np.array(12,2,6,1)*4)`

Note the stride (12,2,6,1) is because the first dimension jumps 1 is from 0 to 12, thus 12; 2nd dimension jumps 1 is from 0 to 2, thus stride is 2; 3rd dimension jumps by 1 is from 0 to 6, thus stride is 6; and the 4th dimension jumps by 1 is from 0 to 1, thus stride is 1. In the end multiply by 4 is simply numpy takes number of bytes instead of number of elements as strides. Now the B's stride (12,2,6,1), which is not monotonically decreasing, means B is not contiguous in memory. **In another word, to judge if an array is contiguous in memory, is very simple: if stride is monotonically decreasing, then array is contiguous in memory. Note it doesn't need to be strictly monotonically decreasing – e.g., an array of size (2,1) has a stride (1,1). This is true if no slicing step is involved, e.g., each index increment only means increment by 1; otherwise, we can have strides that is also monotonically decreasing but we actually make larger steps in each dimension, see `__getitem__` in Section 6.2.1**

If we want a contiguous memory tensor, then we need to do

`C=np.ascontiguousarray(B)`, and now C points to this array:

$$\begin{bmatrix} 0. & 1. & 6. & 7. & 2. & 3. & 8. & 9. & 4. \\ 5. & 10. & 11. & 12. & 13. & 18. & 19. & 14. & 15. \\ 20. & 21. & 16. & 17. & 22. & 23. & 24. & 25. & 30. \\ 31. & 26. & 27. & 32. & 33. & 28. & 29. & 34. & 35. \end{bmatrix}$$

This is the flattened tensor B

im2col TODO: convert an image to a column vector then stack them together to apply matrix multiplication

6.2 Loop Tiling

Baseline Matrix Multiplication Algorithm 1 illustrates the baseline matrix multiplication Note in Algorithm 1 the inner loop (line 4), assume L is number of floats (float is assumed to be 4bytes) in a cache-line. Usually one cache-line is 64 bytes, so $L = 64/4 = 16$, i.e., loading one cache-line will load in 16 floats. Then, we have

- $C[i, j]$ incurs 1 cache miss
- $A[i, k]$ incurs $\frac{N}{L}$ cache misses

Algorithm 1: Matrix Multiplication

```
1 for  $i = 0$  to  $N - 1$  do // Outer loop
2   for  $j = 0$  to  $N - 1$  do // Middle loop
3     for  $k = 0$  to  $N - 1$  do // Inner loop
4        $C[i, j] += A[i, k] \times B[k, j]$ 
```

- $B[k, j]$ incurs N cache misses, assume $N \gg L$

So the total cache misses per inner loop is

$$1 + \frac{N}{L} + N \quad (357)$$

As a result, the total cache misses of entire calculation is

$$N^2(1 + \frac{N}{L} + N) = N^2 + \frac{N^3}{L} + N^3 \quad (358)$$

Loop interchange Algorithm 2 illustrates the baseline matrix multiplication loop interchange. Reminds me Strang's Matmul 3rd way (i.e., row space)

Algorithm 2: Matrix Multiplication Loop Interchange to improve locality

```
1 for  $i = 0$  to  $N - 1$  do // Outer loop
2   for  $k = 0$  to  $N - 1$  do // Middle loop
3     for  $j = 0$  to  $N - 1$  do // Inner loop
4        $C[i, j] += A[i, k] \times B[k, j]$ 
```

We have in the inner loop,

- $C[i, j]$ incurs $\frac{N}{L}$ cache misses
- $A[i, k]$ incurs 1 cache miss
- $B[k, j]$ incurs $\frac{N}{L}$ cache misses

So the per inner loop cache misses is

$$2\frac{N}{L} + 1 \quad (359)$$

As a result, the total cache misses in this case is

$$N^2 \times (2\frac{N}{L} + 1) = 2\frac{N^3}{L} + N^2 \quad (360)$$

Apparently, Equation (360) is a lot less than Equation (358). **To ask: what type of dependence analysis we can do to ensure that the loop interchange is safe ?**

Blocked Matrix Multiplication

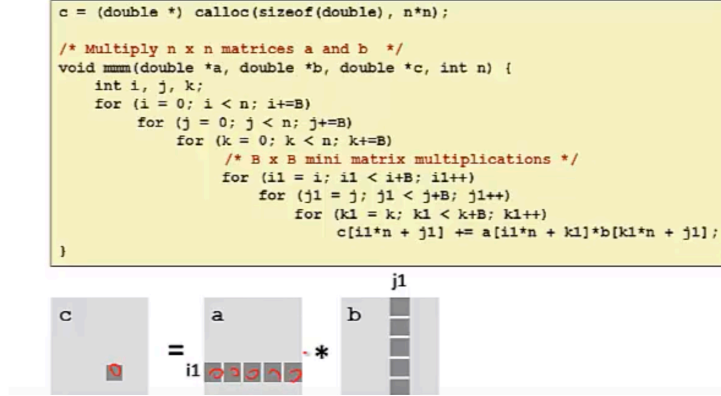


Figure 9: Luis Ceze tiling matrix multiplication

Loop tiling Loop tiling takes me almost 2 weeks to figure out!!! The best video that I found for loop tiling is Luis Ceze's at https://www.youtube.com/watch?v=b_js5-gkZUI Figure 9 is how Luis Ceze interprets loop tiling in matmul.

Tiling can be thought of as block matrix multiplication

Assumption,

- tiling size is $T \times T$, T is how many floats in one dimension of the tile.
- Most 3 tiles fit in the Cache (that is a tile of A, a tile of B and a tile of C)

Memory use analysis There are $\frac{N}{T} \times \frac{N}{T}$ tiles in each A, B, and C. When loading a tile from A, the number of cache misses is $\frac{T}{L} \times T = \frac{T^2}{L}$; when loading a tile from B, the number of cache misses is also $\frac{T^2}{L}$. Then to calculate a tile in C we need $\frac{N}{T}$ tiles from A and $\frac{N}{T}$ tiles from B, thus per inner loop of the tile-level (line4-line10 in Algorithm 3), we have cache misses of

$$\frac{N}{T} \times \frac{T^2}{L} + \frac{N}{T} \times \frac{T^2}{L} = 2 \frac{NT}{L} \quad (361)$$

There are total $\frac{N^2}{T^2}$ tiles in C, so there are $\frac{N^2}{T^2}$ calls to the inner-loop (at tile-level) as of Equation (361). As a result, we have total cache misses in tiling of matmul as in Equation (362). Note we have omitted the cost of bringing in tile $C[ii][jj]$, as we only pay this cost quite infrequently (i.e., after one complete

Algorithm 3: Tiling Matrix Multiplication. This algorithm can be called either Tiling Matrix Multiplication (memory perspective) or Block Matrix Multiplication (math perspective)

Input : A, B (input matrices), C (output matrix), N (size of matrices), T (block size)

Output : Product matrix $C = AB$

```

1 for  $ii = 0$  to  $N$  by  $T$  do
2   for  $jj = 0$  to  $N$  by  $T$  do
3     for  $kk = 0$  to  $N$  by  $T$  do
4       for  $i = ii$  to  $ii + T - 1$  do
5         for  $j = jj$  to  $jj + T - 1$  do
6           for  $k = kk$  to  $kk + T - 1$  do
7              $C[i, j] += A[i, k] \times B[k, j]$ 
8           end
9         end
10      end
11    end
12  end
13 end

```

tiling-level inner loop is done).

$$2 \frac{NT}{L} \frac{N^2}{T^2} = 2 \frac{N^3}{LT} \quad (362)$$

We can see Equation (362) is a lot better than either Equation (358) or Equation (360)

Note one (as myself) might be tempted to think that we could have $A[ii][jj]$ tile stay in the cache, and then iterate over $B[jj][kk]$, where $kk=1..N$ (aka Strang's matmul row-space way at the tiling-level), in that way the tile $A[ii][jj]$ can stay in cache for a long time and we only pay the cost of bring-ing in the tile $B[jj][kk]$. This is sorta like Algorithm 2 but at the tiling-level. The problem is that $C[ii][kk]$ now also have to be brought in cache more frequently (as now the inner loop of the high-level tiling would become $C[ii][kk] += A[ii][jj] * B[jj][kk]$ for $kk=1..N$, than its counterpart of $C[ii][jj] += \sum A[ii][kk] * B[kk][jj]$ as in the vanilla tile matrix multiplication. It is a wash in the end – it is either we bring in $A[ii][jj]$ and $B[jj][kk]$ while pinning $C[ii][jj]$ in cache each time as in vanilla version or we bring in $B[jj][kk]$ and $C[ii][kk]$ while pinning $A[ii][jj]$ in cache each time as my "smart version". As in the assumption, only 3 tiles can fit in cache. It took me a while to figure this out!

TODO: why block matrix multiplication makes sense in math, use the inner product of each row and column and be cut in disjoint pieces to prove.

Finally, matmul tiling is cleared up on Mar 17, 2024!!! – two weeks of thinking!

6.2.1 Python array operations

reshape() The memory is still contiguous, we just need to calculate strides based on new shape by calling

```
new_strides = NDArray.compact_strides(new_shape)
```

Note shape and strides are both tuples.

permute() The memory is not going to be contiguous. We need to permute both the shape and strides. To think why we need to permute strides is because we will access the new axis using the same strides as the permuted axis.

Listing 6: The permute() function

```
def permute(self, new_axes):
    permuted_shape = tuple(self._shape[i] for i in new_axes)
    permuted_strides = tuple(self._strides[i] for i in new_axes)
    return NDArray.make(permuted_shape, strides=permuted_strides,
        ↪ device=self._device, handle=self._handle)
```

Note, when I was doing homework in Section 6.5, I came to realize that **permute()** is actually not a numpy method, the counterpart in numpy is the **transpose()** method which also takes in a list of axes which is a permutation of axes. To permute (or transpose) back to the original tensor **a** and axis permutation **axes**, one can do this `a = np.transpose(a.transpose(axes), np.argsort(axes))` as shown in <https://numpy.org/doc/stable/reference/generated/numpy.transpose.html>

broadcast_to() After **bcast**, the memory is not going to be compact (or contiguous) as the shape and strides no longer has the nice relationship as in Listing 4, because we are going to set the **bcast**-ed dimension stride as 0 – and this is the cool trick! When we set the **bcast** dimension to 0, it means whatever offset on that dimension will have offset 0 in memory thus having the same element as the first element (indexed by 0 along that dimension) in that dimension. **weiz 2024-10-26. Realized that in this course, stride=0 trick is only played at the python level, so when one really needs to do arithmetic operations (e.g., elementwise add), one needs to call a compact() call so that a new (bigger) tensor is created (See below __getitem__() and __setitem__() are used to assign elements in the newly created tensor. In contrast, in numpy internal implementation, the stride=0 trick is all the way played down in the C implementation (e.g., the ufunc call.**

__getitem__() **weiz 2024-06-24, together with __setitem__(), these two methods are dunder methods for array access (w/ slicing).** The input is processed to produce the **slices** tuple. **slices** tuple has the same length as NDArray tensor dimensions. Each corresponding dimension **slice** (a builtin python data structure) has three fields: **start**, **stop** and **step**. Note the range of the slice is **[start, stop)**.

The goal of `__getitem__()` is to simply Make the Array with the right **shape**, **strides** and **offset**. Note `shape[dim]` is simply the number of elements in that dimension `dim`, `strides[dim]` is simply the original `strides[dim]` multiply with the `slices[dim].step`. `offset` is the offset of the first element of the `getitem` Array A (e.g., the offset of `A[slices[0].start][slices[1].start]...[slices[dim-1].start]`)

For each dimension i ,

$$\begin{aligned} \text{shape}[i] &= \text{math.ceil}((\text{slices}[i].\text{stop} - \text{slices}[i].\text{start}) / \text{slices}[i].\text{step}) \\ \text{strides}[i] &= \text{self}._\text{strides}[i] * \text{slices}[i].\text{step} \end{aligned}$$

$$\text{offset} = \text{sum}([\text{slices}[i].\text{start} * \text{self}._\text{strides}[i] \text{ for } i \text{ in range}(\text{len}(\text{slices}))])$$

Note, because `step` might involve with `strides`, even if `strides` are monotonically decreasing, the memory could be still dis-contiguous as indices along the same dimension can jump – see `np.lib.stride_tricks.as_strided` in Section 6.1

6.3 CPU Backend - Compact and setitem

This supposedly a hard question. Note I am using compact to mean memory contiguous and loose to mean memory dis-contiguous. **Update 2024-5-20:** Note compact tensor and loose tensor of the same shape have the exact same memory footprint, loose tensor doesn't mean there is a hole in their memory region it simply means access of elements is not sequentially row major.

1. dimension/shape goes from 0 to N-1, *the higher dimension the smaller the stride!* – sometimes I got confused. **weiz 2024-06-15**, I just came realize, when re-examing **`permute()`** function for HW4, that tensor, axes(), indexing is big-endian – the most significant dimension (e.g., dimension 0, which spans across the largest memory region) lies on the left-most (think about a number line – 0 is the left most, compared to 1, 2, ...) . On a side note, `str="abcd"`, `str[0]='a'`, I was tempted to think that `str[0]='d'`. Fundamentally, it is a struggle between we write numbers from left to right, right being the least significant; whereas number line grows from left to right, right being the most significant. **weiz 2024-10-26**: Another example would be shape (3,4,5), then `shape[0]=3` (most significant bit is at address 0, thus big-endian), `shape[1]=4`, `shape[2]=5`
2. **weiz 2024-10-26** Major issue of my understanding is little-endian (x86, modern) vs big-endian (IBM, old). E.g., 0x12345678, Little-endian: 0x12 (at memory address 03), 0x34 (02), 0x56 (03), 0x78 (04). Little-endian reads very natural to me as a CS student. Big-endian 0x12(00), 0x34(01), 0x56(02), 0x78(03) Many people argue big-endian is more human-readable, as number line grows from left to right, however we write numbers with most significant bit at the left. But I honesely feel little-endian is more intuitive, so as some other people

3. I used a vector repr to represent the indices of each element in a tensor
4. I start from the largest offset and **decrement** so that I don't have to always look up a map to decide if i am going across the boundary (i can simply check if it is lower than 0)

Compact() Map from loose tensor *src* to a compact tensor *dst*.

Setitem() Map from compact tensor *src* to a loose tensor *dst*

A use-case for Compact: update 2024-05-21.

```
A = np.arange(3 * 4 * 5).reshape(3, 4, 5) # A is a compact tensor
B  $\stackrel{\text{Setitem}()}{=}$  A.permute(1, 2, 0) # B is a loose tensor
C  $\stackrel{\text{Compact}()}{=}$  np.ascontiguousarray(C) # C is a compact tensor
```

In Essence , it is just two mappings: (1) Compact(): map from loose strides to compact strides (2) Setitem(): map from compact strides to loose strides. The bridge of mappings is the traversal of shape space. See below

Given a shape, we just want to explore from $[0, \dots, 0]$ to $[\text{dim1}-1, \text{dim2}-1, \dots, \text{dimN}-1]$ space. We could have used a recursive call to explore this tree (aka shape space). Here I use a stack , represented by **repr**, to explore the shape space. To make the bookkeeping easier, I "reversely" explore this space, i.e., start from $[\text{dim1}-1, \text{dim2}-1, \dots, \text{dimN}-1]$ to $[0, \dots, 0]$, so that I only need to check if any of the dimension goes under 0 to mark the exhaustion (otherwise I would always have to look up the upper-bound of each dimension). I have defined following helper functions.

shape space : Given a shape $(\text{dim}_0, \text{dim}_1, \dots, \text{dim}_{N-1})$, exhaust from $[0, \dots, 0]$ to $[\text{dim}_0-1, \dots, \text{dim}_{N-1}-1]$, in total $\text{dim}_0 \times \text{dim}_1 \times \dots \times \text{dim}_{N-1}$ elements.

get_compact_strides : return strides for compact tensor, given tensor's shape, the same as Listing 4

fill_tensor_at : Given strides and pointers of dst and src tensors, index representation and an optional source value to fill a tensor at the index representation

Listing 7: Given strides and pointers of dst and src tensors, index representation and an optional source value to fill a tensor at the index representation

```
void fill_tensor_at(scalar_t * dst_ptr, const scalar_t * src_ptr,
                  std::vector<int32_t> & dst_strides, std::vector<
                      ↪ int32_t> & src_strides,
```

```

        std::vector<int32_t> & repr, scalar_t src_val=0){
    }

```

reload : called when one of the dimensions is exhausted in the searching, need to backtrack to the lower dimension, decrement by 1 and reload the following dimensions in *repr* vector to the largest number possible (aka, $dim_{n+1} - 1$).

Listing 8: called when the last dimension is exhausted, return the dimension where it will start exploring, -1 if the entire repr space is explored

```

int reload(std::vector<int32_t> & repr, std::vector<int32_t> &
    ↪ shape){
}

```

tensor_assign : the core function. Given the source info (ptr and strides) and dest info (ptr and strides), explore the shape **repr** space and assign values. The core of the function is to use **repr** vector as a stack (to keep a track of exploring space) and traverse the entire shape space.

Listing 9: Given the source info (ptr and strides) and dest info (ptr and strides), explore the shape repr space and assign values.

```

void tensor_assign(scalar_t *dest_ptr, const scalar_t *src_ptr,
    ↪ size_t dest_offset, size_t src_offset,
        std::vector<int32_t> dest_strides, std::vector<
            ↪ int32_t> src_strides,
        std::vector<int32_t> shape,
        scalar_t src_val = 0
    ){
}

```

In the end, we just need to prepare the src and dst ptr and strides and then call **tensor_assign** function, for functions **Compact()** and **Setitem()**.

6.4 CPU Backend - Elementwise and scalar operations

There are three type of functions to implement as a template type:

Listing 10: Elementwise function that takes two vectors. E.g., **EwiseAdd**, **EwiseDiv**, **EwiseMaximum**, **EwiseEq**, **EwiseGe**

```

template <typename F>
void EwiseFunc(const AlignedArray& a, const AlignedArray& b,
    ↪ AlignedArray* out, F f){
    assert(a.size == b.size);
}

```

```

    assert(a.size == out->size);
    for (size_t i = 0; i < a.size; i++) {
        out->ptr[i] = f(a.ptr[i], b.ptr[i]);
    }
}

```

Listing 11: Elementwise function that takes one vector and one scalar. E.g., ScalarMul, ScalarDiv, ScalarPower, ScalarMaximum, ScalarEq, ScalarGe

```

template <typename F>
void ScalarFunc(const AlignedArray& a, scalar_t val, AlignedArray
    ↪ * out, F f){
    assert(a.size == out->size);
    for (size_t i = 0; i < a.size; i++) {
        out->ptr[i] = f(a.ptr[i], val);
    }
}

```

Listing 12: Elementwise function that takes one vector. E.g., EwiseLog, EwiseExp, EwiseTanh

```

template <typename F>
void SingleEwiseFunc(const AlignedArray& a, AlignedArray* out, F
    ↪ f){
    assert(a.size == out->size);
    for(size_t i = 0; i < a.size; ++i){
        out->ptr[i] = f(a.ptr[i]);
    }
}

```

Note `./python/needle/backend_ndarray/ndarray_backend_numpy.py` is where the python prototypes are defined, and we can tell explicitly about how many input vectors are taken (e.g., for EwiseLog, EwiseExp, EwiseTanh cases where the professors' notes are not clear).

May ask Dave Grove about if I can even save all these function stubs.

6.5 Part 4: CPU Backend - Reductions

Recall what does `Reduction()` over do ?

$$Result[..., 0, ...] = \sum_{i=0}^{i=L-1} Tensor[..., i, ...] \quad (363)$$

0 and i correspond to the axis reduced over, L is the length of the axis reduced over. $..., ...$ means the entire hyper-plane other than the axis reduced over. One

can think of as an exhaustive state exploration of all the touched dimensions
 In this section, we will use a running Example, say a 3-D tensor A of shape (3,4,5) and the axis to reduce over is 1. The reduction result will be of (3,1,5), i.e., reduce a cube to just xz plane.

In order to understand what do `ReduceMax()` and `ReduceSum()` do and especially what does `reduce_size` mean, let's look at their definitions in their numpy-backend impl.

In `ndarray_backend_numpy.py`, there is the numpy version of `reduce_max()`:

Listing 13: Numpy version of `reduce_max()`

```
def reduce_max(a, out, reduce_size):
    out.array[:] = a.array[:].reshape(-1, reduce_size).max(axis=1)
```

In Listing 13, we can see `reduce_size` is how many elements the reduction function reduce over for a given result tensor element – i.e., the length of the dimension to reduce over. In our running example, a 3-D tensor A of shape (3,4,5), if the axis to reduce over is 1, then the `reduce_size` is 4.

And the whole logic to prepare tensor before calling into `reduce_max()` is

1. Call `reduce_view_out()`, which is **not** a numpy function but a helper function written by Chen&Zolter and is implemented in `./python/needle/backend_ndarray/ndarray.py`. `reduce_view_out()` permutes the tensor so the axis to reduce over is moved to the last one axis of the tensor, but all the other remaining axes remain in the same order (sorta like selection sort). `reduce_view_out()` returns two tensors – `view` and `out`. `view` is the permuted tensor that has the axis to reduce over as the last axis. `out` is a tensor that has the same shape as the resulting tensor (i.e., the corresponding reduced axis length is 1 where as all the other dimensions are the same as `view`). In our running example, `out` would be of shape (3,5,4), and `view` would be of shape (3,1,5).
2. `view` will be `Compacted()` Section 6.3 to have contiguous memory. `out` is a compact tensor by construction.
3. `reduce_size` is simply the length of the last dimension of `view` (aka the original dimension of the axis to reduce over). Reminder this homework only asks to reduce over 1 axis. In general, `reduce()` simply makes a tensor of shape $(dim_1, \dots, dim_k, \dots, dim_N)$ to $(dim_1, \dots, 1, \dots, dim_N)$, suppose k is the axis that we reduce over (when we set `keepdims=True`).
4. The reason that `reduce_view_out()` permutes the axis to the last is to see the next paragraph.

Crux of the problem Per homework description, *Because summing over individual axes can be a bit tricky, even for compact arrays, these functions (in Python) in Python simplify things by permuting the last axis to be the one*

reduced over¹¹ (this is what the `reduce_view_out()` function in `NDArray` does), then compacting the array.

The difficulty of this homework does not lie in the actual implementation, but in understanding how the pre-processing is done by the professors, especially the purpose of function call `reduce_view_out`. Updated:2024-05-19, Look at `ndarray.py:L582`, we know **out** is the final result returned to user after reduction is done.

Listing 14: Chen&Zotler implementation of `reduce_view_out`, `view` is the backend storage of the permuted original tensor, `out` is the backend storage of final result. Both `view` and `out` are contiguous tensors.

```
def reduce_view_out(self, axis):
    """Return a view to the array set up for reduction functions
    ↪ and output array."""
    if axis is None:
        view = self.reshape((1,) * (self.ndim - 1) + (prod(self.
            ↪ shape),))
        out = NDArray.make((1,) * self.ndim, device=self.device)
    else:
        if isinstance(axis, (tuple, list)):
            assert len(axis) == 1, "Only support reduction over a
            ↪ single axis"
            axis = axis[0]

        view = self.permute(
            tuple([a for a in range(self.ndim) if a != axis]) +
            ↪ (axis,)
        )
        out = NDArray.make(
            tuple([1 if i == axis else s for i, s in enumerate(
                ↪ self.shape)]),
            device=self.device,
        )
    return view, out
```

In our running example, `A` starts with shape (3,4,5), after `reduce_view_out()`, the returned tensor `view` is of shape (3,5,4). `view` is a compact tensor. Then we can reduce over the last axis of `view` to get `reduced_view`¹² of shape (3,5,1), then we can do a `permute()` on `reduced_view` to make it (3,1,5), which would be the final result. In my opinion, this would be the most logically clear way of handling the whole `Reduce()` call. Downside is `permute()` call make it tensor loose, and we will need to call a compact call. Note my mis-understanding was

¹¹Wei's comments: This sentence is utterly misleading, it is supposed to mean permute the axis reduced over to be the last axis.

¹²`reduced_view` is defined by me, which is reduction result of `view` along the last axis. I find this definition is useful for understanding this problem.

+	+	*	-	-
+	+	*	-	-

Tensor 2: Original Tensor A. * represents the chosen index to reduce over. + represents all the lower dimensions than * and - represents all the higher dimensions than *.

+	+	-	-	*
+	+	-	-	*

Tensor 3: **view** tensor after `reduce_view_out(A)`. * is permuted to the highest dimension. + remains at the original dimension, - moved lower down (or left) the dimension by 1; but the key part is the ordering of other 4 dimensions + + - - remain the same.

that I thought the reduced axis and the last axis swapped, in fact, it is just reduced axis moving to the last axis, whereas all the following dimensions shifted to the left by 1. To permute **reduced_view** back to the original tensor shape, we shift all the reduction-axis-following dimensions to the right by 1 and move the last axis back to the original reduction-axis.

In the actual implementation, **out** of shape (3,1,5), which is of the desired result shape, is created to retrieve the reduction result of **view**. It is as if **out** is the reshape of **reduced**. But from last paragraph, we know **out** is supposed to be the permute of **reduced**. In general reshape of a tensor is not equivalent to permute of a tensor, even the resulting shape is the same. However, because this is a special case: the reduced axis ends up with length 1, permutation is to permute the last axis (length 1) back to the original axis; reshape is just to add length 1 extra dimension into the original axis while annihilate the last dimension (length 1). As a result they are equivalent. We can rigourously prove in the following:

Now we simply need to prove that **out** and **result** are the same. That is to say given any indices tuple (i,j,k,l), to prove `out[i,j,0,k,l]==result[i,j,0,k,l]`.

First, by construction, we know `result[i,j,0,k,l] = reduced_view[i,j,k,l,0]` as the former is to permute back to the original shape¹³. By the definition of `permute()`, we have `result[i,j,0,k,l] == reduced_view[i,j,k,l,0]`.

Second, we prove `out[i,j,0,k,l] == reduced_view[i,j,k,l,0]`. **out** and **reduced_view**

¹³by doing `permute(reduced_view, argsort(axes))`. In this instance, `axes=(0,1,3,4,2)`.

+	+	0	-	-
---	---	---	---	---

Tensor 4: **out** tensor after `reduce_view_out(A)`. * corresponding dimension has a size of 1. Eventually, this tensor's ptr is populated by reduce every `reduce_size` elements of ptr to the **view** tensor

+	+	-	-	0
---	---	---	---	---

Tensor 5: `reduced_view` tensor after reduction is done over the highest dimension of `view` tensor.

+	+	0	-	-
---	---	---	---	---

Tensor 6: permute `reduced_view` to move the last axis back to the * position, I named it `result`

essentially point to the same pointer ¹⁴. **Key insight is the strides of i, j, k, l are exactly the same** in `out` and `reduced_view` even they are on the different axes for k and l, that is because the * dimension in `out` has size 1 and the last dimension in `reduced_view` has 1, which make k's strides l and l's strides 1. By the same arguments, we can see all in Tensor 4 and Tensor 5 "+" have the same strides and "-" have the same strides. As a result, any legal realization of indices at corresponding "+" and "-" positions in Tensor 4 and Tensor 5 will have the same value. **Note this paragraph can be intuitively explained as: if we `squeeze()` ¹⁵`reduced_view` and `out`, they will be exactly the same tensor. The trick is simply to `unsqueeze()` this backend-storage tensor to add the proper dimension.**

Reduce Implementation Once we know the relationship between `out`, `view` and `reduced_view`, the actual implementation of Reduction is quite simple, and I can template it as shown in Listing 15, **a is the `view` tensor and `out` is the `out` tensor as in Listing 14**. Populate `out[i, j, 0, k, l] = reduced_view[i, j, k, l, 0]`.

Listing 15: Reduce function, where F is the reduction operator (e.g., max or sum), I is the reducer Identity (e.g., 0 for sum, minimum for max, so that `any=reduce(any, ID)`)

```
template<typename F, typename I>
void ReduceTemplateFunc(const AlignedArray& a, AlignedArray* out,
    ↪ size_t reduce_size, F reduce_func, I reduce_id){
    assert(a.size == out->size * reduce_size);
    size_t out_ptr_idx=0;
    for(size_t i = 0; i < a.size; i+=reduce_size){
        scalar_t _reduce_result = reduce_id;
        for(size_t j=i; j < i+reduce_size;++j){
            _reduce_result = reduce_func(_reduce_result,
                ↪ a.ptr[j]);
        }
        out->ptr[out_ptr_idx] = _reduce_result;
        out_ptr_idx += reduce_size;
    }
}
```

¹⁴`out`'s ptr is populated by reading from `view`'s ptr when reducing over `view`'s last dimension, `reduced_view` is the reduction result of `view` over the last dimension. So one can think of `out` and `reduced_view` point to the same pointer.

¹⁵`squeeze()` means remove the dimension of length 1. This will reduce the tensor dimension by 1, adjust its shape accordingly by removing the gone dimension, while not having any effect on underlying memory

```
        }  
        out->ptr[out_ptr_idx++] = _reduce_result;  
    }  
}
```

Reduce over multiple axes Even though the hw didn't ask us to do it. I feel we can simply decompose the question to reduce over axis one by one. Say we can 3 axes to reduce over, we just call `Reduce()` method 3 times, each time with the corresponding axis passed in.

Final note

- I find my proof of equivalent of `reduced_view` and `out` by involving strides calculation and the fact that `i,j,k,l` corresponding strides are exactly the same quite interesting and clever.
- I think in the end it is really have the backend reduction result storage (`i,j,k,l`) and `reduced_view` is `unsqueeze()` this storage to add the 5th dimension in the end (aka, (`i,j,k,l,1`)); whereas `out` is `unsqueeze()` this storage to add the dimension in the original reduced axis position (aka, (`i,j,1,k,l`))
- I spent 3 or 4 days !!! understanding this whole permuting original tensor (by moving the reduction-axis to the last axis) to get `view` and then `out`, with the proper shape, is populated by reduce `view` over its last axis and why then `out` is the desired result.

6.6 CPU Backend - Matrix multiplication

Matmul() Very easy, the only mistake that I made was <https://github.com/weidotwisc/10714/commit/9966233854cd2412d6cd847fc069416bf86f446e>

Tiled Matmul prep-processing Listing 16 shows how to restride the matrix without copying memory.

Listing 16: `as_strided`, used for tiling, reshape the underlying ptr with the given shape and strided

```
def as_strided(self, shape, strides):
    """Restride the matrix without copying memory."""
    assert len(shape) == len(strides)
    return NDArray.make(
        shape, strides=strides, device=self.device, handle=
        ↪ self._handle
    )
```

Listing 17: tiling logic

```
m, n, p = self.shape[0], self.shape[1], other.shape[1]

# if the matrix is aligned, use tiled matrix
↪ multiplication
if hasattr(self.device, "matmul_tiled") and all(
```

```

    d % self.device.__tile_size__ == 0 for d in (m, n, p)
):

def tile(a, tile):
    return a.as_strided(
        (a.shape[0] // tile, a.shape[1] // tile, tile,
         ↪ tile),
        (a.shape[1] * tile, tile, self.shape[1], 1),
    )

t = self.device.__tile_size__
a = tile(self.compact(), t).compact()
b = tile(other.compact(), t).compact()
out = NDArray.make((a.shape[0], b.shape[1], t, t),
                  ↪ device=self.device)
self.device.matmul_tiled(a._handle, b._handle, out,
                        ↪ _handle, m, n, p)

return (
    out.permute((0, 2, 1, 3))
    .compact()
    .reshape((self.shape[0], other.shape[1]))
)

```

I should look closely at Section 6.1 especially the `np.lib.stride_tricks.as_strided` paragraph.

As in the same paragraph, the teacher-provided pre-processing and post-processing actually is the hardest part to comprehend (so the heavy-lifting job really has been done by the teachers). I have written a jupyter notebook to demonstrate how to tile a matrix to send to tiled-matmul and how to convert it back at https://github.com/weidotwisc/10714/blob/main/hw_code/weiz/tiling.ipynb

Once we understand the pre-process and post-processing. The actual coding of TiledMatMul is quite straightforward.

AlignedDot() is just a regular matrix multiplication function, the inputs and output are all TILE by TILE matrices. I find interesting things are : *Specifically, the `__restrict__` keyword indicates to the compiler that `a`, `b`, and `* out` don't have any overlapping memory (which is necessary in order for vector operations to be `*` equivalent to their non-vectorized counterparts (imagine what could happen otherwise if `a`, `b`, `*` and `out` had overlapping memory). Similarly the `__builtin_assume_aligned` keyword tells the `*` compiler that the input array will be aligned to the appropriate blocks in memory, which also `*` helps the compiler vectorize the code.*

Qualifier Keyword	Callable From	Executed On	Executed By
<code>__host__</code> (default)	Host	Host	Caller host thread
<code>__global__</code>	Host (or Device ¹⁶)	Device	New grid of device threads
<code>__device__</code>	Device	Device	Caller device thread

Table 7: CUDA C keywords for function declaration (Page 39 of [7])

MatmulTiled() This is just block matrix multiplication, and treat each block(tile) as one element and call into `AlignedDot()` function for each block(I,J). Note the only thing worths thinking is given (I,J) of a Blocked-Matrix A, whose shape is $M \times N$ (the original matrix is a, whose shape is $m \times n$, $m = M \times \text{TILE}$, $n = N \times \text{TILE}$, how to retrieve the starting pointer of $A[I,J]$. Note, that $A[I,J]$ is really the $(I \times N + J)$ -th tile, and each tile is of TILE by TILE size. As a result, the pointer of A is defined in Equation (364)

$$\text{ptr}(A[I, J]) = \text{ptr}(A) + (I \times N + J) \times \text{TILE} \times \text{TILE} \quad (364)$$

Final note , this is done on May 10, 2024. So finally, I have finished all the CPU related numpy implementation (hopefully!) This section's most difficult part is the pre and post processing of tiling matrix, which is done by the professors. I have written to Tian, Ewout and Alex to ask if they have some theory pointers to prove sequence of numpy operations are the same.

6.7 GPU programming

I learnt most of the content from [7].

6.8 Part 6: CUDA Backend - Compact and setitem

1. Interface between host and device code can only pass by values, thus pointers (unless we manage by ourselves via `cudamalloc` and etc) cannot be passed around. Also `CudaVec` is needed due to the lack of support of `std::vector` in CUDA.
2. The reason that `CudaArray`'s pointer field can be passed between host and device code is because we `cudamalloc` `CudaArray`'s pointer field, thus it is recognizable in device code.
3. One interesting thing about CUDA programming is that we can launch threads by either the input's dimension (e.g., `ElementwiseSetitem`) or use the output's dimension (e.g. `Matrix Multiplication`)
4. The APIs design from the professors is very bad. One only need to implement a function as in Listing 18

¹⁶When dynamic parallelism is supported

Listing 18: Each GPU thread calls this function to fill a tensor at the corresponding slot. Given a gid (aka rank), first find its co-ordinate representation (has the same size as shape). With the repr, one can find the corresponding src and dst offset (via inner product of repr and corresponding strides) then assign src to dst.

```
__device__ void fill_tensor_at(scalar_t *dst, const scalar_t *src,
    ↪ CudaVec dst_strides, CudaVec src_strides,
    CudaVec shape, size_t gid, scalar_t val=0){
    CudaVec repr;
    repr.size = 0;
    int rank = gid;
    int divisor = 1;
    // step 1 get the repr vector
    for(size_t i = 1; i < shape.size; ++i){
        divisor *= shape.data[i];
    }
    // populate repr so that repr and shape have the same size
    while(repr.size < shape.size){
        int quotient = rank / divisor;
        rank = rank % divisor;
        assert(repr.size < MAX_VEC_SIZE);
        repr.data[repr.size++] = quotient;
        divisor = divisor / shape.data[repr.size];
    }
    // step 2 get the corresponding index of src and dst and then
    ↪ assign src to dst
    size_t dst_idx = 0;
    for(size_t i = 0; i < repr.size; ++i){
        dst_idx += repr.data[i] * dst_strides.data[i];
    }
    if(src != NULL){
        size_t src_idx = 0;
        for(size_t i = 0; i < repr.size; ++i){
            src_idx += repr.data[i] * src_strides.data[i];
        }
        dst[dst_idx] = src[src_idx];
    }else{
        dst[dst_idx] = val;
    }
}
```

6.9 Part 7: CUDA Backend - Elementwise and scalar operations

How to pass a function as a cuda kernel parameter? This seems hard: <https://stackoverflow.com/questions/31057870/passing-host-function-as-a-function-pointer-in-global-or-device-function/31058123#31058123> <https://forums.developer.nvidia.com/t/how-can-i-use-device-function-pointer-in-cuda/14405/8> is one solution to have a switch-case style, indexed into a function pointer array. On the other hand, Tong suggested template specialization as an alternative. E.g.

```
/*
template < typename T,0> twoOperandsFuncWrapper(T input1, T
    ↪ input2) {
    EwiseFunc<T>(input1, _mul)
}
*/
```

Also per Tong, the functions at interface between host and device can only pass by value, not by pointer (as pointer address in host will not make sense in device and vice versa).

Also note that we almost always need to check if the gid is within the bounds, because threads can be oversubscribed (as it is likely a multiple of 32)

This might help https://corecppil.github.io/Meetups/2018-08-18_Core-C++TLV/CUDACpp.pdf

In the end, I just use `enum` and `__device__` function pointer arrays to implement this.

6.10 Part 8: CUDA Backend - Reductions

First review Section 6.5

1. The number of threads to launch is `out->size`
2. `view->size = out->size × reduce_size`
3. Because of the `reduce_view_out()` in Section 6.5, we simply need to make each element in `out` to be the reduction of the corresponding `view`'s `reduce_size` elements. `out` and `view` are already laid out in the nice 1-D sequential row-major memory region.
4. This section can only pass tests only when Section 6.8 is done as `reduce_view` requires `Compact()` and `Setitem()` functions are done.

6.11 Part 9: CUDA Backend - Matrix multiplication

1. 2D launching grid make sure the `griddim.x * thread_num_x = X`, `griddim.y * thread_num_y = Y` **I had a bug that actually passed test cases**

(because test cases matrix are too similar to square matrices): I first calculated number of blocks needed and then made a smallest containing square grids that covers these blocks. This is wrong, as in the kernel code we cannot recover the proper shape of the original matrix: see <https://github.com/weidotwisc/10714/commit/598fd791512bb377655bd5a00e02aaaca213258f>

2. Even [7] argues to use y to represent M and x to represent N for a $M \times N$ matrix. It is perfectly fine to use x to represent M and y to represent N so that it is more comfortable for a programmer – we simply swap the x, y axis definition.
3. **TODO: a tiled matmul in CUDA**

Finally, this hw3 is done on May 23, 2024, after roughly 2 months of work! I have also made a dyce development environment.

7 HW4: CNN, RNN and LSTM

7.1 Software architecture of the whole dlsys homework sets 1-4

I feel this is a good place to recapture the software architecture and that will be useful if later on I decide to introduce a Triton backend for the final project.

7.2 python/needle/backend_ndarray/ndarray.py

This one really is my own impl of numpy library, and it has two backends CPU and GPU. This one bridges the C world and Python world and will be exported as `array_api`. Look at the C++/CUDA code where I register C++ impl for their python counter parts (e.g., `src/ndarray_backend_cpu.cc`'s `m.def("fill", Fill);`). Several things to note:

1. `ndarray.py` has a very slim core (Most important methods being `make` and `compact_strides`).
2. `ndarray.py` In hw3, we were asked to implement `reshape`, `permute`, `broadcast_to` and `__getitem__` (weiz: maybe even `__setitem__` ?? but it is not mentioned in hw4 description). Note they were solely dependt on the `Make()` method in `ndarray.py`. These methods would be useful for implementing full-blown Auto-diff ops. **weiz: 2024-06-09, I believe this file will be the file that I will need to modify when I introduce Triton based impl.**
3. `ndarray.py` Note all the functions like `__truediv__` (which is how operator `/` is overloaded), `log`, `exp`, `tanh` which calls into the backend (cpu, gpu) code. Also special attention to `ewise_or_scalar` code. **weiz 2024-06-09 I should check what are the avaiable functions provided in `array_api` MUST**

so that I can peacefully know what functions that I can use to implement the `ops_mathematic.py`

This has been the focus of HW3. `weiz 2024-06-09` I should check what are the available functions provided in `array_api` MUST so that I can peacefully know what functions that I can use to implement the `ops_mathematic.py`

`python/needle/autograd.py` TODO: this file defines `Value`, `Tensor` (subclass of `Value`). The `cached_data` in `Value` is of `NDArray`. I.e., `NDArray` is the back-up storage of `Value`, which is the node in the AD graph. `TensorOp` is also defined here. This has been the focus of HW1 and in the end we wrote a simple MNIST 2 FC layer type of network that achieved 98% test accuracy.

`python/needle/ops/ops_mathematic.py` Most of initial hw4 should be done here, check any `array_api` inconsistency (now that we have CPU GPU `NDArray` backend instead of previously omnipotent numpy backend. This is the focus of HW1 and will be the focus of hw4 when implementing dilate/conv/flip for CNN.

`python/needle/nn` This is we make lots of abstract: `Parameter(Tensor)`. The focus of hw2, also achieves 98% test accuracy for mnist. This will also be the focus of hw4, when we implement CNN, RNN and LSTM.

`python/needle/data/` Focus of HW2 (when writing MNIST datasets abstraction) and of HW4 (when doing CIFAR10, Penn Treebank)

TODO: Draw a diagram that shows which homework covers which part of the code, or which part of the code is the focus of which homework.

7.3 `array_api` capabilities

NDArray functions : write a simple driver program for both cpu and gpu backends and list all the available functions. See https://github.com/weidotwisc/10714/blob/main/hw_code/hw4/print_nd_methods.py and https://github.com/weidotwisc/10714/blob/main/hw_code/hw4/NDArray_func_list_weiz.txt

`array_api`, `backend_ndarray`, `BackendDevice`, `NDArray` Figure 10 illustrates the `backend_ndarray` directory. Several things to note:

1. There are three backends – `cpu`, `cuda` and `numpy`. The first two are implemented in C++ and linked as a `python-c` module. The `numpy` backend is implemented in `ndarray_backend_numpy.py`, which should have deprecated the `backend_numpy.py`¹⁷.

¹⁷One can see https://github.com/weidotwisc/10714/blob/369f78949d41eba6b2d1dd22d3369f691beb4dd9/hw_code/hw4/python/needle/autograd.py#L3 and https://github.com/weidotwisc/10714/blob/369f78949d41eba6b2d1dd22d3369f691beb4dd9/hw_code/hw4/python/needle/autograd.py#L20 for comments

2. `ndarray.py` is the main class that does the heavy lifting: (1) `class BackendDevice` dispatches Array impl to different backends by using `mod` property setting trick (see Figure 10 red dashed line-arrow) (2) `class NDArray` implements numpy-like operations. Note dunder functions such as `__setitem__()` and `__getitem__()` are implemented here so that `NDArray` can be indexed using slicing and other pythonic ways. They are especially useful when implementing Ops such as `Stack` and `Split` (3) `ndarray.py` is imported as the `array_api` in `ops_mathematic.py` and `ops_logarithmic.py`, so there are some auxiliary functions (e.g., `swapaxes()` and `squeeze()`) implemented to make `array_api` compatible w/ `numpy`.
3. `ndarray_backend_numpy.py` implements several methods (e.g., `scalar_xxx` and `ewise_xxx`) to be compatible w/ `array_api`.

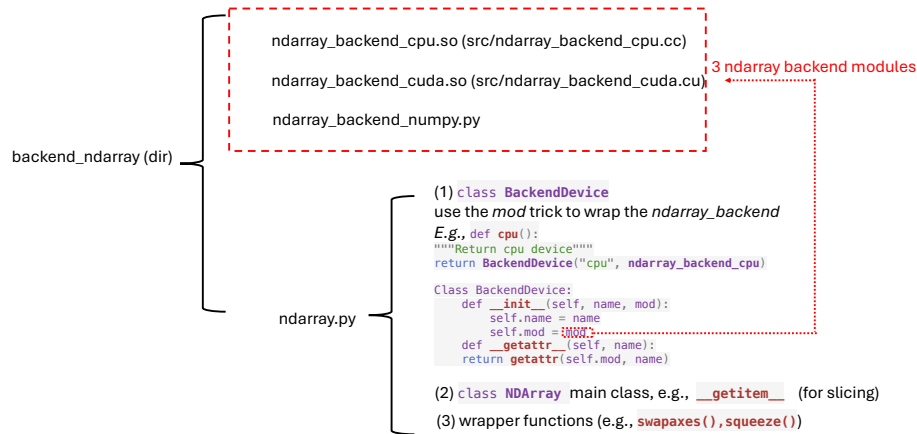


Figure 10: `ndarray_backend` directory structure

Finally, Table 8 compares the compatibility of `array_api` (imported as the `ndarray.py`) and `numpy`.

A Note on `NDArray` and `Tensor`

- `NDArray` (defined in `ndarray.py`) is what backs-up `Tensor` (defined in `autograd.py`).
- `Tensor` class is similar to PyTorch's `Tensor` class and also has `requires_grad` field.
- `NDArray` is used in `Op`'s `compute()` function, `Tensor` is used in `Op`'s `gradient()` function. In PyTorch, `fwd()` and `bwd()` functions both use `Tensor` class. See next paragraph for details.

ops_mathematic	api	numpy	NDArray(needle)
DivScalar	divide	✓	✗
	__truediv__ (aka /)	✓	✓
EwiseDiv	divide	✓	✗
	__truediv__ (aka /)	✓	✓
Negate	negative	✓	✗
	__neg__ (aka -)	✓	✓
Relu	maximum(0,a)	✓	✗
	maximum(a,0)	✓	✓
PowerScalar	power()	✓	✗
	__pow__ (aka **)	✓	✓
MatMul	matmul	✓	✗
	__matmul__ (aka @)	✓	✓
Transpose	swapaxes()	✓	✗->✓
	permute()	✗	✓
logsumexp	max()	✓	✗->✓
	squeeze()	✓	✗->✓
	ewise_or_scalar broadcast-able	✓	✗->✓

Table 8: **array_api** and **numpy** compatibility. Note **array_api** is imported from **ndarray.py**. The last two rows in this table use the auxiliary wrapper functions from **ndarray.py**, whereas the rest of rows use the **NDArray** class methods.

How NDArray and Tensor are connected ? In Listing 19, when a TensorOp is called (e.g., tanh) through a dunder `__call__`, it will call `make_from_op`, which calls into `realize_cached_data`, which will call into `op.compute()` method which will use the NDArray operations.

Listing 19: "Relationship between NDArray and Tensor in needle"

```
class TensorOp(Op):
    """Op class specialized to output tensors, will be alternate
    ↳ subclasses for other structures"""

    def __call__(self, *args):
        return Tensor.make_from_op(self, args)

class Tensor(Value):
    @staticmethod
    def make_from_op(op: Op, inputs: List["Value"]):
        tensor = Tensor.__new__(Tensor)
        tensor._init(op, inputs)
        if not LAZY_MODE:
            if not tensor.requires_grad:
                return tensor.detach()
            tensor.realize_cached_data()
        return tensor

class Value
    def realize_cached_data(self):
        """Run compute to realize the cached data"""
        # avoid recomputation
        if self.cached_data is not None:
            return self.cached_data
        # note: data implicitly calls realized cached data
        self.cached_data = self.op.compute(
            *[x.realize_cached_data() for x in self.inputs]
        )
        return self.cached_data
```

In a nutshell, `compute()` (or `forward()` function) will rely on NDArray operations, `gradient()` (or `backward()` function) will rely on Tensor operations. Unfortunately, these two sets of operations are not overlapped completely, thus some confusion could rise.

A Note on tensor axes Tensor axes are essentially big-endian, when the 0-th dimension is the most significant. See Section 6.3 red text of struggle of hand-written number and number line!

Numerical Checking gradients and Use PyTorch as an reference to check Especially when I debug something like Summation backward, it would be quite useful. See *backward_check()* paragraph in Section 7.6

Re-thinking Summation and Bcast (Possible) Two ways to fix it: (1) Summation sum over multiple axes, one at a time I have implemented this in `compute()` in `Summation` class in `ops_mathematic.py`. Note i have also made effort to make it work w/ `keepdims=True`, the reason that I had to call underlying `array_api.sum` with `keepdims=False` is purely for making tests happy as they expect `keepdims=False` shape (2) Make Bcast more like numpy bcast, in that it can broadcast can happen between two tensors but with different yet amenable shape, see <https://numpy.org/doc/stable/user/basics.broadcasting.html>. In essence, align the tensor of smaller rank¹⁸ with the tensor with larger rank by padding 1 in shape, so that two tensors have the same rank. And the each element of their shape is either same or one of them is 1. I have implemented in `broadcast_to()` in `ndarray.py` (see https://github.com/weidotwisc/10714/blob/3b15cf53088ed6c29eb148f56603ecc51d8eb9b7/hw_code/hw4/python/needle/backend_ndarray/ndarray.py#L314-#L325, which alone will pass the `summation_backward` test. *weiz 2024-06-20 Just realized that bcast should have never changed its own shape or strides, otherwise it will change its own shape, and might not be bcastable to another shape later. Think of (1,) first bcast to (1,1,1), but then cannot be bcast-ed to (1,1) later – although strictly speaking it can in theory, but the current implementation mandates that only lower rank tensor bcasts to higher rank tensor. In any case, it will be good to maintain the original tensor's shape and shouldn't change its shape (even though an isomorphic change*

I find `hw4's def reduce_view_out(self, axis, keepdims=False)` is a little different from `hw3's def reduce_view_out(self, axis)`. So the `reduce_sum` tests that have passed in `hw3` will fail in `hw4`, as in `hw4` the default behavior is not keeping the dimension 1 around. But hopefully `hw4` tests will not be affected.

Logsumexp and ewise ops w/ bcast In `Logsumexp`, we use `sum/`subtract of a tensor and some reduction (e.g. `Z - max_z`). Numpy can bcast `max_z` to the shape of `Z`. But `NDArray`, I now support this in https://github.com/weidotwisc/10714/blob/3b15cf53088ed6c29eb148f56603ecc51d8eb9b7/hw_code/hw4/python/needle/backend_ndarray/ndarray.py#L447-#L462, w/ only caveat that I would assume the `RHS`(e.g., `other` or `max_z`) is always broadcast to `LHS` (e.g., `self`), i.e., `RHS` is "smaller" than `LHS`.

7.4 How Value, Tensor, TensorOp, NDArray got connected

Some Python dunder methods :

¹⁸The length of the shape of a tensor is called its rank. The rank of a tensor refers to the number of dimensions or axes it has.

- `__new__`: allocate a piece of memory when a class's constructor is called.
- `__init__`: the defac to class constructor, called after `__new__` during construction.
- `__call__`: called on an object instance as if that instance is a function call. A typical use in Needle is a `TensorOp` object is called and then return a tensor as in Listing 20

Listing 20: How a `TensorOp` call becomes a `Tensor`, thru `TensorOp`'s dunder call `__call__`

```
class TensorOp(Op):
    """Op class specialized to output tensors, will be alternate
    ↳ subclasses for other structures"""

    def __call__(self, *args):
        return Tensor.make_from_op(self, args)

class Tensor(Value):
    @staticmethod
    def make_from_op(op: Op, inputs: List["Value"]):
        tensor = Tensor.__new__(Tensor)
        tensor._init(op, inputs)
        if not LAZY_MODE:
            if not tensor.requires_grad:
                return tensor.detach()
            tensor.realize_cached_data()
        return tensor

class Value:
    def _init(
        self,
        op: Optional[Op],
        inputs: List["Tensor"],
        *,
        num_outputs: int = 1,
        cached_data: List[object] = None,
        requires_grad: Optional[bool] = None
    ):
        global TENSOR_COUNTER
        TENSOR_COUNTER += 1
        if requires_grad is None:
            requires_grad = any(x.requires_grad for x in inputs)
        self.op = op
        self.inputs = inputs
```

```

self.num_outputs = num_outputs
self.cached_data = cached_data
self.requires_grad = requires_grad

```

A Running example Listing 21 illustrates a running example of how a computation graph in Needle is constructed. Note `v1`, `v2`, `v3` and `v4` are of type `Tensor`, which is a subclass of `Value`. `Value` is the node in a computational graph, it has the following fields:

- `op`: `Optional[Op]`
- `inputs`: `List["Value"]`
- `cached_data`: `NDArray`
- `requires_grad`: `bool`

The `inputs` are the edges that connect nodes (aka `Value`), these nodes are created via the `_init()` call inside class `Value`, see the last function in Listing 20 (note `Tensor` is a subclass of `Value`, thus it inherits that `_init()` call. This `_init()` call prepares all the fields of `Value` (aka computational graph Node)

Also note that in `v2= ndl.exp(v1)`, `exp()` is from `ops_mathematic.py`¹⁹, and it returns `{Exp() (a)}`, which calls into the `__call__()` of `Exp` class (which is a subclass of `TensorOp` and will call into `Tensor.make_from_op()`, as shown in the 1st method in Listing 20 to return a `v2` as a `Tensor`. In the similar fashion, `v3` and `v4` are created. And their edges are connected via the `Value._init()` call as the last method demonstrated in Listing 20.

Listing 21: "Running example of Needle tensor buildup and computation graph build up"

```

def ndl_test():
    v1 = ndl.Tensor([0], dtype="float32")
    v2 = ndl.exp(v1) # exp() is from ops_mathematics.py
    v3 = v2 + 1
    v4 = v2 * v3
    v4.backward()

```

Connection between Tensor and NDArray `Tensor` and `NDArray` are connected via `value.realize_cached_data()` call (see Listing 22) and materialize all the data storage (i.e., `NDArray`). As we can see `compute()` call in `Op` takes `NDArray`

¹⁹When `needle` is imported via `__init__.py`, it imports `ops`, and doesn't import `backend_ndarray`. Thus `exp()` is from `ops_mathematic.py`, not from `ndarray.py`

Listing 22: How a tensor materializes data (aka NDArray)

```
class Value:
    def realize_cached_data(self):
        """Run compute to realize the cached data"""
        # avoid recomputation
        if self.cached_data is not None:
            return self.cached_data
        # note: data implicitly calls realized cached data
        self.cached_data = self.op.compute(
            *[x.realize_cached_data() for x in self.inputs]
        )
        return self.cached_data
```

In Listing 23, we demonstrate how `Tensor` is used in the `gradient()` call. Note `out_grad` is the adjoint w.r.t this node (aka the gradient of loss w.r.t this node), the results of `gradient()` call is the partial adjoint of this node's inputs w.r.t this node, **not** the gradient of loss w.r.t this node. The adjoint of this node is passed in `out_grad`, which was populated via the call in `compute_gradient_of_variables()` from Listing 24. **What I feel the most important missing part when I first learnt this course is that: `gradient()` does not calculate the node's adjoint, which is the higher level `compute_gradient_of_variables()`'s job. `gradient()` actually only calculates this node's input nodes' partial adjoints. These partial adjoints will be used in `compute_gradient_of_variables()` to sum up as those input nodes' adjoints. 重要的事情说三遍!!!**

Listing 23: A example of **Op** calculate gradients

```
class EwiseMul(TensorOp):
    def compute(self, a: NDArray, b: NDArray):
        return a * b

    def gradient(self, out_grad: Tensor, node: Tensor):
        lhs, rhs = node.inputs
        return out_grad * rhs, out_grad * lhs
```


Listing 24: Automatic Differentiation core function. It traverses the gradient calculation graph (aka the transpose graph of computation graph) in the topological order, calculates each node's adjoint using the summation equation in Equation (300). Along the way, the algorithm populates the partial adjoints of each (input) node by calling the `gradient()` call of each (output) node.

```
def compute_gradient_of_variables(output_tensor, out_grad):
    """Take gradient of output node with respect to each node in
    ↪ node_list.

    Store the computed result in the grad field of each Variable.
    """
    # a map from node to a list of gradient contributions from
    ↪ each output node
    node_to_output_grads_list: Dict[Tensor, List[Tensor]] = {}
    # Special note on initializing gradient of
    # We are really taking a derivative of the scalar reduce_sum(
    ↪ output_node)
    # instead of the vector output_node. But this is the common
    ↪ case for loss function.
    node_to_output_grads_list[output_tensor] = [out_grad]

    # Traverse graph in reverse topological order given the
    ↪ output_node that we are taking gradient wrt.
    reverse_topo_order = list(reversed(find_topo_sort([
    ↪ output_tensor])))
    for node in reverse_topo_order:
        if node.requires_grad:
            node.grad = sum_node_list(node_to_output_grads_list[
            ↪ node])
            if node.op is None: # if true inputs node, no ops are
            ↪ defined
                continue
            partial_adjoints = node.op.gradient_as_tuple(node.grad,
            ↪ node) # weiz 2023-12-30, note, we need to use
            ↪ node.op not input_node.op, this is most
            ↪ IMPORTANT!!!
            for input_node, input_node_partial_adjoint in zip(node.
            ↪ inputs, partial_adjoints):
                if (input_node not in node_to_output_grads_list):
                    node_to_output_grads_list[input_node] = [
                    ↪ input_node_partial_adjoint]
                else:
                    node_to_output_grads_list[input_node].append(
                    ↪ input_node_partial_adjoint)
```

One thing to note is that in Listing 24, because it traverses in the gradient calculation graph's topological order, so when the node is traversed, it is guaranteed that its dependencies have all been traversed thus all its partial adjoints have been calculated, and we can sum all its partial adjoints to get its own adjoint by using Equation (300)

Finally, the gradient calculation is kick-started by calling the loss node (aka the top node in the gradient calculation graph topological order)'s `backward()` function. Note the `out_grad` for this node, obviously not calculated as all the other nodes, is typically initialized as a `one` tensor.

Listing 25: Connect everything together – when a **Tensor's** `backward()` is called and now the gradient calculation begins! Note typically a `one` tensor is initiated as the `out_grad` for this very top node in the topological order.

```
class Tensor(Value):
    def backward(self, out_grad=None):
        out_grad = (
            out_grad
            if out_grad
            else init.ones(*self.shape, dtype=self.dtype, device=
                ↪ self.device)
        )
        compute_gradient_of_variables(self, out_grad)
```

A misc note of the `detach()` in `Tensor()`. It simply wraps the `realize_cached_data()` `NDArray`, create a new `Tensor` and set this tensor's `inputs` as `None`, so that it won't be attached to any computation graph. See Listing 26.

Listing 26: `detach()` method in `Tensor`. Note that in `_init()` call, this returned tensor's inputs as `None`, so that it won't be attached to any computation graph.

```
class Tensor(Value):
    def detach(self):
        """Create a new tensor that shares the data but detaches
        ↪ from the graph."""
        return Tensor.make_const(self.realize_cached_data())

    @staticmethod
    def make_const(data, requires_grad=False):
        tensor = Tensor.__new__(Tensor)
        tensor._init(
            None,
            [], # weiz note this are the inputs, which are None!
            cached_data=data
        )
        if not isinstance(data, Tensor):
            data.realize_cached_data()
        tensor.requires_grad = requires_grad
        return tensor
```

7.5 Numerical Gradient Checking

Definition of partial derivative and gradients in the limit notion

$$\frac{\partial f(\theta)}{\partial \theta_i} = \lim_{\epsilon \rightarrow 0} \frac{f(\theta + \epsilon e_i) - f(\theta)}{\epsilon} \quad (365)$$

$$= \lim_{\epsilon \rightarrow 0} \frac{f(\theta + \epsilon e_i) - f(\theta - \epsilon e_i)}{2\epsilon} + o(\epsilon^2) \quad (366)$$

$$\nabla_{\theta} f(\theta) = \left(\dots, \frac{\partial f(\theta)}{\partial \theta_i}, \dots \right)^T \quad (367)$$

$$\theta \in R^n, f : R^n \mapsto R, \epsilon \in R, e_i = (0, \dots, \underbrace{1}_{i\text{-th}}, \dots, 0)^T \quad (368)$$

Note e_i is the standard basis at i -th dimension (i.e., $(\dots, 1, \dots)^T$ w/ i -th element being 1 and all the rest is 0). So the partial derivative limit notion requires all the other dimensions of parameter θ is fixed, while change is only allowed at the i -th dimension. Equation (366) is the numerical stable version of Equation (365) and serves as the backbone of all the numerical gradients checking. Stefan used Listing 27 in his class so does Chen in his HW4. Note each iteration will perturb only one dimension and then will reset that dimension afterwards. So there are $2n$ fwd calculation to examine one gradients – very expensive.

Listing 27: "Use partial derivative limit definition for numerical gradient checking"

```

def finite_diff(f, x, delta):
    """estimate the gradient by finite-differencing method"""
    grad_f, dx = np.zeros_like(x), np.zeros_like(x)
    for i in range(x.size):
        dx.flat[i] = delta
        grad_f.flat[i] = f(x+dx) - f(x-dx)
        dx.flat[i] = 0.0
    return grad_f / (2*delta)

```

Alternative (invariant checking) As we can see it is very expensive to calculate gradients numerically by using partial derivative definition Equation (366) as it involves $2n$ forward calls to check one gradient. The question is once we have $\nabla|_{\theta}(\theta)$ calculated via Automatic-Differentiation, is there a cheaper way to check it ? It is based on the following Equation (369)

$$\delta^T \nabla_{\theta} f(\theta) = \frac{f(\theta + \epsilon \delta) - f(\theta - \epsilon \delta)}{2\epsilon} + o(\epsilon^2) \quad (369)$$

$$f : R^n \mapsto R, \theta \in R^n, \delta \in R^{n^{20}}, \epsilon \in R$$

Proof

$$f(x) \stackrel{\text{Taylor Expansion}}{=} f(x_0) + (x - x_0)^T \nabla f(x_0) + \frac{1}{2}(x - x_0)^T H(x - x_0) + \dots \quad (370)$$

As a result,

$$f(\theta + \epsilon \delta) = f(\theta) + \epsilon \delta^T \nabla_{\theta} f(\theta) + \frac{1}{2} \delta^T H(\theta) \delta + o(\epsilon^2) \quad (371)$$

$$\begin{aligned} f(\theta - \epsilon \delta) &= f(\theta) - \epsilon \delta^T \nabla_{\theta} f(\theta) + \frac{1}{2} (-\delta^T) H(\theta) (-\delta) + o(\epsilon^2) \\ &= f(\theta) - \epsilon \delta^T \nabla_{\theta} f(\theta) + \frac{1}{2} \delta^T H(\theta) \delta + o(\epsilon^2) \end{aligned} \quad (372)$$

$$f(\theta + \epsilon \delta) - f(\theta - \epsilon \delta) = 2\epsilon \delta^T \nabla_{\theta} f(\theta) + o(\epsilon^2)$$

$\implies$

$$\delta^T \nabla_{\theta} f(\theta) = \frac{f(\theta + \epsilon \delta) - f(\theta - \epsilon \delta)}{2\epsilon} + o(\epsilon^2) \quad \square$$

As we can see, Equation (369) requires 2 fwd calls to check one gradient. Further note that cannot construct a gradients like in Equation (366), but given a AD calculated gradients, it can tell if that gradients is correct, thus it is an invariant checking.²¹

²⁰ δ is a unit random vector

²¹Note that even Chen talked about Equation (369), but in the real implementation, it basically uses the Equation (366) for each element of the θ vector, precisely like what Stefan does.

But a key question is given a gradient, how many Equation (369) do we need to do to have a good confidence that that gradient is correctly calculated (note if we use Equation (366) we only need to check once, as it will construct the full gradient). Yingdon Lu modeled this question as an "equivalent" question: given a n dimensional vector v supposed to contain all 0s, if we pick one element is zero and how confident we are that v is all 0s, furthermore if we pick k elements and they happen to be all 0s, how confident we are that all elements in v are zeros.²² We model each invariant checking is an independent event w/ probably 0.5 to fail or succeed (we don't have much better modeling other than this).

Then if we use A to denote the event that all elements are 0s and B_k to denote the event that we have chosen k elements and they are all 0s event, we would have the following probability in Equation (374), the trick is that to have the k elements that we have chosen is a $\frac{1}{\binom{n}{k}}$ probability event, which is a pretty rare event. As we can see from Equation (374), even when k is small, $\binom{n}{k}$ can be quite large ($\sim O(n^k)$), thus bumping up the probability/confidence quite significantly. In another word, we can choose rather small k , if the invariant checking passed all of these k tests, we can be rather confident that the gradient is correct (**TODO: maybe I should draw a plot of this probability function and see how probability varies given different k under the same n**). Overall there is no free lunch, but Equation (369) seems quite a good compromise of computation and confidence of correct gradient implementation.

$$P(A|B_k) = \frac{(\frac{1}{2})^n}{\frac{1}{\binom{n}{k}}(\frac{1}{2})^k} \quad (373)$$

$$= \binom{n}{k} (\frac{1}{2})^{n-k} \quad (374)$$

How does Chen's backward_check() work During the process of debugging, I have also learnt `backward_check()` function in detail. In addition to using an algorithm similar to Listing 27, it uses a random vector c and the inner product of c and the output vector of $f()$ as the loss function, assumably to achieve better numerical stability. But do NOT confuse this with the last paragraph – Chen didn't use the invariant checking stated in the last

$$x \in R^n, y \in R^m, c \in R^m$$

$$y = f(x) \quad (375)$$

$$L = c^T y \quad (376)$$

$$\frac{\partial L}{\partial y} = c^T \quad (377)$$

$$\implies \nabla L|_y = c \quad (378)$$

²²This is as if saying if we conduct invariant checking once we get it right, how confident we are about this gradient; further how many times of checks should we do so that we reach a confident threshold ?

In `backward_check()` ²³,
`backward_grad = out.op.gradient_as_tuple(ndl.Tensor(c, device=args[0].device), out)`
sends the `c` as the `out_grad`, and `out` as the `node` into
`def gradient_as_tuple(self, out_grad: "Value", node: "Value")`. Seen
in Equation (378), the $\nabla L|_y$ is precisely `c`, and that is why `c` is the `out_grad` in
the call to `gradient_as_tuple()`.

TODO: (1) Maybe I should use Equation (369) to write my own checking
code (2) Use `pytorch` as a ground truth.

7.6 Part1: ND Backend

Tanh(hyperbolic tangent) 双曲正切

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (379)$$

$$\begin{aligned} &= \frac{e^{2x} - 1}{e^{2x} + 1} \\ &= \frac{e^{2x} + 1 - 2}{e^{2x} + 1} \\ &= 1 - 2 \cdot \frac{1}{e^{2x} + 1} \end{aligned} \quad (380)$$

Note,

$$\begin{aligned} \tanh^2(x) &= \frac{e^{2x} + e^{-2x} - 2}{e^{2x} + e^{-2x} + 2} \\ &= 1 - \frac{4}{e^{2x} + e^{-2x} + 2} \\ &= 1 - \frac{4e^{2x}}{e^{4x} + 2e^{2x} + 1} \\ &= 1 - \frac{4e^{2x}}{(e^{2x} + 1)^2} \end{aligned} \quad (381)$$

Taking derivative in Equation (380), we get

$$\begin{aligned} \tanh'(x) &= 2 \cdot (e^{2x} + 1)^{-2} \cdot e^{2x} \cdot 2 \\ &= \frac{4e^{2x}}{(e^{2x} + 1)^2} \end{aligned} \quad (382)$$

$$\stackrel{\text{Equation (381)}}{=} 1 - \tanh^2(x) \quad (383)$$

`__rsub__` properly implement `__rsub__`: `self.__rsub__(other) = other - self`²⁴,
so that `1 - tahn(a)**2` makes sense, as 1 is a scalar, and it doesn't have a

²³https://github.com/weidotwisc/10714/blob/df805bcb4b9ccfa2956a859d5ad850fa2b120536/hw_code/hw4/apps/weiz_test.py#L14

²⁴in https://github.com/weidotwisc/10714/blob/389227257a2bfe08ed4b675f1db4fabd29b4b61f/hw_code/hw4/python/needle/autograd.py#L341-#L345

`__sub__` impl; but `tanh(a)**2` is a tensor, as long as it has a proper `__rsub__` impl, it is okay. Previously, the professors have a messed-up impl that `__rsub__ = __sub__`, which was so wrong.

Finally, the `forward()` and `backward()` for Tanh are pretty simple: (1) for `fwd()`, we just need to call `array_api`'s `tanh()` impl. (2) for `bwd()`, we just need to apply element-wise Equation (383) to `node`'s inputs and then element-wise multiply that result with $\frac{\partial L}{\partial out}$

Listing 28: "fwd() and bwd() for Tanh"

```
class Tanh(TensorOp):
    def compute(self, a):
        ### BEGIN YOUR SOLUTION
        return array_api.tanh(a)
        ### END YOUR SOLUTION

    def gradient(self, out_grad, node):
        ### BEGIN YOUR SOLUTION
        return out_grad * (1 - tanh(node.inputs[0])**2)
        ### END YOUR SOLUTION
```

Stack Stack tensors **of the same size** along dimension **axis** to construct to a bigger stack. Several things stand out:

1. To allocate a new `NDArray` with `shape`, one should use `empty(shape, device)` method in `array_api`, don't forget the `device` argument.
2. the KEY thing is given an index of one axis, and retrieve all the elements of that tensor. The trick, taught by chatgpt, is to use `slice(None)` for all the other dimensions. See Listing 29

The `fwd()` in `Stack` is to first allocate a new tensor that is of the shape of stacked tensor. Then along the specified axis, assign each slot on that axis using the trick in Listing 29.

The `bwd()` in `Stack` is simply `split` (see next paragraph) the $\frac{\partial L}{\partial out}$ along the specified `self.axis`. This is a typical operation in the category of 摆摆好. A special trick we used is `slice(None)`, which means takes everything along this dimension. `needle` faithfully implemented indexing via `slice` (including `slice(None)`, which takes the whole range of that dimension). See inside `__getitem__()`²⁵; specially `process_slice()`²⁶; and `__setitem__()`²⁷.

²⁵https://github.com/weidotwisc/10714/blob/2e2d5ffe7778c0317cff1524b73e87ae7f984fe0/hw_code/hw4/python/needle/backend_ndarray/ndarray.py#L397

²⁶https://github.com/weidotwisc/10714/blob/2e2d5ffe7778c0317cff1524b73e87ae7f984fe0/hw_code/hw4/python/needle/backend_ndarray/ndarray.py#L342

²⁷https://github.com/weidotwisc/10714/blob/2e2d5ffe7778c0317cff1524b73e87ae7f984fe0/hw_code/hw4/python/needle/backend_ndarray/ndarray.py#L421

Listing 29: How to index along an axis while retrieving all the other dimensions' elements. `self.axis` is along which axis to stack (or split), `i_th_tensor` is the `i`-th index along `self.axis`. `slice(None)` returns the entire range of one axis/dimension.

```
indexing_tuple = tuple(slice(None) if i != self.axis else
    ↪ i_th_tensor for i in range(stacked_tensor.ndim)) # slice(
    ↪ None) means includes all the elements in that dimension
stacked_tensor[indexing_tuple] = args[i_th_tensor]
```

Split Split a tensor along `self.axis` to n tensors, where n is the length of the original's tensor's `self.axis`. We only need to support splitting evenly to n -pieces (i.e., we don't need to support splitting by size of chunks). Two things stand out:

1. We also need to use similar trick in Listing 29 to index into the original tensor to get each piece.
2. Once we get each piece, we need to `squeeze()` the `self.axis` as its current dimension is 1 and the test cases are looking for annihilating that dimension. I have implemented `sequeeze()` in `ndarray.py` for this purpose. Also I called `compact()` before squeezing just to make sure the split tensor memory is contiguous.

The `fwd()` method is simply to use Listing 29 to slice the original tensor to get each piece.

The `bwd()` method is simply **stack** the $\frac{\partial L}{\partial out}$. Again, this is a typical operation in the category of 摆摆好.

weiz 2024-07-06 Note it takes me about 1.5 months, after 2024-05-18 finishing hw3, to just finish the first 10 points of this hw4. Because I also solidify my knowledge of the following:

1. [8] treatment of linear layer.
2. `array_api` consistencies w/ `numpy`, via introduce auxiliary functions in `NDArray`
3. How `NDArray` and `Tensor` are playing together²⁸ via detailed re-study of automatic differentiation engine lecture 4 and 5.
4. Numerical gradient checking
5. I have set up the development environment of `VSCode` on `dyce` (both `C/C++` and `python`, and debugging mode).

I feel the time well spent!

²⁸Specially, I get to know `gradient()` calculates input nodes' partial adjoint, not current node's adjoint

7.7 Part 2: CIFAR-10 dataset

- cifar10 dataset is obtained from <http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
- This dataset has 6 parts of training data (`data_batch_1` – `data_batch_6`) and 1 testing data (`test_batch`)
- Each row of data is of (3072) and need to reshape to (3,32,32)
- I used `np.concatenate(data_list, axis=0)` to concatenate those data files w/o creating a new axis. If I want to create a new axis, I can use `np.stack()`
- Each data item is of range [0,255], need to divide by 255 to make it a float between [0,1]
- Copy `python/needle/data/data_transforms.py` and `python/needle/data/data_basic.py` from hw2
- I haven't applied `transform()` for each data item, I might need to do it later when testing convergence.

7.8 Convolutional neural network

7.8.1 flip()

weiz 2024-09-10 `flip()` is used to flip the kernel during backprop ? Per <https://numpy.org/doc/stable/reference/generated/numpy.flip.html>,

1. **Reverse** the order of elements in an array **along the given axis**.
2. The shape of the array is preserved, but the elements are reordered.

Alternative 1 Using `slice(None)` trick (my method): A example of `np.flip()` is given in Listing 30, I was a little confused by `np.flip(A, 2)` at the beginning, then I realized that you can use the same slicing technique we used in `Split()` and `Stack()` in Section 7.6. Intuitively, you can either (1) think of this tensor `A` as in xyz 3-D space, and only flip elements along the z axis, or (2) fix all the other dimensions, and do assign each element i (which is a $n - 1$ rank tensor, n is the dimension of the original tensor) along the flipping axis $axis$ by using Equation (384)

$$B[..., i, ...] = A[..., dim_{axis} - 1 - i, ...] \quad (384)$$

Listing 30: A running example of np.flip()

```
>>> A = np.arange(8).reshape((2,2,2))
>>> A
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
# flip the first dimension
>>> np.flip(A,0)
array([[[4, 5],
        [6, 7]],

       [[0, 1],
        [2, 3]]])
# flip the 2nd dimension
>>> np.flip(A,1)
array([[[2, 3],
        [0, 1]],

       [[6, 7],
        [4, 5]]])
# flip the last dimension
>>> np.flip(A,2)
array([[[1, 0],
        [3, 2]],

       [[5, 4],
        [7, 6]]])
# One easy implementation of flipping, notice that I used the
# slicing technique used in Split() and Stack()
>>> B=np.zeros(A.shape)
>>> B[:, :, 0]=A[:, :, 1]
>>> B[:, :, 1]=A[:, :, 0]
>>> B==np.flip(A,-1)
array([[[ True,  True],
        [ True,  True]],

       [[ True,  True],
        [ True,  True]])
```

So we only need to create an `NDArray` that have the same shape of original tensor and then apply Equation (384) as done in Listing 31. Note that when calling `NDArray.make()`, make sure to pass in the `device` parameter, otherwise

the flipped tensor can reside on default device and test cases are not going to be happy.

Listing 31: `flip()` implementation by assigning each element (a n-1 rank tensor) along the flipped axis

```
result = NDArray.make(self.shape, device=self.device) # weiz
    ↪ 2024-07-12 important: put in device, otherwise it routes
    ↪ to the default device which could be different
for axis in axes:
    dim_along_axis = self.shape[axis]
    for i in range(dim_along_axis):
        dst_indexing_tuple = tuple(slice(None) if idx_axis !=axis
    ↪ else (i) for idx_axis in range(self.ndim) )
        src_indexing_tuple = tuple(slice(None) if idx_axis !=axis
    ↪ else (dim_along_axis-1-i) for idx_axis in range(
    ↪ self.ndim))
        result[dst_indexing_tuple] = self[src_indexing_tuple]
```

Alternative 2 Using Zico's method (negative stride trick). Zico provided some code to inspect flipped numpy tensor at https://github.com/weidotwisc/10714/blob/main/hw_code/hw4/apps/np_flip_inspection.py. And we can inspect the flipped tensor at Listing 32

From there, we can see two things:

1. We simply need to negate **stride**
2. We move the **offset** to proper location – the last element of each flipped axis. Details are seen in Listing 33

Listing 33: `flip()` implementation by negating stride

```
new_offset = 0
new_strides_list=list(self.strides)

for axis in axes:
    new_offset += (self.shape[axis]-1)*self.strides[axis]
    new_strides_list[axis] = self.strides[axis]*(-1)

result = NDArray.make(shape=self.shape, strides=tuple(
    ↪ new_strides_list), offset=new_offset, handle=self._handle,
    ↪ device=self.device).compact()
```

Note we would need to call `.compact()` in the end because **needle** doesn't support negative strides, but calling `.compact()` we go into the our own C implementation, which supports negative stride properly, and return a python **needle** NDArray that has positive **strides** which is equivalent to the negative one.

Listing 32: Flipped tensor inspection from Zico

```

-----
Inspection 1: A: A = np.arange(1, 25).reshape(3, 2, 4)
Data: |( 1 2 3 4) ( 5 6 7 8)| |( 9 10 11 12) (13 14 15 16)| |(17
    ↪ 18 19 20) (21 22 23 24)|
Offset: 0
Strides: 8, 4, 1
-----
Inspection 2: np.flip(A, (2,))
Data: |( 4 3 2 1) ( 8 7 6 5)| |(12 11 10 9) (16 15 14 13)| |(20
    ↪ 19 18 17) (24 23 22 21)|
Offset: 3
Strides: 8, 4, -1
-----
Inspection 3: np.flip(A, (1,))
Data: |( 5 6 7 8) ( 1 2 3 4)| |(13 14 15 16) ( 9 10 11 12)| |(21
    ↪ 22 23 24) (17 18 19 20)|
Offset: 4
Strides: 8, -4, 1
-----
Inspection 4: np.flip(A, (0,))
Data: |(17 18 19 20) (21 22 23 24)| |( 9 10 11 12) (13 14 15 16)|
    ↪ |( 1 2 3 4) ( 5 6 7 8)|
Offset: 16
Strides: -8, 4, 1
-----
Inspection 5: np.flip(A, (0,1,2))
Data: |(24 23 22 21) (20 19 18 17)| |(16 15 14 13) (12 11 10 9)|
    ↪ |( 8 7 6 5) ( 4 3 2 1)|
Offset: 23
Strides: -8, -4, -1
-----

```

Flip(TensorOp) In its `compute()` method, we simply need to call the `flip()` in `NDArray`. In its `gradient()` method, we simply need to flip the `out_grad` along the flipped axes.

7.8.2 `pad()` in `NDArray`

weiz 2024-09-10, `pad()` is used to pad the input image! Given a **tuple** of 2-tuple, which marks the left padding size and right padding size long the padded axis, of the **same length** of the tensor dimension; pad the tensor always with 0s. Again we use the `slice(None)` trick. We just need to do two things:

1. Create the result tensor with the proper shape, accounting for the paddings. Fill this tensor w/ 0s, as we only pad with 0s.
2. For each dimension of the padded tensor, we just assign the original tensor's corresponding dimension to the proper locations of this padded tensor. See Listing 34

Listing 34: `pad()` in `NDArray`

```
def pad(self, axes):
    """
    Pad this ndarray by zeros by the specified amount in '
    ↪ axes',
    which lists for all axes the left and right padding
    ↪ amount, e.g.,
    axes = ((0, 0), (1, 1), (0, 0)) pads the middle axis
    ↪ with a 0 on the left and right side.
    """
    ### BEGIN YOUR SOLUTION
    assert(len(axes) == self.ndim)
    new_shape_list = []
    for pad_axis, dim_axis in zip(axes, self.shape):
        assert(len(pad_axis) == 2)
        new_shape_list.append(dim_axis+pad_axis[0]+pad_axis[1])
    ↪
    result = NDArray.make(tuple(new_shape_list), device=self.
    ↪ device)
    result.fill(0)
    dst_index_slice_list = []
    for idx, pad_axis in enumerate(axes):
        dst_index_slice_list.append(slice(pad_axis[0], self.
        ↪ shape[idx]+pad_axis[1], 1))
    result[tuple(dst_index_slice_list)] = self
    #result[tuple(dst_index_slice_list)] = self[:] # weiz
    ↪ 2024-07-13, I could have just use result[tuple(
    ↪ dst_index_slice_list)] = self, but i just want to
```

```

    ↪ show that because I have added some code in
    ↪ __getitem__() to make things like [:], [1] work
    ↪ with variable dimensions, this work too
    return result
    ### END YOUR SOLUTION

```

Notice that `pad()` is only used in `NDArray`, `ops` didn't require it.

When I was implementing `pad()`, I find `NDArray`'s support of indexing like `[:]`²⁹, `[1]`³⁰ (where just few dimension indices are specified) was not done. So I went ahead and fixed it by introducing some code listed in Listing 35 at the beginning of `__getitem__()` in `NDArray`. Also notice when only one scalar value is sliced out via `__getitem__()` call in `NDArray`, we still get `Tensor` (of 1 element) back; in contrast, `numpy` would just convert it to a scalar value.

Listing 35: Allowing `__getitem__()` using slice tuple that is smaller than tensor rank.

```

# weiz 2024-07-13 add treatment to make idxs the same length as
    ↪ ndim, fill in slice(None) when dimensions are smaller
assert(len(idxs) <= self.ndim)
if(len(idxs) < self.ndim):
    filled_slice_none_tuple = (slice(None),)* (self.ndim - len(
        ↪ idxs))
    idxs = idxs + filled_slice_none_tuple
# end of weiz 2024-07-13 add treatment to make idxs the same
    ↪ length as ndim, fill in slice(None) when dimensions are
    ↪ smaller
    # now stuff like A[:,], A[1] would work :)

```

7.8.3 Dilation

weiz 2024-09-10, `dilate()` is used to dilate the filter!

`dilate()` I chose to implement it in `NDArray.py` so that `Dilate(TensorOp)` can directly call into it, instead of directly in `Dilate(TensorOp)` as asked from the homework. Two things need be done:

1. Create a `NDArray` **result** of the proper shape. Each dimension of the dilated-axis is increased by `1+dilation` times. **result** should be filled w/ 0s.
2. For each dilated axis, **result**`[:,:(1+dilation)]`=**source**`[:,:]`

`dilate()` implementation is at Listing 36

²⁹`[:]` is translated as `slice(None)` inside Python

³⁰`[1]` is translated as `[1,:]`

Listing 36: dilate() implementation

```
def dilate(self, axes:tuple, dilation:int):
    # step 1 create shape w/ proper shape, each dilated
    ↪ dimension is increased by (1+dilation) times
    new_shape_list=list(self.shape)
    for axis in axes:
        new_shape_list[axis]*=(1+dilation)
    result = NDArray.make(shape=tuple(new_shape_list), device=
        ↪ self.device)
    result.fill(0)

    # step 2 assign elements
    dest_indexing_slices=[]
    for axis in range(self.ndim):
        if (axis in axes):
            s1 = slice(None, None, 1+dilation)
            dest_indexing_slices.append(s1)
        else:
            s1 = slice(None)
            dest_indexing_slices.append(s1)
    result[tuple(dest_indexing_slices)] = self # always use
    ↪ tuple to contain slicing for __getitem__
    return result
```

undilate() `undilate()` is just the inverse operation of `dilate()`. Two things are:

1. Prepare the `NDArray` of proper size (i.e., corresponding dimension need to decreased by `1+dilation`)
2. Assign each dimension of each undilated axis by `dst[:] = src[:,1+dilation]`

`undilate()` implementation is at Listing 37

Listing 37: `undilate()` implementation

```
def undilate(self, axes:tuple, dilation:int):
    # step 1 create shape w/ proper shape, each undilated
    # dimension is decreased by (1+dilation) times
    new_shape_list = list(self.shape)
    for axis in axes:
        new_shape_list[axis] //= (1+dilation) # weiz
        # 2024-07-14 use inter division to make sure each
        # dimension is integer (e.g., 2) not a float (e.g
        # ., 2.0)
        # the intricacy is that later when we call
        # NDArray.make it will call into C code, it
        # has strong requirement of type,
        # elementsize 4.0 is not acceptable
    result = NDArray.make(shape=tuple(new_shape_list), device=
        self.device)

    # step 2 assign elements
    src_indexing_slices=[]
    for axis in range(self.ndim):
        if (axis in axes):
            sl = slice(None, None, 1+dilation)
            src_indexing_slices.append(sl)
        else:
            sl = slice(None)
            src_indexing_slices.append(sl)
    result = self[tuple(src_indexing_slices)]
    return result
```

Three things worth noticing:

1. `__getitem__()` returns a view of the underlying `NDArray` (aka they share the same **handle** or the pointer to the memory region). As a result, if you want to get a copy of the tensor, don't use slicing, but use constructor `__init__()`

2. Always use a tuple of slices to do the slicing of `NDArray` (because of the code says so :) ,not other sequences.
3. When calculating `shape`, if there is a chance to get a float number (e.g., through `undilate()` shape division), make sure to make it an integer (e.g., by integer division `//` instead of a general division `/`). Otherwise, when calculating `size`, when we might get a float and the underlying C code doesn't like to take a float when it should have been int. Even C has a stronger type check than Python ;)

`compute()` and `gradient()` methods in `Dilate(TensorOp)` and `UnDilate(TensorOp)`: for `compute()` we simply call `NDArray`'s `dilate()/undilate()` methods; for `gradient()` we simply call its inverse operation (e.g. `undilate()` for `Dilate(TensorOp)` and `dilate()` for `UnDilate(TensorOp)`) on tensor `out_grad` to get that node's input's adjoint.

In summary, `split()`, `stack()`, `pad()`, `flip()`, `dilate()`, `undilate()` are all 摆摆好 methods.

TODO: understand how `dilate()` and `undilate()` are used in Conv

7.8.4 NHWC vs NCHW

Most of this section is taken from ChatGPT

NHWC vs NCHW NHWC is Zico's format (Tensorflow) and NCHW is PyTorch's format (also Nvidia's preference). **PyTorch: NCHW (input) + OIKK (kernel), Tensorflow: NHWC (input) + KKIO (kernel).** weiz:2024-09-01

Per ChatGPT, In deep learning, especially when working with Convolutional Neural Networks (CNNs), the data format (i.e., how the image or feature map data is stored in memory) can significantly impact the efficiency and performance of training and inference. The two most common data formats are NHWC (batch size, height, width, channels) and NCHW (batch size, channels, height, width). Here are the pros and cons of each: NHWC (Batch Size, Height, Width, Channels)

Pros:

- Compatibility with TensorFlow: NHWC is the default format for TensorFlow, which means using this format can lead to better performance optimization on this framework.
- Intuitive Format: The NHWC format is more intuitive for humans because it follows the natural order of how images are typically represented (height x width x channels).
- Memory Access Patterns: For certain types of hardware, NHWC can be more efficient due to better memory access patterns when accessing pixel data.

Cons:

- Performance on GPUs: Some GPUs are optimized for NCHW format, so using NHWC can lead to suboptimal performance. This is particularly true for NVIDIA GPUs which have optimized kernels for NCHW.
- Conversion Overhead: If your model or training framework expects NCHW, using NHWC may require data format conversions, adding overhead.

NCHW (Batch Size, Channels, Height, Width)

Pros:

- Performance on GPUs: NVIDIA's cuDNN library, which is highly optimized for convolution operations, performs better with NCHW format. This can lead to faster training and inference times.
- Framework Support: Frameworks like PyTorch use NCHW as their default format, which can lead to better out-of-the-box performance and optimization. Memory Layout: For convolution operations, the NCHW format can result in more efficient memory access patterns, leading to better performance on many hardware accelerators.

Cons:

- Less Intuitive: NCHW is less intuitive compared to NHWC since it places the channels dimension before height and width, which is not the natural representation of images.
- Compatibility Issues: Using NCHW in a framework or environment optimized for NHWC can lead to compatibility issues and may require format conversion.

Key Considerations:

- Framework Choice: Choose NHWC for TensorFlow and NCHW for PyTorch, given their respective optimizations.
- Hardware: Consider your hardware's optimization. NVIDIA GPUs often perform better with NCHW due to cuDNN optimizations.
- Memory Access Patterns: Different hardware has different memory access patterns, so testing both formats on your specific hardware might be necessary to determine the best performance.

In summary, the choice between NHWC and NCHW often depends on the framework and hardware you are using. NHWC is generally better for TensorFlow and certain types of hardware, while NCHW is more optimized for PyTorch and NVIDIA GPUs.

I honestly find NCHW more intuitive (weiz)

Weights (Kernel) Tensor Ordering for NHWC and NCHW Formats

The weights (kernel) tensor will be ordered differently depending on whether you are using NHWC or NCHW data formats. The correct arrangement of dimensions in the kernel tensor is essential for the convolution operation to be performed correctly.

NHWC (Batch Size, Height, Width, Channels) In the NHWC format, the kernel tensor is ordered as:

[Height, Width, Input Channels, Output Channels]

NCHW (Batch Size, Channels, Height, Width) In the NCHW format, the kernel tensor is ordered as:

[Output Channels, Input Channels, Height, Width]

Explanation

- **Height and Width:** These dimensions represent the spatial dimensions of the kernel.
- **Input Channels:** This dimension corresponds to the number of input channels in the feature maps (e.g., RGB channels for color images).
- **Output Channels:** This dimension corresponds to the number of filters or kernels being used, which determines the number of output channels after the convolution.

Example Assume you have a convolutional layer with the following characteristics:

- Kernel size: 3×3
- Input channels: 3 (e.g., an RGB image)
- Output channels: 64 (e.g., 64 filters applied to the input)

For NHWC The kernel tensor will be ordered as:

[3, 3, 3, 64]

For NCHW The kernel tensor will be ordered as:

[64, 3, 3, 3]

Impact on Convolution Operation The way kernels are applied in the convolution operation depends on the ordering:

- In the NHWC format, the convolution operation will traverse height and width dimensions first, while handling input and output channels accordingly.
- In the NCHW format, the convolution will handle the channels more effectively for GPU operations, making it optimized for certain hardware and libraries like cuDNN.

Conversions When converting between NHWC and NCHW formats, the kernel tensors must also be appropriately reordered. This reordering can be done programmatically within deep learning frameworks like TensorFlow and PyTorch, which provide utilities to handle such conversions.

Conclusion Choosing the correct kernel tensor ordering is crucial for ensuring that convolution operations are performed correctly and efficiently. Always ensure that the kernel tensor matches the data format used by the specific deep learning framework and hardware being utilized.

7.8.5 Convolution Forward

Convolution of a image of $H \times W$ with a kernel k by k

$$H' = H - k + 1 \quad (385)$$

$$W' = W - k + 1 \quad (386)$$

H' and W' are the output image's height and width, H and W are the input image's height and width. Apparently, the output image is shrinking by $\frac{k-1}{2}$ on each side: to make output image and input image the same side, we would need to pad input image's each 4 sides (top, bottom, left and right) by $\frac{k-1}{2}$. Equation (387) is the definition of how convolution is done (from the perspective of each output element).

Conv Naive Implementation

$$Out[i, j] = \sum_{s=0}^{k-1} \sum_{t=0}^{k-1} In[i + s, j + t] \times Kernel[s, t] \quad (387)$$

Notice that given input rank-4 tensor $NHWC$ and kernel rank-4 tensor $KKIO$, the naive implementation contains 7-level loops (see Listing 38): (1) batch -> (2) channel in -> (3) channel out -> (4) h -> (5) w -> (6) kernel at h dim -> (7) kernel at w dim. When I implemented this 7-loop conv and compared against pytorch's Conv2D, and realized that if integer number can go over the float32 (23-bit Mantissa, 8-bit exponent, 1-bit sign) precision quite easily when we have multiple channels. 23-bit prevision is only around 8 million value. Thanks to Brian for the help!

Listing 38: "Naive 7-level loop impl of Conv"

```
def conv_naive(Z, weight):
    N,H,W,C_in = Z.shape
    K,_,_,C_out = weight.shape

    out = np.zeros((N,H-K+1,W-K+1,C_out));
    for n in range(N):
        for c_in in range(C_in):
            for c_out in range(C_out):
                for y in range(H-K+1):
                    for x in range(W-K+1):
                        for i in range(K):
                            for j in range(K):
                                out[n,y,x,c_out] += Z[n,y+i,x+j,c_in
                                ↪ ] * weight[i,j,c_in,c_out]

    return out
```

Convolutions as Matmul Two things:

1. When $k=1$ Convolution degenerates to matmul over the channel dimensions
2. We can think of Convolution as an overlay (叠加) of matmuls when $k > 1$

Notice we can

1. First think of when $k=1$, $C_{in}=C_{out}=1$, then it is nothing but a scalar multiplication (scalar being the kernel) with the input tensor.
2. Next, we can think when $k>1$, $C_{in}=C_{out}=1$, then it is like $k \times k$ matmuls overlay to construct the final output size $(H - k + 1) \times (W - k + 1)$ image. Illustrated in Listing 39. Notice since $C_{in}=C_{out}=1$, the @ in Listing 39 is degenerated to a scalar multiplication. This step requires some thinking, but it is not too hard. Take a 3×3 kernel as an example, convolution process is by summing up 9 $(H - 2) \times (W - 2)$ output images to become one $(H - 2) \times (W - 2)$ image. Each one of the 9 output images is obtained by cropping a $(H - 2) \times (W - 2)$ image at the offset (i, j) from the input image and then multiply it with the scalar at kernel matrix (i, j) .
3. In parallel to the bullet point 2, we can think when $k=1$, $C_{in}>1$, $C_{out}=1$, then the input is a rank-3 $(H \times W \times C_{in})$ tensor, the weight (kernel) is a $C_{in} \times 1$ vector. The convolution is now a simple tensor multiplication – one can think of each output image pixel at position (i, j) is the result of an inner product of $\text{Input}[i, j]$ with the kernel vector. Thus, the formula in Listing 39 still holds.

Listing 39: Convolutions as overlay of matmul. Notice when $K=1$, the convolution is equivalent to performing matrix multiplication over the (input/output) channel dimensions. When $k > 1$ and input channel and output channel are 1, one can think of convolution as overlaying (i.e. summing over) the multiplication result of input image from the (i,j) offset w/ kernel (i,j) to get the final output image.

```
def conv_matrix_mult(Z, weight):
    N,H,W,C_in = Z.shape
    K,_,_,C_out = weight.shape
    out = np.zeros((N,H-K+1,W-K+1,C_out))

    for i in range(K):
        for j in range(K):
            out += Z[:,i:i+H-K+1,j:j+W-K+1,:] @ weight[i,j]
    return out
```

4. We now consider when $k=1$, $C_{in} > 1$, $C_{out} > 1$. The convolution output will be a $H \times W \times C_{out}$ tensor. Each $H \times W$ image along the C_{out} axis is a result of a last bullet point calculation, thus can be calculated as a matrix multiplication. It is straightforward to see that the formula in Listing 39 still holds in this case.
5. Finally, we can think when $k > 1$, $C_{in} > 1$ and $C_{out} > 1$, combine bullet point 2 and 4, we can see Listing 39 holds for this general case.

Even though this piece of code is not really that relevant to what our final implementation required in homework, which is based on `im2col`, I still think this is a cute interpretation of what does Conv do. [weiz 2024-09-10](#). **Convolution is a linear operator (the input is the image, and the operator is to apply the kernel). So to prove this operator is linear, we just need to prove:**

1. $\alpha * \text{conv}(\text{img}) = \text{conv}(\alpha * \text{img})$
2. $\text{conv}(\text{img1} + \text{img2}) = \text{conv}(\text{img1}) + \text{conv}(\text{img2})$

The first property (i.e., Homogeneity) is pretty straightforward. The 2nd property (Additivity) is easier to prove to use the convolution interpretation used in Listing 39 (on a fair note, even using the conventional interpretation of convolution, it is not too difficult to prove convolution Additivity).

im2col (Matlab) `im2col` is a matlab command that given an input 2D image $H \times W$ and a kernel $k \times k$ returns an 2D image $(k \times k) \times ((H - k + 1) \times (W - K + 1))$ that can be readily multiplied by the vectorized kernel a size $(k \times k)$ row vector.

E.g., First, create a matrix A by reshaping the linearly spaced vector from 0 to 1 into a 4×4 matrix and then transpose it:

$$A = \text{reshape}(\text{linspace}(0, 1, 16), [4, 4])'^{31}$$

The matrix A is:

$$A = \begin{bmatrix} 0 & 0.0667 & 0.1333 & 0.2000 \\ 0.2667 & 0.3333 & 0.4000 & 0.4667 \\ 0.5333 & 0.6000 & 0.6667 & 0.7333 \\ 0.8000 & 0.8667 & 0.9333 & 1.0000 \end{bmatrix}$$

Next, rearrange the values of A into a column-wise arrangement using a block size of 2×2 :

$$B = \text{im2col}(A, [2, 2])$$

The resulting matrix B is:

$$B = \begin{bmatrix} 0 & 0.2667 & 0.5333 & 0.0667 & 0.3333 & 0.6000 & 0.1333 & 0.4000 & 0.6667 \\ 0.2667 & 0.5333 & 0.8000 & 0.3333 & 0.6000 & 0.8667 & 0.4000 & 0.6667 & 0.9333 \\ 0.0667 & 0.3333 & 0.6000 & 0.1333 & 0.4000 & 0.6667 & 0.2000 & 0.4667 & 0.7333 \\ 0.3333 & 0.6000 & 0.8667 & 0.4000 & 0.6667 & 0.9333 & 0.4667 & 0.7333 & 1.0000 \end{bmatrix}$$

In this example, the input image is 4×4 , and the kernel is 2×2 . The output image would be 3×3 . Notice the first 2×2 tile of A is

$\begin{pmatrix} 0 & 0.0667 \\ 0.2667 & 0.3333 \end{pmatrix}$, apparently after `im2col`, matlab vectorize this tile exactly the way Maguns [8] vectorizes a matrix (i.e., append each columns of a matrix)

so that the 1st column of B is $\begin{pmatrix} 0 \\ 0.2667 \\ 0.0667 \\ 0.3333 \end{pmatrix}$

Apply K (kernel) to B (input), KB is a 1×9 vector. (K is the vectorized kernel in a row vector, of size 1×4). The transpose of KB is the vectorized format of final output 3×3 image.

Notice Matlab uses column vector, so each image is a column vector, and we apply K to B (e.g., $K @ B$). In Zico (PyTorch) treatment, each image is a row vector, it would be " $B @ K$ " (see Equation (391) in next paragraph).

im2col in 2D (Zico) Example: Convolve a 6×6 image A with a 3×3 kernel W .

1. Notice that the output image D would be a 4×4 image (See Equation (385) and Equation (386)). Notice we need to arrange W to a 9×1 vector by applying Equation (388).

^{31/} means transpose, apparently Matlab treats each vector as a column vector. W/o transpose, the 1st column of A would have been $[0, 0.0667, 0.1333, 0.2000]'$

2. As a result, we need to arrange **A** to 16x9 matrix by (1) first, re-stride **A** (6x6) to a larger tensor of size 4x4x3x3 as in Equation (389). Notice **(6,1,6,1) – 6 is the strides of H, 1 is the strides of W.** and (2) reshape 4x4x3x3 tensor to a 16x9 matrix by applying Equation (390).
3. Apply matmul by Equation (391)
4. Reshape the final result by Equation (392)

$$W' = W.reshape(9,) \quad (388)$$

$$B = as_strided(A, shape = (4, 4, 3, 3), strides = (6, 1, 6, 1)) \quad (389)$$

$$C = B.reshape(16, 9) \quad (390)$$

$$D = C @ W' \quad (391)$$

$$E = D.reshape(4, 4) \quad (392)$$

im2col in 4D (Zico) Now we need to handle the multi-channel case, i.e. the input **Z** is of NHWC, kernel **K** is KKIO. Follow Listing 40 and notice that the key line is the

`as_strided(Z, shape = (N, H-K+1, W-K+1, K, K, C_in), strides = (Ns, Hs, Ws, Hs, Ws, Cs))`
which is the same as Equation (389), w/ two caveats

1. the **N** and **C** dimension strides keep the same, as they are not affected (and rather, independent). weiz 2024-09-07: **N, C** 以外的4个dimension取址都在本dimension内部通过strides解决了(无论是im2col之前还是之后), 所以N,C这两个dimension和其它dimension都是独立的- im2col之前每个element对应的N,C上的index是什么, im2col之后N,C上还是相应的index
2. `(..., Hs, Ws, Hs, Ws, ...)` part is exactly the same as the `(6,1,6,1)` in Equation (389). **Hs** is the stride of the **H** dimension in **Z**, **Ws** is the stride of the **W** dimension in **Z**. Notice we consider conv stride is 1 here.

Listing 40: im2col 4D

```
def conv_im2col(Z, weight):
    N,H,W,C_in = Z.shape
    K,_,_,C_out = weight.shape
    Ns, Hs, Ws, Cs = Z.strides

    inner_dim = K * K * C_in
    A = np.lib.stride_tricks.as_strided(Z, shape = (N, H-K+1, W-K
        ↳ +1, K, K, C_in), strides = (Ns, Hs, Ws, Hs, Ws, Cs)).
        ↳ reshape(-1,inner_dim)
    out = A @ weight.reshape(-1, C_out)
    return out.reshape(N,H-K+1,W-K+1,C_out)
```

im2col in 4D, w/o input padding and conv strides (Zico)

Conv op compute() (w/o padding and strides) Listing 41 is the forward() computation for Conv when there is no padding nor strides. It implements exactly the same logic as Listing 40 w/ two caveats (first one being **needle**-specific and the second one is just due to :

1. When we `reshape()` an `NDArray`, we need to make sure that `NDArray` is compact (see line 10).
2. I used `W` instead of `W_kernel` in line 12, as a result line 15 is not the integer defined in line 4

Listing 41: Conv compute() no padding, no strides

```
1 # weiz 2024-07-30, A is Z, B is W
2 def compute(self, A, B):
3     ### BEGIN YOUR SOLUTION
4     N,H,W, C_in = A.shape
5     Ns,Hs,Ws,C_ins = A.strides
6     K, _, _, C_out = B.shape
7     inner_dim = K * K* C_in
8     Z_shape = (N, H-K+1, W-K+1, K, K, C_in)
9     Z_strides = (Ns,Hs,Ws,Hs,Ws, C_ins)
10    Z = A.as_strided(shape=Z_shape, strides=Z_strides).compact()
11    Z = Z.reshape((N*(H-K+1)*(W-K+1), inner_dim))
12    W_kernel = B.compact().reshape((inner_dim, C_out)) # weiz
        ↳ 2024-09-08, bug fix , I was using W in LHS, and W is
        ↳ unfortunately also used\
13    as in shape calculation two lines below
14    out = Z @ W_kernel
15    out = out.reshape((N, H-K+1, W-K+1, C_out))
```

```

16 |     return out
17 |     ### END YOUR SOLUTION

```

Conv op compute() (padding and strides) weiz 2024-09-10, convolution strides doesn't change input or kernel, it simply slides kernel on the image with a stride so that the output image is of less resolution (i.e., smaller than input image). As usual, padding is to pad input image.

Two things worth noting:

1. `pad` in `NDArray` takes a tuple (of the size size as the same number of axes), each element in the tuple is a pair that describes the padding for the height and weight.
2. The purpose of doing strided convolution is to down-sample the images. We would simply need to figure out the right down-sampled height and width of the output images (see line 12,13 in Listing 42 and corresponding Equation (393) and Equation (394) ("`//`" means floor division) and the rest is almost the same as in Listing 41 except setting the higher dimension of H,W strides properly (see line 15,16 in Listing 42).

$$H_{output} = ((H - K) // stride) + 1 \quad (393)$$

$$W_{output} = ((W - K) // stride) + 1 \quad (394)$$

Here we prove Equation (394), notice similar as what Jiamai learnt in skip-counting,

- We start at 0, then the first pixel starts at 0, and the x-th pixel starts at x-1. A width of x image occupies number line from 0 to x

Now, we can model the question as how many pixels (let this variable be x) can we fit in the input, that is to say the last pixel (starting at co-ordinate $x-1$) plus kernel size should still be within W (aka Equation (395))

$$(x - 1) \times S + K \leq W \quad (395)$$

$$Sx - S + K \leq W$$

$$Sx \leq W - K + S$$

$$x \leq \frac{W - K + S}{S}$$

$$x \leq \frac{W - K}{S} + 1$$

$$x = \left\lfloor \frac{W - K}{S} + 1 \right\rfloor$$

$$x = \left\lfloor \frac{W - K}{S} \right\rfloor + 1 \quad \square \quad (396)$$

Example: $W=5, K=3, S=2$. There would be 2 output pixels along horizontal direction ⁽³²⁾

Listing 42: Conv compute() with input padding and convolution strides

```

1 def compute(self, A, B):
2     ### BEGIN YOUR SOLUTION
3     # pad A first
4     A = A.pad( ((0,0), (self.padding, self.padding), (self.padding
5         ↪ , self.padding), (0,0)) )
6
7     N,H,W, C_in = A.shape
8     Ns,Hs,Ws,C_ins = A.strides
9     K, _, _, C_out = B.shape
10    inner_dim = K * K* C_in
11
12    # when there is convolution striding
13    H_output = ((H-K) // self.stride) + 1 # weiz bug fix
14    ↪ 2024-09-16, previously it was (H-K+1) // self.stride, H
15    ↪ =5,K=3,stride=2 would fail
16    W_output = ((W-K) // self.stride) + 1 # weiz bug fix
17    ↪ 2024-09-16, previously it was (W-K+1) // self.stride, W
18    ↪ =5,K=3,stride=2 would fail
19    Z_shape = (N, H_output, W_output, K, K, C_in) # when no
20    ↪ striding
21    Z_strides = (Ns,Hs * self.stride,Ws * self.stride,Hs,Ws, C_ins
22    ↪ )
23    Z = A.as_strided(shape=Z_shape, strides=Z_strides).compact()
24    Z = Z.reshape(( N*H_output*W_output, inner_dim ))
25    W_kernel = B.reshape((inner_dim, C_out)) # weiz 2024-09-08,
26    ↪ bug fix , I was using W in LHS, and W is unfortunately
27    ↪ also used\
28    as in shape calculation two lines below
29    out = Z @ W_kernel
30    out = out.reshape((N, H_output, W_output, C_out))
31    return out
32    ### END YOUR SOLUTION

```

weiz Conv forward finished on 2024-09-10

7.8.6 Tricks in Convolution

This section

1. Padding: applied to input. Purpose: output image can be the same size as the input image

³²If we use PKU solution $(W-K+1)//S$, then it would be 1, which is wrong. weiz 2024-09-18, I learnt this when studying

2. Striding: neither input nor kernel is changed, just slides kernel with a bigger stride on top of input image. Purpose: output image can be a down-sample of input image
3. Dilation: applied to kernel. Purpose: kernel works with a larger region of input image (aka larger receptive field)
4. Grouped Convolution: Applied to kernels so that groups of channels in output only depend on corresponding groups of channels in input (equivalently, enforce filter weight matrices to be block-diagonal). Purpose: saving computation, sorta like multi-head purpose in multi-head attention.

7.8.7 Convolution Backward

Two takeaways from Zico's class:

1. Adjoint (of Loss) w.r.t input X is to apply convolution to `out_grad` (as input) and flipped kernel W . See Equation (397). Its Jacobian is sparse, and it makes more sense to use Conv operation here when doing the multiplication to calculation matrix.
2. Adjoint (of Loss) w.r.t kernel W is to apply some kind of `img2col` on input X (**TODO!!**). Its Jacobian is sparse and it makes more sense to use `im2col` and get the dense matrix to do the matmul directly.

$$\bar{v} \frac{\partial \text{Conv}(x, W)}{\partial x} = \text{Conv}(\bar{v}, \text{flip}(W)) \quad \bar{v} \text{ is out_grad} \quad (397)$$

We can **vec** input X and filter F exactly the way done in The process of studying convolution backward:

1. stride=1, loss w.r.t Kernel
 - (a) padding = 0
 - (b) padding > 0
 - (c) Figure how to make placement change (e.g., via **permutation**) to work with the NHWC, KKIO
2. stride=1, loss .w.r.t Input
 - (a) padding = 0
 - (b) padding > 0
 - (c) Figure how to make placement change (e.g., via **permutation**) to work with the NHWC, KKIO
3. Above two items with stride > 1

The first two items, running an example of ($\mathbf{X}$: 3 by 3, $\mathbf{F}$: 2 by 2), should pass many test cases already.

The 3rd item, running an example of ($\mathbf{X}$: 4 by 4, $\mathbf{F}$: 2 by 2) should involve only 1 line of additional coding on top of the first two items.

Notice that fundamentally we just use the `vec()` operator (and `de-vec()` to put it back into the gradients form) in [8] to calculate Jacobians of convolution operator.

1.a Gradient of Loss w.r.t Kernel Assume Input $\mathbf{X}$ is 3×3

$$\begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}$$

and the Kernel $\mathbf{F}$ is 2×2

$$\begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix}$$

Then the output $\mathbf{Y}$ is 2×2

$$\begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix}$$

where, Y_{11} is the convolution of the boxed 4 $\mathbf{X}$ elements (see below) w/ $\mathbf{F}$

$$X_{T_{11}} = \begin{bmatrix} \boxed{\begin{matrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{matrix}} & X_{13} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \quad (398)$$

$$X_{T_{21}} = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ \boxed{\begin{matrix} X_{21} & X_{22} \\ X_{31} & X_{32} \end{matrix}} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \quad (399)$$

$$X_{T_{12}} = \begin{bmatrix} X_{11} & \boxed{\begin{matrix} X_{12} & X_{13} \\ X_{22} & X_{23} \end{matrix}} \\ X_{21} & X_{32} & X_{33} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \quad (400)$$

$$X_{T_{22}} = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & \boxed{\begin{matrix} X_{22} & X_{23} \\ X_{32} & X_{33} \end{matrix}} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} \quad (401)$$

Then

$$\left[\begin{array}{c|c|c|c} \text{vec}(X_{T_{11}}) & \text{vec}(X_{T_{21}}) & \text{vec}(X_{T_{12}}) & \text{vec}(X_{T_{22}}) \end{array} \right] = \underbrace{\begin{bmatrix} X_{11} & X_{21} & X_{21} & X_{22} \\ X_{21} & X_{31} & X_{22} & X_{32} \\ X_{12} & X_{22} & X_{13} & X_{23} \\ X_{22} & X_{32} & X_{23} & X_{33} \end{bmatrix}}_{\text{im2col}(X,[2,2])} \quad (402)$$

Equation (402) is precisely **im2col(Matlab)** that we described earlier in Section 7.8.5!!!

Equation (398) is $X_{T_{11}}$ Then

$$\underbrace{\begin{bmatrix} X_{11} & X_{21} & X_{21} & X_{22} \\ X_{21} & X_{31} & X_{22} & X_{32} \\ X_{12} & X_{22} & X_{13} & X_{23} \\ X_{22} & X_{32} & X_{23} & X_{33} \end{bmatrix}}_{\text{im2col}(X)} \begin{bmatrix} F_{11} \\ F_{21} \\ F_{12} \\ F_{22} \end{bmatrix} = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{12} \\ Y_{22} \end{bmatrix} \quad (403)$$

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial Y} \frac{\partial Y}{\partial F} \quad (404)$$

$$= \left[\frac{\partial L}{\partial Y_{11}} \quad \frac{\partial L}{\partial Y_{21}} \quad \frac{\partial L}{\partial Y_{12}} \quad \frac{\partial L}{\partial Y_{22}} \right] \begin{bmatrix} X_{11} & X_{21} & X_{21} & X_{22} \\ X_{21} & X_{31} & X_{22} & X_{32} \\ X_{12} & X_{22} & X_{13} & X_{23} \\ X_{22} & X_{32} & X_{23} & X_{33} \end{bmatrix} \quad (405)$$

$$= \left[\langle \text{vec}(\frac{\partial L}{\partial Y}), \text{vec}(X_{T_{11}}) \rangle \quad \langle \text{vec}(\frac{\partial L}{\partial Y}), \text{vec}(X_{T_{21}}) \rangle \quad \langle \text{vec}(\frac{\partial L}{\partial Y}), \text{vec}(X_{T_{12}}) \rangle \quad \langle \text{vec}(\frac{\partial L}{\partial Y}), \text{vec}(X_{T_{22}}) \rangle \right] \quad (406)$$

$$= \text{vec} \left(\underbrace{\text{Conv}(X, \frac{\partial L}{\partial Y})}_{\text{def of Conv w/ img2col}} \right)^T \quad (407)$$

As a result, we can write (by using vec, de-vec in [8])

$$\nabla_F L = \text{Conv}(X, \nabla_Y L) \quad (408)$$

1.b Gradient of Loss w.r.t Kernel, w/ padding We provide two alternatives to figure out the paddings.

Alternative 1: In the derivation of Equation (408), X is the padded original input X_{orig} , so we have

$$X = \text{pad}(X_{orig}, p) \quad (409)$$

$$\begin{aligned} \nabla_F L &\stackrel{\text{Equation (408)}}{=} \text{Conv}(X, \nabla_Y L) \\ &= \text{Conv}(\text{pad}(X_{orig}, p), \nabla_Y L) \\ &= \text{Conv}(X_{orig}, \nabla_Y L, \text{padding} = p) \end{aligned} \quad (410)$$

Alternative 2:

$$W_{x_{orig}} + 2p - k + 1 \stackrel{\text{Equation (386)}}{=} W_y \quad (411)$$

$$k \stackrel{(408), 386}{=} W_{x_{orig}} + 2p' + 1 - W_y \quad (412)$$

$$k = k - 1 - 2p + 2p' + 1 \quad (413)$$

$$\Rightarrow p = p' \quad (414)$$

That is $p' = p$ is what the amount of padding applied to X_{orig} in Equation (408)

1.3b Placement change(摆摆好) In this paragraph, we consider making the proper permutation of tensor shapes to work w/ higher-dimension input. Recall

$$\text{shape}(X) = NH_X W_X C_i$$

$$\text{shape}(F) = KK C_i C_o$$

$$\text{shape}(Y) = NH_Y W_Y C_o$$

$$H_Y = H_X + 2p + 1 - k$$

$$W_Y = W_X + 2p + 1 - k$$

$$\nabla_F L = \text{Conv}(X, \nabla_Y L)$$

In order for X to convolve w/ $\nabla_Y L$, we know that the "kernel" in $\nabla_Y L$ should be $H_Y \times W_Y$, the "input channel dimension" (which input and kernel must agree on) can only be N . As a result, $\nabla_Y L$, used as a kernel, must be of the following shape

$$\text{shape}(\nabla_Y L^{\text{Perm33}}) = \underbrace{H_Y}_k \underbrace{W_Y}_k \underbrace{N}_{C_i} \underbrace{C_o}_{C_o} \quad (415)$$

This is done by `permute(  $\nabla_Y L$  , axis=(1,2,0,3))`

Consequently, we need to permute X in the following way

$$\text{shape}(X^{\text{Perm}}) = \underbrace{C_i}_N \underbrace{H_X}_{H_X} \underbrace{W_X}_{W_X} \underbrace{N}_{C_i} \quad (416)$$

This is done by `permute(X, axis=(3,1,2,0))`

The convolved result looks like

$$\text{shape}(\nabla_F L^{\text{Perm}}) = \underbrace{C_i}_N \underbrace{K_H}_H \underbrace{K_W}_{W}^{34} \underbrace{C_o}_C \quad (417)$$

³³Perm means permuted

³⁴We use K_H, K_W

To permute it back to the proper $\nabla_F L$ shape, we need to `permute( $\nabla_F L^{Perm}$ , axis=(1,2,0,3))`
To summarize,

$$X^{perm} \stackrel{416}{=} \text{permute}(X, \text{axis} = (3, 1, 2, 0)) \quad (418)$$

$$\nabla_Y L^{perm} \stackrel{415}{=} \text{permute}(\nabla_Y L, \text{axis} = (1, 2, 0, 3)) \quad (419)$$

$$\nabla_F L^{perm} \stackrel{408}{=} \text{conv}(X^{perm}, \nabla_Y L^{perm}, \text{padding} = \text{self.p}) \quad (420)$$

$$\nabla_F L \stackrel{417}{=} \text{permute}(\nabla_F L^{perm}, \text{axis} = (1, 2, 0, 3)) \quad (421)$$

2.a Gradient of loss w.r.t Input In this paragraph, we use the same running example of Input X 3×3 and Filter F 2×2 , and output Y 2×2 .

$$\text{vec}(X) = \begin{bmatrix} X_{11} \\ X_{21} \\ X_{31} \\ X_{12} \\ X_{22} \\ X_{23} \\ X_{13} \\ X_{23} \\ X_{33} \end{bmatrix} \quad (422)$$

$$\text{vec}(Y) = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{12} \\ Y_{22} \end{bmatrix} \quad (423)$$

$$\begin{bmatrix} F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 \\ 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 & 0 & 0 \\ 0 & 0 & 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 \\ 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} \end{bmatrix} \begin{bmatrix} X_{11} \\ X_{21} \\ X_{31} \\ X_{12} \\ X_{22} \\ X_{23} \\ X_{13} \\ X_{23} \\ X_{33} \end{bmatrix} = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{12} \\ Y_{22} \end{bmatrix} \quad (424)$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \frac{\partial Y}{\partial X} \quad (425)$$

$$= \begin{bmatrix} \frac{\partial L}{\partial Y_{11}} & \frac{\partial L}{\partial Y_{21}} & \frac{\partial L}{\partial Y_{12}} & \frac{\partial L}{\partial Y_{22}} \end{bmatrix} \begin{bmatrix} F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 \\ 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 & 0 & 0 \\ 0 & 0 & 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} & 0 \\ 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & F_{12} & F_{22} \end{bmatrix} \quad (426)$$

From Equation (426), we can see $\nabla_X L$ (de-vec of Equation (426)) can be calculated by the convolution of Equation (427) and Equation (428). **In**

particular, (1) Equation (427) is X padded by 0s and (2) more importantly Equation (428) is F flipped in both axes.

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & \frac{\partial L}{\partial Y_{11}} & \frac{\partial L}{\partial Y_{12}} & 0 \\ 0 & \frac{\partial L}{\partial Y_{21}} & \frac{\partial L}{\partial Y_{22}} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (427)$$

$$\begin{bmatrix} F_{22} & F_{21} \\ F_{12} & F_{11} \end{bmatrix} \quad (428)$$

$$\nabla_X L = \text{conv}(\text{pad}(\nabla_Y L, \text{pad} = 1), \text{flip}(F)) \quad (429)$$

Equation (429) calculates $\nabla_X L$, in next paragraph, we calculate how to pad $\nabla_Y L$.

weiz 2024-10-05 Now a key question is how much padding on $\nabla_Y L$. Does it have anything to do with input size or kernel size, or both.

$$H_Y \stackrel{385}{=} H_X + 1 - K \quad (430)$$

$$H_X = H_Y + 2p - K + 1 \quad (431)$$

$$\Rightarrow p = k - 1 \quad (432)$$

We can use induction method.

Let's consider kernel size 1.

$$\text{Diag}(F_{11}) \begin{bmatrix} X_{11} \\ X_{21} \\ X_{31} \\ X_{12} \\ X_{22} \\ X_{23} \\ X_{13} \\ X_{23} \\ X_{33} \end{bmatrix} = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{31} \\ Y_{12} \\ Y_{22} \\ Y_{23} \\ Y_{13} \\ Y_{23} \\ Y_{33} \end{bmatrix} \quad (433)$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \frac{\partial Y}{\partial X} \quad (434)$$

$$= \left[\frac{\partial L}{\partial Y_{11}} F_{11} \quad \frac{\partial L}{\partial Y_{21}} F_{11} \quad \frac{\partial L}{\partial Y_{31}} F_{11} \quad \frac{\partial L}{\partial Y_{12}} F_{11} \quad \frac{\partial L}{\partial Y_{22}} F_{11} \quad \frac{\partial L}{\partial Y_{23}} F_{11} \quad \frac{\partial L}{\partial Y_{13}} F_{11} \quad \frac{\partial L}{\partial Y_{23}} F_{11} \quad \frac{\partial L}{\partial Y_{33}} F_{11} \right] \quad (435)$$

$$\Rightarrow \nabla_X L = \text{conv}(\nabla_Y L, F = \text{flip}(F), \text{pad} = k - 1 = 0) \quad (436)$$

kernel size 2 was the running example

kernel size 3

$$\begin{bmatrix} F_{11} & F_{21} & F_{31} & F_{12} & F_{22} & F_{23} & F_{13} & F_{23} & F_{33} \end{bmatrix} \begin{bmatrix} X_{11} \\ X_{21} \\ X_{31} \\ X_{12} \\ X_{22} \\ X_{23} \\ X_{13} \\ X_{23} \\ X_{33} \end{bmatrix} = [Y_{11}] \quad (437)$$

$$\Rightarrow \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y_{11}} \begin{bmatrix} F_{11} & F_{21} & F_{31} & F_{12} & F_{22} & F_{23} & F_{13} & F_{23} & F_{33} \end{bmatrix} \quad (438)$$

$$\Rightarrow \nabla_X L = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{\partial L}{\partial Y_{11}} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix} \quad (439)$$

$$\Rightarrow \nabla_X L = \text{conv}(\nabla_Y L, \text{flip}(F), \text{pad} = k - 1 = 2) \quad (440)$$

weiz 2024-10-04 kernel=3 might be the best case to study why we need to convolve with the flipped kernel. Also, we need to be able to provide an induction proof to say why k=n+1, it is also padding by k-1. TODO, but we are reasonably close enough to continue the homework.

Finally, we have

$$\nabla_X L = \text{conv}(\nabla_Y L, \text{flip}(F), \text{pad} = k - 1) \quad (441)$$

2.b Gradient of Loss w.r.t Input, w/ padding In this paragraph, we calculate how much padding to do if the input X is padded.

Theorem to prove

$$\text{unpad}(\text{Conv}(\text{pad}(X, p1), F), p2) = \text{Conv}(\text{unpad}(\text{pad}(X, p1), p2), F), p1 \geq p2^{35} \quad (442)$$

Lemma1

$$\text{unpad}(\text{Conv}(X, F), p) = \text{Conv}(\text{unpad}(X, p), F) \quad (443)$$

Proof

If we let $X' = \text{unpad}(X, p)$, this is to prove

$$\text{unpad}(\text{Conv}(\text{pad}(X', p), F), p) = \text{Conv}(X', F)$$

By renaming, i.e. $X' \rightarrow X$

$$\text{unpad}(\text{Conv}(\text{pad}(X, p), F), p) = \text{Conv}(X, F) \quad (444)$$

³⁵This is for simplicity

Recall that

$$Y = \text{Conv}(X, F) \quad (445)$$

$$Y[x][y] = \sum_{i=0}^{i=k-1} \sum_{j=0}^{j=k-1} X[x+i][y+j] \times F[i][j] \quad (446)$$

So when we unpad Y from top or left, we effectively get the original X position. Now consider we unpad from bottom or right, it will strip off the paddings until it uses the original X 's last row or column. As a result, Equation (444) is valid. In turn, Equation (443) is true, i.e., **Lemma1** is obtained.

Now, we simply let $X = \text{pad}(X_1, p_1)$ and apply Equation (443) (**Lemma1**). We then get

$$\text{unpad}(\text{Conv}(\text{pad}(X_1, p + p_1), p) = \text{Conv}(\text{pad}(X_1, p_1), F) \quad (447)$$

Let

$$\begin{cases} p1' = p + p_1 \\ p2' = p \end{cases} \quad (448)$$

Then we have

$$\begin{cases} p = p2' \\ p_1 = p1' - p2' \end{cases} \quad (449)$$

$$\Rightarrow \quad (450)$$

$$\begin{cases} \text{pad}(X_1, p + p_1) = \text{pad}(X_1, p1') \\ \text{pad}(X_1, p_1) = \text{pad}(X_1, p1' - p2') = \text{unpad}(\text{pad}(X_1, p1'), p2') \end{cases} \quad (451)$$

$$\stackrel{447}{\Rightarrow} \text{unpad}(\text{Conv}(\text{pad}(X_1, p1'), p2') = \text{Conv}(\text{unpad}(\text{pad}(X_1, p1'), p2')) \quad (452)$$

$$\stackrel{\text{Rename } X_1 \text{ to } X}{p1' \text{ to } p1, p2' \text{ to } p2}{\Rightarrow} \quad (453)$$

$$\text{unpad}(\text{Conv}(\text{pad}(X, p1), p2)) = \text{Conv}(\text{unpad}(\text{pad}(X, p1), p2)) \quad \square \quad (454)$$

Now we combine we apply Equation (442), here X is $\nabla_Y L$ ($Y = \text{conv}(\text{Input}, F, \text{pad} = p_2)$), p_1 is the $k-1$ padding used in Equation (441). Then $\text{unpad}(\text{Conv}(\text{pad}(\nabla_Y L, p_1), F), p_2)$ is precisely $\nabla_{\text{Input}} L$; and $\text{Conv}(\text{unpad}(\text{pad}(X, p_1), p_2), F)$ is $\text{conv}(X, F, \text{pad} = p_1 - p_2)$

Verification of Theorem

$$W_Y = W_X + 2p - k + 1 \quad (455)$$

$$W_X = W_Y + 2 + 2p' - k + 1 \quad (456)$$

$$\Rightarrow W_Y = W_Y + 2 + 2p' - k + 1 + 2p - k + 1 \quad (457)$$

$$\Rightarrow 2k = 2(p' + p + 1) \quad (458)$$

$$\Rightarrow p' = k - p - 1 \quad (459)$$

weiz 2024-10-04 In this paragraph we first proved why $p'=p-k-1$, and then verify it is the case. The verification alone is just a necessary (but not sufficient) condition and not enough to be a proof. In what we did in the last paragraph of Equation (432) – it is just a verification it is not a proof. Unfortunately, we haven't come up w/ a strong proof in the last paragraph for padding $\nabla_Y L$ and the padding size $k-1$ yet.

Finally, we have

$$\begin{aligned} Y &= \text{conv}(X, F, \text{pad} = p) \\ \text{shape}(F) &= k \times k \\ \nabla_X L &= \text{conv}(\nabla_Y L, \text{flip}(F), \text{pad} = k - 1 - p) \end{aligned} \quad (460)$$

Equation (460) is what we are after $\square$

2.c Placement In this paragraph, we study how to permute tensors so that they work w/ the batched input and multi-channels (i.e., inputs with higher ranks).

Recall that

$$\begin{aligned} \text{shape}(X) &= NH_X W_X C_i \\ \text{shape}(\nabla_Y L) &= NH_Y W_Y C_o \\ \text{shape}(\text{flip}(F)) &= \text{shape}(F) = K K C_i C_o \end{aligned}$$

In order to conv $\nabla_Y L$ w/ flipped kernel, we just need to permute the last two axes of F_{flip} , conv, then the result will be in $NH_X H_Y C_i$, which is the desired X shape. **To summarize,**

$$F_{\text{flip}} = \text{flip}(F, (0, 1)) \quad (461)$$

$$F_{\text{flip}}^{\text{perm}} = \text{transpose}(F_{\text{flip}}) \quad (462)$$

$$\nabla_X L = \text{conv}(\nabla_Y L, F_{\text{flip}}^{\text{perm}}, \text{padding} = k - \text{self.p} - 1) \quad (463)$$

weiz: 2024-09-29, I implemented the `permute()` op in `ops_mathematics.py` to facilitate `conv backward()` operation. Notice that there is no `permute()` support in `numpy`, so it might become a problem if needles runs on a `numpy` backend.
weiz: 2024-09-29, I finished and passed all the tests that `stride=1`. Notice if were not for a stupid tooth infection caused by deep cleansing, I could have been here at least one week earlier!

Finally, Equation (463) is what this homework asks for $\nabla_X L$ when there is no striding and input is **NHWC** and kernel is **KKIO**.

3.a Gradient of Loss w.r.t Kernel, w/ striding=s Before talking about convolution w/ strides, I need to make one thing clear: Zico's definition of dilate is wrong for filters. We should only pad 0s between filter for dilation. For a

filter 2×2 , $\text{dilateF}(\mathbf{F})$ should look like $\begin{bmatrix} F_{11} & 0 & F_{12} \\ 0 & 0 & 0 \\ F_{21} & 0 & F_{22} \end{bmatrix}$, instead of Zico's always double-sizing dilation, which wastes computing at the all 0s boundaries when convolving.

We use $4 \times$ input convolves w/ a 2×2 kernel w/ a stride=2 as a running example.

$$\begin{bmatrix} X_{11} & X_{12} & X_{13} & X_{14} \\ X_{21} & X_{22} & X_{23} & X_{24} \\ X_{31} & X_{32} & X_{33} & X_{34} \\ X_{41} & X_{42} & X_{43} & X_{44} \end{bmatrix} *_{s_2} {}^{36} \begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \end{bmatrix} \quad (464)$$

$$\begin{bmatrix} X_{11} & X_{21} & X_{12} & X_{22} \\ X_{31} & X_{41} & X_{32} & X_{42} \\ X_{13} & X_{23} & X_{14} & X_{24} \\ X_{33} & X_{43} & X_{34} & X_{44} \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{21} \\ F_{12} \\ F_{22} \end{bmatrix} = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{12} \\ Y_{22} \end{bmatrix} \quad (465)$$

$$\frac{\partial L}{\partial \mathbf{F}} = \begin{bmatrix} \frac{\partial L}{\partial Y_{11}} & \frac{\partial L}{\partial Y_{21}} & \frac{\partial L}{\partial Y_{12}} & \frac{\partial L}{\partial Y_{22}} \end{bmatrix} \begin{bmatrix} X_{11} & X_{21} & X_{12} & X_{22} \\ X_{31} & X_{41} & X_{32} & X_{42} \\ X_{13} & X_{23} & X_{14} & X_{24} \\ X_{33} & X_{43} & X_{34} & X_{44} \end{bmatrix} \quad (466)$$

$$\nabla_F L = \text{conv}(\text{dilateF}(\nabla_Y L), X) \quad (467)$$

3.b Gradient of Loss w.r.t. Input, w/striding=s

$$\begin{bmatrix} F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} \end{bmatrix} \begin{bmatrix} X_{11} \\ X_{21} \\ X_{31} \\ X_{41} \\ X_{12} \\ X_{22} \\ X_{32} \\ X_{42} \\ X_{13} \\ X_{23} \\ X_{33} \\ X_{43} \\ X_{14} \\ X_{24} \\ X_{34} \\ X_{44} \end{bmatrix} = \begin{bmatrix} Y_{11} \\ Y_{21} \\ Y_{12} \\ Y_{22} \end{bmatrix} \quad (468)$$

$$\frac{\partial L}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial L}{\partial Y_{11}} & \frac{\partial L}{\partial Y_{21}} & \frac{\partial L}{\partial Y_{12}} & \frac{\partial L}{\partial Y_{22}} \end{bmatrix} \begin{bmatrix} F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & F_{11} & F_{21} & 0 & 0 & F_{12} & F_{22} \end{bmatrix} \quad (469)$$

³⁶We use $*_{s=2}$ to represent a convolution w/ stride=2.

As a result,

$$\begin{cases} \frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial Y_{11}} F_{11} \\ \frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial Y_{11}} F_{21} \\ \frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial Y_{21}} F_{11} \\ \frac{\partial L}{\partial X_{41}} = \frac{\partial L}{\partial Y_{21}} F_{21} \end{cases} \begin{cases} \frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial Y_{11}} F_{12} \\ \frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial Y_{11}} F_{22} \\ \frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial Y_{21}} F_{12} \\ \frac{\partial L}{\partial X_{42}} = \frac{\partial L}{\partial Y_{21}} F_{22} \end{cases} \begin{cases} \frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial Y_{12}} F_{11} \\ \frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial Y_{12}} F_{21} \\ \frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial Y_{22}} F_{11} \\ \frac{\partial L}{\partial X_{43}} = \frac{\partial L}{\partial Y_{22}} F_{21} \end{cases} \begin{cases} \frac{\partial L}{\partial X_{14}} = \frac{\partial L}{\partial Y_{12}} F_{12} \\ \frac{\partial L}{\partial X_{24}} = \frac{\partial L}{\partial Y_{12}} F_{22} \\ \frac{\partial L}{\partial X_{34}} = \frac{\partial L}{\partial Y_{22}} F_{12} \\ \frac{\partial L}{\partial X_{44}} = \frac{\partial L}{\partial Y_{22}} F_{22} \end{cases} \quad (470)$$

$$\begin{bmatrix} \frac{\partial L}{\partial X_{11}} & \frac{\partial L}{\partial X_{12}} & \frac{\partial L}{\partial X_{13}} & \frac{\partial L}{\partial X_{14}} \\ \frac{\partial L}{\partial X_{21}} & \frac{\partial L}{\partial X_{22}} & \frac{\partial L}{\partial X_{23}} & \frac{\partial L}{\partial X_{24}} \\ \frac{\partial L}{\partial X_{31}} & \frac{\partial L}{\partial X_{32}} & \frac{\partial L}{\partial X_{33}} & \frac{\partial L}{\partial X_{34}} \\ \frac{\partial L}{\partial X_{41}} & \frac{\partial L}{\partial X_{42}} & \frac{\partial L}{\partial X_{43}} & \frac{\partial L}{\partial X_{44}} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{\partial L}{\partial Y_{11}} & 0 & \frac{\partial L}{\partial Y_{12}} & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{\partial L}{\partial Y_{21}} & 0 & \frac{\partial L}{\partial Y_{22}} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} F_{22} & F_{21} \\ F_{12} & F_{11} \end{bmatrix} \quad (471)$$

Consequently, we have

$$\nabla_X L = \text{conv}(\text{dilate}(F(\nabla_Y L, \text{stride} - 1), \text{flip}(F), \text{pad} = k - 1) \quad (472)$$

Notice both `stride-1` used in dilation and `k-1` used in padding can be found some running example at <https://medium.com/@mayank.utexas/backpropagation-for-convolution-with-strides-8137e4fc2710>.³⁷ Notice that i have done something for the `k-1` padding in the last paragraph. **TODO: We might also be able do some induction base proof on these two arguments**

3.c Placement Also, we can reuse the arguments in the last paragraph for the tensor permutation to have the proper dimension aligned.

weiz 2024-10-09 I have finally finished the `conv backward()` implementation. **TODOS:**

1. implement the backward part of `dilatefilter` op
2. clean up code
3. layout the assumptions of padding (uneven and how it can cut the effective image) when `stride > 1`. Clean up my writing to summarize the key equations w/o thinking too much about padding. Argue what an awful organization of this homework really is.

³⁷ Another two useful links are: (1) strided convolution backward wrt kernel: <https://medium.com/@mayank.utexas/backpropagation-for-convolution-with-strides-fb2f2efc4faa> and (2) non-strided convolution : <https://pavisj.medium.com/convolutions-and-backpropagations-46026a8f5d2c>

4. Maybe write a Tensor slicing mechanism, i.e., we can index into `Tensor __getitem__` type of stuff. That may be even helpful for RNN and LSTMs.
5. Maybe include some tests to the test `_conv` suite so show that what Zico had in mind is a wrong way to implement Conv when there is a stride, an particularly his misunderstanding of dilate operation.

weiz 2024-10-09 at this point, I should be rather confident about taking derivatives of various ops. So I should be able to move on to the more computer-science-y parts of the homework (e.g., see how the NNs are built and write the RNN LSTM Transformer part).

7.8.8 nn.Conv

Fixing `init._calculate_fans` for convolution

Number of bias terms This paragraph deals with how many *bias* terms for each layer has. Recall as in Equation (473), bias term `b` has the same dimension as `y` (aka output).

$$y \in R^m, W \in R^{m \times n}, x \in R^n, b \in R^m$$

$$y = Wx + b \quad (473)$$

For the linear layer, each layer's neurons are the outputs, so if a linear layer is of size $m \times n$ (a sample is a row vector of dimension of m), the number of neurons is the number of outputs, i.e., n . **Linear layer of size $m \times n$, bias term number is n .**

For the Conv layer, the number of neurons of each *kernel* filter is o (aka the output channels size) because the output dimension of this filter is o , weiz 2024-10-17, notice BatchNorm, LayerNorm learnable parameters only work on outputs, as a result, learnable parameters of BatchNorm and LayerNorm after Conv layer is $2o$ (scaling parameter and shift parameter, thus 2). Also notice output dimension is the number of output filter maps, aka o . For example, if the output convolution of a filter $k \times k$ w/ input is simply a scalar value (aka the output dimension is 1).

Review NN library abstractions We need to briefly review the NN library abstractions lecture. Question: What is Parameter, what is Module, and how `forward()` automatically connects the Ops.

- `Tensor(Value)`: `backward()` iterates compute graph and calculates adjoints. E.g., a loss is a Tensor. `Value` is the node in compute graph.
- `TensorOp(Op)`: `Op` has `compute()` (input is `NDArray`, output is `NDArray`) `gradient()` (input is `Tensor`, output is `Tensor`). `__call__` (returns `Tensor` that is made of the returned values of `compute()` call). E.g., `matmul` and `Conv` are of `TensorOp`.

- **Parameters:** it is a subclass of `Tensor` and doesn't have any real implementations. It usually uses the copy constructor of a real `Tensor` to initialize it self, e.g., using `kaiming_init` returned `Tensor` to initialize the `Parameters`. E.g., linear weights, bias are all of `Parameters` type.
- **Module:** contains a list of `Parameters`, input is `Tensor` and output is `Tensor`, which you can call `backward()` to start the gradients calculation. `Module`'s `__call__()` will call into `Module`'s `forward()` function to calculate loss. E.g., `Linear`, `Flatten` and `ReLU` are all examples of `Module`. **Model:** Technically, there is no `Model` class, instead it is usually a group of `Modules` that embody a model architecture. `Sequential` (of `Module` class) represents how a typical `Model` is organized. E.g., `MLPResNet`, `ResNet9`

weiz: TODO draw a UML diagram of `Model`->list(of `Modules`)->list of (`Parameters`). `Parameters` are nothing but `Tensors`, initialized by copy constructors. `Module` `forward()` returns `Tensor`, which call into `backward()` to start the whole gradients calculation. `Tensors` are materialized using the `__call__()` into `compute()` of `TensorOp`, gradients `Tensors` are calculated using `gradient()` call in `TensorOp`. `Tensors` are of `Value` type, which are the nodes of the compute graph (which makes sense, as `backward()` in `Tensor` class does the traversal of compute graph and calculate adjoints and gradients.)

weiz 2024-10-12, <https://www.youtube.com/watch?v=uB81vGRrH0c> at 5:00 has the content that talks about unnecessary tensor tracking in some naive weights tensor update in `nn.Conv` if not handled properly, by TQ

weiz 2024-10-15 `nn.Conv` is done, but we should look at the test cases more carefully to see when `stride > 1`, are they really snuggy. Also we need to finish my writing of the `Conv` op just to articulate my principled way of doing `conv2d`, write out equations clearly (w/o considering padding). Also, try search for principled operator theory to explain why that `conv2d` backward is to `conv` w/ flipped kernel.

Conv(Module) In this paragraph, we describe how to implement `Conv` as a `Module`. Two things:

1. Initialize weights, which are of shape `kkio`. Optionally, `stride` can be passed
2. Initialize bias, which are of shape `o` (aka output channels)
3. `forward()`: Take the input tensor in the format of `NCHW` (aka `pytorch`), and call into `conv` ops, which deals with `NHWC` (aka `TF` and `Zico` format). In particular when dealing with bias, we need to first reshape it to `(1,1,1,o)` and then broadcast it explicitly to the output `NHWC_{o}` shape (due to we don't handle implicitly broadcast during summation yet). My `permute()` `TensorOp` came very handy for the conversion between `NCHW` and `NHWC`

Notice (1) in this homework, input tensor is of shape `NCHW` (to comply w/ `PyT`) but the filter shape is `kkio` (to comply w/ `TF`) – weird choice! `Zico`'s argument

about NCHW is to make BatchNorm easier, I am not clear on that yet. (2) The weird thing about this HW is that `conv` ops really is as powerful as `Conv2D` module in pytorch in that you can specify padding, but `Conv` module, which supposedly is the counterpart of `Conv2D` module has no control over padding, but dictates that every padding will be $(k-1)/2$ just to maintain when `stride=1`, input and output H and W dimensions will remain the same – Ridiculous!

7.8.9 ConvBN (Module)

Notice that the `BatchNorm` deals with `o` output units for a `kkio` filter weights! Recall `bias`, `BatchNorm`, `LayerNorm` and numbers of neurons all refer to the output channel size (aka number of output filter maps). [weiz 2024-10-20, make sure discuss how BatchNorm in Zico implementation work better with NHWC format \(or NCHW ?\)](#)

7.8.10 ResNet9 model

Dimension matchings . Cifar10 input tensor is 3x32x32 (channel, H, W). Recall that

$$H_Y = (H_X - K) // stride + 1$$

$$PAD = (K - 1) // 2 \text{ required by HW to match}$$

stride=1 dimensions, non-sense

Let's see when HW doesn't allow to specify padding, if the dimension and the assumption that padding is always $(k-1)/2$ makes sense. After calculation, it is actually true that before the last linear layer, the images spatial dimensions are downsampled to 1x1 (via strides and increasing output channels). Notice that [In practice, usually when we downsample an image, we would increase the output channels, so that the "representation power" remains constant.](#) In the original ResNet paper, it also uses (1) increasing output channels (2) stride and (3) always use a global averaging pooling in the end to make sure the image spatial dimension is 1x1 right before the linear projection layer! [weiz 2024-10-19 very nice learning about how the image spatial dimension downsampled and always get to 1 by 1 before linear layer :\)](#)

Let's figure out input/output dimension and parameter count for each layer Figure 11 illustrates the ResNet architecture this homework employs.

In total, 431,946 parameters.

Example (how to calculate): Layer1, padding=3 , output...

Also, my `ResidualBasicalBlock` let the main path as self part, which unnecessary blows up the trainable parameter counting (though the forward calculation would still be correct).

7.8.11 Handling NDArra of size (n,)

Notice the shape of `shape` is always the shape of `strides`

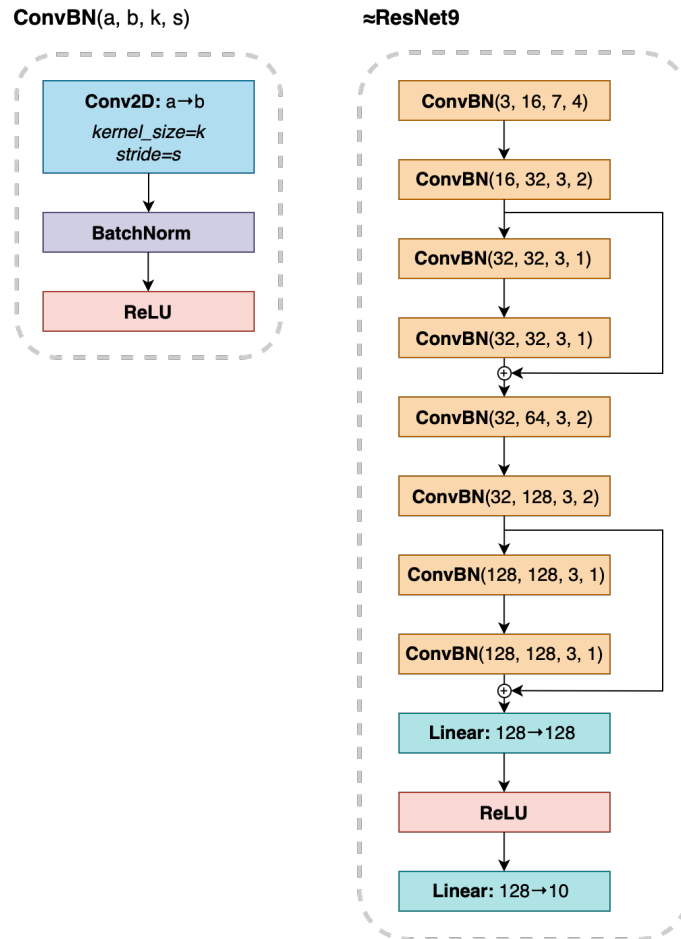


Figure 11: ResNet 9 Architecture this homework employs

Layer ID	Sub-modules (if Any)	Padding (if any)	Input Dim (bs=1)	Output Dim (bs=1)	Trainable Parameters
1	Conv(3,16,7,4)	3	3x32x32	16x8x8	2368 (weights and bias)
	BN	NA	16x8x8	16x8x8	32 (scale and shift)
	ReLU	NA	16x8x8	16x8x8	0
2	Conv(16,32,3,2)	1	16x8x8	32x4x4	4640
	BN	NA	32x4x4	32x4x4	64
	ReLU	NA	32x4x4	32x4x4	0
3	Conv1(32,32,3,1)	1	32x4x4	32x4x4	9248
	BN1	NA	32x4x4	32x4x4	64
	ReLU1	NA	32x4x4	32x4x4	0
4	Conv2(32,32,3,1)	1	32x4x4	32x4x4	9248
	BN2	NA	32x4x4	32x4x4	64
	ReLU2	NA	32x4x4	32x4x4	0
5	Conv(32,64,3,2)	1	32x4x4	64x2x2	18496
	BN	NA	64x2x2	64x2x2	128
	ReLU	NA	64x2x2	64x2x2	0
6	Conv(64,128,3,2)	1	64x2x2	128x1x1	73856
	BN	NA	128x1x1	128x1x1	256
	ReLU	NA	128x1x1	128x1x1	0
7	Conv1(128,128,3,1)	1	128x1x1	128x1x1	147584
	BN1	NA	128x1x1	128x1x1	256
	ReLU1	NA	128x1x1	128x1x1	0
8	Conv2(128,128,3,1)	1	128x1x1	128x1x1	147584
	BN2	NA	128x1x1	128x1x1	256
	ReLU2	NA	128x1x1	128x1x1	0
9	Flatten	NA	128x1x1	128	0
	Linear1(128,128)	NA	128	128	128x128+128=16512
	Linear2(128,10)	NA	128	10	128x10+10=1290

7.8.12 Summary of ConvNet

1. Zico has a nice Conv forward impl python notebook, espeically the usage of `as_strided()`
2. Use two CNN medium webpages to do Conv forward and backwrd. Then use SnuggyConv to handle striding. Zico's test cases on striding and conv suck. In particular, I need to use the right `dilate()` implementation (i.e., insert $k-1$ zeros in each dimension into the $k \times k$ filter, assuming dilation=1). **TODO: summarize (1) padding size and H W dimension when there is striding calculation (2) the backward conv equations wrt input and filter , first w/o striding and then w/striding, also explain how to 摆摆好**
3. Kernel out channles is the number of bias and feature size used in batch-norm. The out channels are the output filter feature map space.
4. in practice, usually downsample means increasing output channel (to maintain representation power similar). In the last conv layer before linear layer, we usually make the output channel to be exact 1. In ResNet paper, they use a Global Averaging Pooling (GAP) layer to downsample to exactly 1. Typically in ResNet paper, after the last conv layer, the output is of $N \times 7 \times 7 \times C$, then apply GAP of kernel size 7×7 , to get output $N \times 1 \times 1 \times C$ so now we can see that output features are exactly of size C . In this homework, it is a series of strided convolution downsampling.
5. padding is not training related, only used to match input and output dimension (e.g., for Residual component)
6. striding is sorta for downsampling the images
7. Notice Residual BottleNeck (which handles larger model and need to save some compute.) and Basic block.
8. TF uses NHWC and `kkio`; while PyT uses NCHW and `oikk`. In this homework, the `conv` ops uses NHWC and `kkio`, while `conv` module uses NCHW (arguably for the sake of BatchNorm) and `kkio`.

weiz 2024-11-09 We have passed all the ConvNets related tests. But we have OOM issues, see <https://github.com/weidotwisc/10714/issues/1>

7.9 Fix the hw4 on hw1, hw2,hw3 test cases

7.9.1 Why HW4 failed on HW1

If we use `numpy` as `NEEDLE_BACKEND` (i.e. by `export NEEDLE_BACKEND=np`), all the test cases would still pass. So the question is why `nd` backend failing hw1 test cases? The effort here is to understand the discrepancies between `needle.NDArray` and `numpy`.

1. Added support for `ewise_power` in `ndarray.py`

2. `matmul` test would fail, because `hw1` tests on tensors of rank more than 2 (`numpy` backend would be fine, but `ndarray` backend only supports 2D matrices multiplication. Luckily, all the `Conv` operators (even input is NCHW rank-4 tensors, the actual calculation all lower to 2D matrices by reshaping and take a giant `matmul`)
3. an assertion bug was fixed in `transpose()` in `ndarray.py` where it should have been asserted on `expanded_shape` instead of `self._shape` now that I support transpose from a lesser rank to a bigger rank of tensor.
4. **Important one** Should use `Tensor` operation instead of `NDArray` operation in `ops_mathematics.py`'s `gradient()` function (e.g., `PowerScalar`) The reason that `HW1` test cases can pass is that `array_api` is `numpy`, which in general, when `numpy.power` is called on an object that has its own `__pow__` method, `numpy` will attempt to use that method instead of its own internal handling (per chatgpt). As a result when I can call `array_api.power(Tensor t, 2)` (in `HW1`, `array_api` is `numpy`), `numpy` will first find if my `Tensor` class has the `__pow__` implemented, if so it will go call my `Tensor` class's `__pow__` implementation. Whereas in `HW4`, `array_api` is my `NDArray`, which doesn't implement this delegation policy, e.g., had I implemented the following in Listing 43, it might have worked. So it is really the weird Python dunder calling sequence that confused me
5. **Important one** **Numerical stability for the numerical_gradient_check()** **method** `HW1` tests on `numpy`, which uses `float64` as default. However, `NDArray`, which closely follows `PyTorch`, uses `float32` as default. The test cases in `hw1` use $\epsilon=1e-4$ and $tol=1e-6$ that are too strict for `float32`. For `float32`, I use $\epsilon=1e-3$ (actually $1e-2$ might be even safer, as sometimes even $1e-3$ might fail) and $tol=1e-2$, that seem to be the right experience number, which nobody has a clue (including Naigang) about, to use. Per <https://pytorch.org/docs/stable/generated/torch.autograd.gradcheck.gradcheck.html>, `Pytorch autograd` uses $\epsilon=1e-6$ and $tol=1e-5$, with the note The default values are designed for input of double precision. This check will likely fail if input is of less precision, e.g., `FloatTensor`. Listing 44 demonstrates when using `fp32`, even $\epsilon=1e-4$ has problems with identity function gradient (see `t2` and `t3` cannot be even properly displayed due to $1e-4$ cannot be accurately enough represented using `fp32`. In contrast, `fp64` has no such a issue. Which begs the question how the hell do people use `fp32` to do numerical gradient check()!
6. **Important one** The `softmax_loss` implemented in `hw1` returns a `summation Tensor`. When using `numpy` as a backend as in `hw1`, the `summation` drops axes of length 1 (e.g., via `squeeze()` method available in `numpy`), the end result is that tensor is of `shape()`, i.e., a scalar-value. This is desirable, as `hw1` relies on `numdifftools`'s `Gradient()` method to do the numerical gradient checking and `Gradient()` requires input `lambda` function to be a scalar-value function. Now if we use `hw4`'s `NDArray`

backend, the `summation` tensor will be of shape $(1,)$ ³⁸. We can simply call `numpy().squeeze()` operation on top of the summation tensor to get desired value for the `Gradient()` method.

Listing 43: A possible implementation in `NDArray` to delegate power to `Tensor` class

```

1 # Example of NDArray's power function
2 def power(base, exponent):
3     # Check if 'base' has a '__pow__' method
4     if hasattr(base, '__pow__'):
5         # Use the custom '__pow__' method if available
6         return base ** exponent
7     else:
8         # Otherwise, use the library internal power logic
9         # Assume internal_pow is your NDArray-specific
10        ↪ implementation for numeric types
        return internal_pow(base, exponent)

```

Listing 44: Numerical stability issue in `fp32`.

```

1 def weiz_test_fp32():
2     eps = np.float32(1e-4)
3     t1 = np.float32(1)
4     t2 = t1 + eps
5     t3 = t2 - np.float32(2)*eps
6     t4 = t3 + eps
7     total_err= np.abs((t2-t3)/ (np.float32(2)*eps) - np.float32(1)
8     ↪ )
9     print("Total_Error(fp32):", total_err)
10    print(f"t1={t1},t2={t2},t3={t3},t4={t4}")
11    $ Total Error(fp32): 0.0001320839
12    $ t1=1.0, t2=1.000100016593933, t3=0.9999000430107117, t4=1.0
13 def weiz_test_fp64():
14     eps = 1e-4
15     t1 = 1
16     t2 = t1 + eps
17     t3 = t2 - 2*eps
18     t4 = t3 + eps
19     total_err= np.abs( (t2-t3)/(2*eps) - 1)
20     print("Total_Error(fp64):", total_err)
21     print(f"t1={t1},t2={t2},t3={t3},t4={t4}")
22    $ Total Error(fp64): 1.1013412404281553e-13
23    $ t1=1, t2=1.0001, t3=0.9999, t4=1.0

```

³⁸I recall somewhere in the class or howework description, it is mentioned that even a scalar is represented as a $(1,)$ shaped `Tensor` in this course, I need to find out where it is mentioned later.

weiz 2024-11-02, hw4 now pass all hw1's test cases. What I find interesting is that (1) even though each batch incurred much larger discrepancy in terms of weights, gradients or loss between fp32 (NDArray backend) and fp64(numpy backend). The overall convergence when training the whole epochs remains almost exactly the same, i.e., I can use the same tolerance number for fp32 and fp64 for the tests that test on the whole epochs training. This shows that fp32 might really work for deep learning training. (2) ND_cuda (~48 sec for minist training) is faster than numpy (~1min10sec) and much much faster than ND backend (~26min40sec)

7.9.2 Why HW4 failed on HW2

Most failures in HW2 are caused by

1. The test cases rely on `np.testing.assert_allclose()`, which only takes numpy array, so when it compares against my own `Tensor`, we will need to convert it to numpy (e.g. `logsumexp_forward((3, 3, 3), None) if BACKEND=="np" else logsumexp`).
2. `NDArray` only supports reduction over single axis, so reduction over multiple axes will fail (e.g., `logsumexp_forward((3, 3, 3), (1, 2))`)

logsumexp "error" In my `logsumexp` implementation, I use `squeeze()`, which is consistent w/ `numpy squeeze()`, that will make scalar-value of shape (). However, in `NDArray`'s `reduce_view_out` method is the one that powers all the reduction method (e.g, summation and max), which is the foundation of loss function related operations (e.g., `logsumexp`). A key difference (not sure good or bad) between `NDArray` (also `PyTorch`) and `NDArray` is the former will reduce to a scalar-value of shape (), whereas the latter will reduce to an array of shape (1,). And the reason it passed previous HW4 tests is that it only test on forward and the `np.testing.assert_allclose()` method will treat () and (1,) the same as long as they have the same actual scalar values.

type casting in C vs type casting in Python In C, a `float` multiplies a `int`, the result will be `float`. In python, a `fp32` multiples (or divides) a `int`, the result will be `fp64`. In order to make `ScalarDivision` happy, we add the triage on the backend, if it is `numpy` (aka following the python type casting), we need to first convert the scalar (aka the divider) to the type of the array (which is commonly is `fp32`), this is not a principled of doing things. But it is probably the pain we need to take if we want to support multiple backends when the typecasting rules differ. I like the C rules though (no surprise!)

Correct triage of default_device I implemented the correct triage of `default_device()` in `needle.backend_selection.py` (Notice I don't know exactly what is `needle.backend_numpy` is used for, but its `cpu()` is the device `init_basics.py` originally called.). Also I changed the `needle.init.init_basic.py` calls from `device=ndl.cpu()` to `device=default_device()` to get the device info correctly.

CUDA OOM when running apps/mlp_resnet.py weiz 2024-11-07 need to figure out where that AD graph OOM!

7.9.3 Why HW4 failed on HW3

In HW3, in `ndarray.py`, the `reduce_view_out()` didn't have the `keepdims` support, as a result, `sum()` and `max()`, both of which are reduction functions that depend on `reduce_view_out()` didn't support `keepdims`. Consequently, the test cases in hw3 didn't pass `keepdims`. But hw4 supports `keepdims`, so we simply need to copy hw4's `reduce_view_out()`, `max()`, `sum()` into hw3 and change hw3's test case to include `keepdims`. This is also what dlsys cmu folks have come to realize and fixed recently, so if you have taken this course in Fall 2024, you will see hw3 and hw4 are consistent. weiz 2024-10-26, I didn't do anything wrong, it is just dlsys folks did wrong in hw3.

7.9.4 Why HW4 failed on HW4

For the `test_train_cifar10()`

1. Reshape ops need to call `NDArray's compact()` before calling `NDArray's reshape()` as there's no guarantee that underlying `NDArray` is compact.
2. ReLU needs to rely on `NDArray's` dunder method `__gt__` to implement indexing and then return a `Tensor` during `gradient()` calculation.

7.10 Part 4: Recurrent neural network

7.10.1 RNN Cell Math formulation

The computation for an RNN cell at a single time step is as follows:

$$h_t = \text{nonlinearity}(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

We use the column major representation below for math clarity, in the code implementation, we will adopt the row-major representation.

- $h_t \in R^d$: Hidden state at time t .
- $x_t \in R^{n*}$: Input at time t .
- $h_{t-1} \in R^d$: Previous time-step hidden state.
- $W_{ih} \in R^{d \times n*}$: Weight matrix for input-to-hidden connections.
- $b_{ih} \in R^d$: Bias for input-to-hidden connections.
- $W_{hh} \in R^{d \times d}$: Weight matrix for hidden-to-hidden connections.
- $b_{hh} \in R^d$: Bias for hidden-to-hidden connections.

*: Notice only in the first RNN layer n could be different from d , in all later RNN layers $n \equiv d$.

Notice two separate linear layer involved: input-to-hidden (aka from previous layer but in the same timestep, i.e., down-to-up) and hidden-to-hidden (aka from previous step within the same layer, i.e., left-to-right). Figure 12 represents an RNN cell

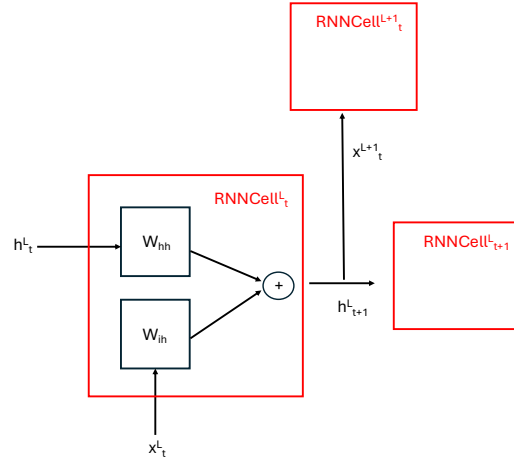


Figure 12: RNN Cell. At layer L , time sequence t , input from previous layer $L - 1$ is x_t^L , hidden state input from previous time sequence $t - 1$ is h_t^L . The learnable Parameters are W_{hh} and W_{ih} along with biases. The output is a vector of hidden dimension h_d , which is being used as the input to both next time step RNN cell in the same layer and the same time step RNN cell in the next layer. Further notice, the RNN cell parameters are shared in the same layer.

Lastly, in code (row-major representation) the input to `RNNCell`'s `forward()` function is

- **x** of shape `(bs, input_size)`: Tensor containing input features.
- **h** of shape `(bs, hidden_size)`: Tensor containing the initial hidden state for each element in the batch. Defaults to zero if not provided.

The output of `RNNCell`'s `forward()` function is

- **h'** of shape `(bs, hidden_size)`: Tensor containing the next hidden state for each element in the batch.

7.10.2 RNN (Module)

In L-layer RNN module, each layer contains one static `RNN_cell`, which is going to be unrolled t times, where t is the input sequence length. But in terms

of learnable parameters, each layer has only one `RNN_cell` worth of learnable parameters. The following discussions are in row-major.

Inputs

- `X` of shape `(seq_len, bs, input_size)` containing the features of the input sequence.
- `h_0` of shape `(num_layers, bs, hidden_size)` containing the initial hidden state for each element in the batch. Defaults to zeros if not provided.

Outputs

- `Y` of shape `(seq_len, bs, hidden_size)` containing the output features `h_t` from the last layer of the RNN, for each `t`.
- `h_n` of shape `(num_layers, bs, hidden_size)` containing the final hidden state for each element in the batch.

Figure 13 illustrates the RNN module.

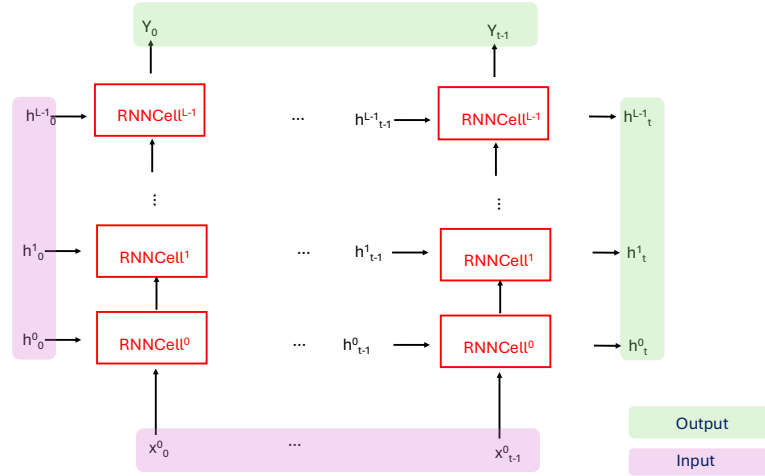


Figure 13: RNN Module. The input is initial `h` states and input `X`. The output is the last layers output `Y` and the final hidden state of each dynamically unrolled RNN cell in each layer. Further notice, the RNN cell parameters are shared in the same layer.

Key logic in `RNN forward()` . Three things worth noting:

1. The first layer is using the input `X` as the input. All the other layers are using the output hidden state of the previous layers' corresponding time step positions as inputs.

2. Within each layer, one just need to apply the RNN Cell math formula that takes in the input from previous layer and hidden state from previous step to generate the output for the current cell. Each layer is unrolled t times, where t is the sequence length.
3. The input X is of shape `(seq_len, bs, input_size)`, notice this is a bit unusual as the batch size is not the leading dimension. That is because the sequential nature of RNN, with in each layer, we need to unroll `seq_len` times, so each time the input to the `RNN cell` is `X[i]`, i being the time step, of shape `(bs, input_size)`.
4. weiz 2025-02-19, when I was looking at RNN again, I thought about how the BPTT algorithm is naturally implemented in auto-diff. I realized that each RNN cell parameters are actually the leaf node in the compute graph, and they will be used as in the input to each time step's matmul. Figure 14 illustrates how to unroll RNN cell for 3 times. Notice this is for simple illustration, I omit the input X part. But the gist is that RNN parameters matrix A is **input** (aka leaf node) in compute graph, and the gradients w.r.t A at each time-step t is naturally accumulated through its partial adjoints – E.g., if there are 3 steps unrolled as in the Figure 14, A will have 3 output edges that will collect 3 partial adjoints to sum up, each partial adjoint represents the gradients w.r.t each time-step.

Misc `TensorTuple`, notice `ops` must returns a `Value`. That is why `split` returns `TensorTuple`, which is a subclass of `Value`. As long as the sequence is greater than 1, the W_{hh} will be used and will be involved in backprop even the initial state `h_0` could be `None` ³⁹

7.10.3 LSTM Cell Math Formulation

Notice that this is very much like RNN cell, except that the **intermediate** hidden state is expanded 4 times, and there are bunch of element-wise operations on four sections of the intermediate hidden state that will lead to the final hidden state h_t , which is still of the dimension d . Following equations are in

³⁹This is the reason that even my impl is perfectly fine, but it failed the `language_model_implementation` test when initial state is `None` and seq length is 1.

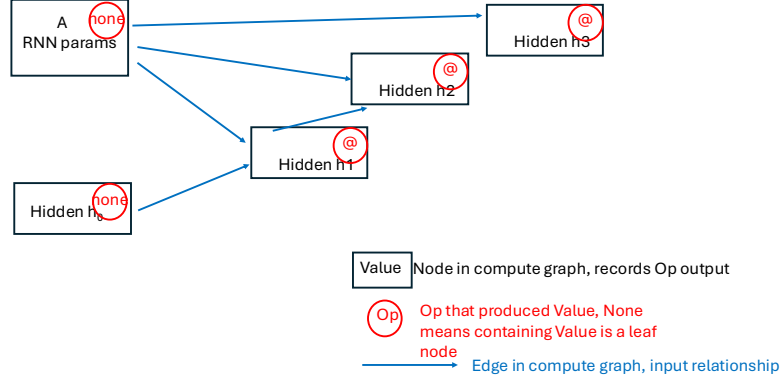


Figure 14: Unroll a RNN cell 3-times, depicted in Auto-Diff computation graph. @ means matmul. Each node is of **Value** class in the compute graph, its **Op** field represents the ops that generates the output for this node, edges represent the input relationship in the graph. When op is None, it means a leaf node (aka pure input). All trainable parameters are actually leaf nodes, this is the same design as in pytorch see <https://discuss.pytorch.org/t/are-the-parameters-of-nn-module-leaf-nodes-of-the-graph/110046/3>

column-major

$$W_{hh} \in R^{4d \times d}, W_{hx} \in R^{4d \times n}, b_h \in R^{4d} \quad (474)$$

$$x_t \in R^n$$

$$i_t, f_t, g_t, o_t, c_t, h_t \in R^d$$

$$\begin{bmatrix} i_t \\ f_t \\ g_t \\ o_t \end{bmatrix} = \begin{pmatrix} \text{sigmoid} \\ \text{sigmoid} \\ \text{tanh} \\ \text{sigmoid} \end{pmatrix} (W_{hh} h_{t-1} + W_{hx} x_t + b_h) \quad (475)$$

$$c_t = c_{t-1} \odot f_t + i_t \odot g_t ** \quad (476)$$

$$h_t = \tanh(c_t) \odot o_t \quad (477)$$

$$(478)$$

*: Similar to RNN cell, only in the first LSTM layer, where n can be different from d , all the other LSTM layers, $n \equiv d$.

**: Equation (476) is the key as f_t (forget gate) is a sigmoid output thus between 0 and 1, scales down c_{t-1} . In the meantime, g_t (cell gate) is a tanh output thus between [-1,1] can change i_t more flexibly.

Inputs to LSTMCell forward() Following are in row-major. `input_size` $\equiv n$. `hidden_size` $\equiv d$.

- X : Input tensor of shape `(bs, input_size)`, containing the input features.
- h : Tuple (h_0, c_0) , where:
 - h_0 : Tensor of shape `(bs, hidden_size)`, containing the initial hidden state for each element in the batch. Defaults to zero if not provided.
 - c_0 : Tensor of shape `(bs, hidden_size)`, containing the initial cell state for each element in the batch. Defaults to zero if not provided.

As we can see, the key difference from RNN cell is that we need to pass a $c_0 \in R^d$ in addition to $h_0 \in R^d$, all the rest are more or less the same.

Outputs to LSTMCell forward() Following are row-major

- Outputs: Tuple (h', c') , where:
 - h' : Tensor of shape `(bs, hidden_size)`, containing the next hidden state for each element in the batch.
 - c' : Tensor of shape `(bs, hidden_size)`, containing the next cell state for each element in the batch.

Again, the only difference from RNN cell is the output contains additional cell state in c' .

7.10.4 LSTM (Module)

It is very similar to RNN (Module) just with extra cell states.

Inputs to LSTM forward()

- X : Input tensor of shape `(seq_len, bs, input_size)`, containing the features of the input sequence.
- h : Tuple (h_0, c_0) , where:
 - h_0 : Tensor of shape `(num_layers, bs, hidden_size)`, containing the initial hidden state for each element in the batch. Defaults to zeros if not provided.
 - c_0 : Tensor of shape `(num_layers, bs, hidden_size)`, containing the initial cell state for each element in the batch. Defaults to zeros if not provided.

Outputs of LSTM forward()

- Outputs: Tuple (output, (h_n, c_n)), where:
 - **output**: Tensor of shape (seq_len, bs, hidden_size), containing the output features (h_t) from the last layer of the LSTM for each time step t .
 - Tuple (h_n, c_n), where:
 - * h_n : Tensor of shape (num_layers, bs, hidden_size), containing the final hidden state for each element in the batch.
 - * c_n : Tensor of shape (num_layers, bs, hidden_size), containing the final cell state for each element in the batch.

Logic of LSTM forward() It is exactly the same as RNN module, just with additional accounting of cell states.

7.10.5 One-hot encoding

E.g. class encoding for total number of 3 classes,

Class 0 $\rightarrow [1, 0, 0]$

Class 1 $\rightarrow [0, 1, 0]$

Class 2 $\rightarrow [0, 0, 1]$

notice that an array is populated from left to right, so an array [1,0,0] has the address 0 mapped to 1, address 1 mapped to 0, and address 2 mapped to 0. I mistakenly thought that class 0 should have been represented as [0,0,1], but that would map address 2 to 1. Typical confusion caused by how westerners read from left to right (aka having the MSB on the left, significance decreases from left to right), but the line number grows from left to right.

Notice how **one_hot** is implemented using **numpy's eye**, indexed with the category array. The category **array** can be of any shape, as long as the last dimension contains the category label. The resulting tensor will just replace the last column of **array** with **np.eye(category_column)**

7.10.6 Overall Seq-to-Seq Language Model

The input to LM is **X** of shape (seq_len, bs), each element is the index to the vocab, the embedding is done within the LM class's **forward()** function. The output of LM is the logits of shape (seq_len*bs, vocab_size)

Overall architecture of LM Figure 15 illustrates the RNN-based Language Modeling Architecture. A tokenizer first tokenizes the incoming texts as IDs into the vocab as the inputs to the LM, it then goes through embedding layer, RNN(LSTM) module, and then a linear project layer to project back to the vocab_size dimension before calculating the softmax-cross-entropy loss.

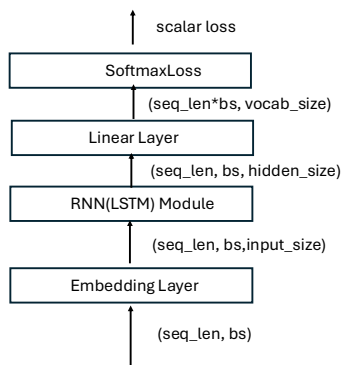


Figure 15: RNN-based Language Modeling architecture. The core LM stops at the output of linear layer, aka logits. I include the Softmax-Crossentropy-Loss for completeness.

Misc At the initial try of `python3 -m pytest -l -v -k "language_model_implementation,` I failed about 25% test cases that are of `seq=1` and `init_hidden_state=False`, because in this case I never involve the `W_{hh}` into the calculation (as the result would be all 0s anyway) and test failed at `assert W_{hh}'s grad` filed is not None (since it is never inserted into compute graph). But my implementation is just fine, the it is the stupid assertion required by the test cases are bad. In the end, I changed my implementation to initialize all those zeros just to make it comply with task description.

7.10.7 Penn Tree Bank (PTB) dataset

Build Vocab, Tokenize and Batchify Nothing special, except that the result of batchify is an array of `(seq_len, bs)`, each element is between `[0, vocab_size-1]`. Do notice that the first dimension is `seq_len`.

Misc

- Python has a funny way for `__getitem__` in a `Sequence` protocol, one would need to `raise IndexError("Index out of range")` to signal that the iterator has exhausted. (e.g., `PTBDataset` of my own impl). Otherwise, the iterator will just continue to go even if index is `leq` than `__len__`. On the other hand, for the `Iterator` protocol requires `raise StopIteration` to signal that the iterator has exhausted.
- One bug-fix is this: `if index >= (len(self) - 1): raise IndexError("Index out of range"),`

because if there is only one last element, we cannot make it a batch, as it won't have a label.

7.10.8 Align PyTorch implementation w/ Needle implementation

I wrote some utils functions to convert between pytorch and needle models. In particular, `Embedding`, `RNN` and `Linear`. Along the way, I found `Linear` and `RNN` in PyTorch have the weight matrix as `output_size x input_size` instead of the `input_size x output_size` as in needle. On the other hand, `Embedding` weight matrix is of the same shape in both PyT and NDL. Regardless, the expected input and output for all these three layers are always of the same shape (in the row-major shape), for both PyT and NDL.

Also, `Parameter` is nothing but `Tensor`

7.10.9 Use case: Train language modeling with PTB dataset

TODO: It is import to have the generated initial hidden states to be detached. See <https://discuss.pytorch.org/t/stupid-question-why-do-you-have-to-detach-the-hidden-state-of-lstms-but-not-the-hidden-state-of-a-linear-network/95089>, <https://discuss.pytorch.org/t/solved-why-we-need-to-detach-variable-which-contains-hidden-representation/1426/14> and possibly <https://dennybritz.com/posts/wildml/recurrent-neural-networks-tutorial-part-3/>

7.10.10 A simple test-case that compares needle and pytorch

Listing 45: Example Python Code

```
1  # In PyTorch, gradients are accumulated by default (compute graph  
   ↪ nodes are re-used)  
2  >>> x_t = torch.tensor(1.0, requires_grad=True) >>> y_t = x_t * 2  
3  >>> y_t.backward()  
4  >>> x_t.grad  
5  tensor(2.)  
6  >>> z_t = x_t * 3  
7  >>> z_t.backward()  
8  >>> x_t.grad  
9  tensor(5.)  
10 # In Needle, gradients are refereshed each time (e.g., new  
    ↪ compute graph nodes are inserted)  
11 >>> x=ndl.Tensor(1)  
12 >>> y=x*2 >>> y.backward() >>> x.grad >>> needle.Tensor(2.0)  
13 >>> z=x*3  
14 >>> z.backward()  
15 >>> x.grad  
16 needle.Tensor(3.0)
```


TODO: We should use this simple case to illustrate how the compute nodes and graph are constructed in needle and possibly answer how come we are using this much memory.

7.10.11 What I have learnt from the RNN-based LM parity against PyTorch

1. `Linear` layer (and the like e.g., RNN cell) has the weight matrix in the column-major to create the illusion of the LADR way of doing things, though the input is still row-major based. Per chatgpt, it is at no cost to do a transpose when doing a matmul.
2. `model.zero_grad()` (or the like done in optimizer reset grad) is a must for pytorch model, as otherwise the gradients will be accumulated) see https://github.com/weidotwisc/10714/blob/0cd5f1f00fe413efc8d90422cb925d4df70b2b26/hw_code/hw4/apps/utlis.py#L345, see last section.
3. `hidden_state` used between batches (i.e., the output of the last batch as the initial hidden states for the next batch) need to be detached from the compute graph. See https://github.com/weidotwisc/10714/blob/0cd5f1f00fe413efc8d90422cb925d4df70b2b26/hw_code/hw4/apps/weiz_test.py#L790 and https://github.com/weidotwisc/10714/blob/0cd5f1f00fe413efc8d90422cb925d4df70b2b26/hw_code/hw4/apps/simple_ml.py#L302
4. `(num_layers, bs, hidden_size)`-shape `init_hidden_states` tensors are used for the RNN module. So that in each layer, the first token in the sequence has something input horizontally.
5. The difference between RNN and LSTM is just the in LSTM internally the `hidden_size` is expanded 4 times so have some more element-wise operations. And the `init_hidden_states` is also expanded twice, so that there is also a `cell` initial hidden states.
6. Finally, the the shape of input to RNN-based language modeling is (i) `(seq_len, bs)`, each element is between `[0, vocab_size-1]` (ii) after embedding layer, it becomes `(seq_len, bs, emb_dim)` (iii) after RNN module, it becomes `(seq_len, bs, hidden_size)` (iv) after FC layer, it becomes `(seq_len x bs, vocab_size)` (aka logits). (v) Finally, logits will take a softmax and then (negation) inner product w/ `one_hot(target)` (`target` is of `(seq_len x bs,)` shape, `one_hot(target)` is of `(seq_len x bs, vocab_size)` shape) to get the loss.

Key thing is the `seq_len` is the leading dimension and `bs` is the 2nd dimension for the input, this is also the default shape for PyT RNN module (but you can change it by setting `batch_first` to be `True`)

weiz 2024-12-07 I finished pyt and ndl RNN-based LM parity test! Finally! Now ready to move on to the last homework the transformer one! oh, yay! Notice that I did a 2-epoch train of the entire ptb dataset, needle gets `train_loss: [6.05945193]`, `eval_loss: [6.08410817]`, whereas pyt gets `train_loss: 6.0902`, `eval_loss: 6.1008`. If I just train few batches, I can get very close parity, so maybe there is something difference between the optimizer update rule. But overall, the parity is established. I feel the most fulfilling part of this homework is that I did an RNN-based language modeling implementation and converter between pytorch and needle to find discrepancies and establish parity, which can be applied to any pytorch Module learning!

8 Homework 4 Extension – Transformer

8.1 Why previous approaches are not as good ? (A recap from lecture)

8.1.1 RNN "latent state" approach

Pros: Potentially "infinite" history, compact representation

Cons: Long "compute path" between history and current time $\Rightarrow$ vanishing /exploding gradients, hard to learn.

8.1.2 CNNs for direction prediction

Pros: Often can map from past to current with shorter compute path. I.e., through a combination of kernel length and depth of the network.

Cons: No compact state representation, finite history in practice. See Figure 16

Potential Solutions :

- Increase kernel size
- Pooling layers (weiz: pooling is sorta like reduction operator, e.g., averaging or max). But not well suited here, as it takes too much a summary not the sequence in details
- Dilated convolution (e.g, Shiyu, Wei Tan and Xiaodong's paper :))

8.2 How Transformer works ?

8.2.1 Overall architecture

Notice the Feed forward layers are 2-layer FC layer, the first FC layer has non-linearity (e.g., ReLU or GeLU functions), the 2nd FC layer just does a projection to project it to the space that is of `hidden_size` dimension.

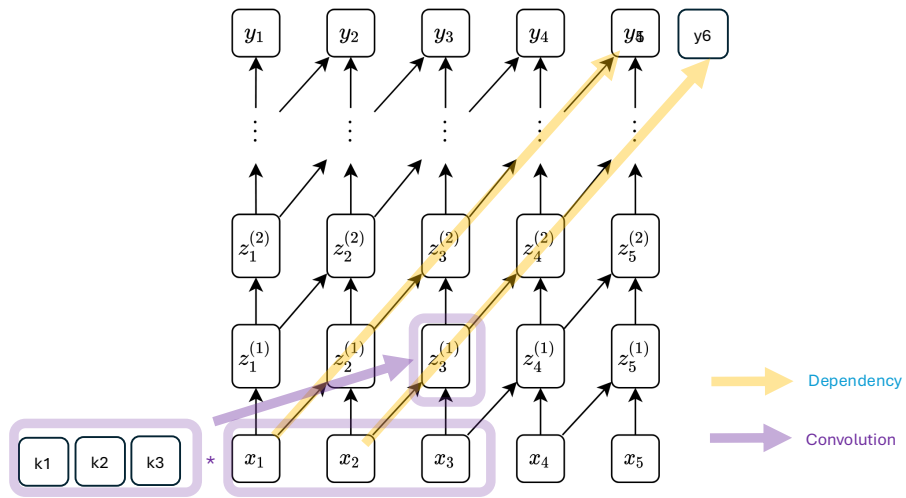


Figure 16: A Convolution-based sequence modeling (aka Wavenet). Two things worth noting: (1) larger kernel size can have larger dependency window, but we need to make sure it also obeys temporal constraints. e.g., $z_2^{(1)}$ can only be the convolution result of kernels and x_1 and x_2 , but no x_3 is allowed, even though the kernel size is 3 in our case (2) y_6 depends only on x_2 and onward, that is the dependency chain is dictated by kernel size and depth of network. So it is difficult to model arbitrary dependency chain in CNN.

The two cons of naive self-attn mechanism:

1. All outputs depend on all inputs. Fixed by causal masking in Section 8.2.2
2. No order of data (transformers are equivariant to permutations of the sequence) . Fixed by positional encoding (embedding) in Section 8.2.5

8.2.2 Attention matrix (in presence of causal masking)

What is $A[i, j]$? It means three things

- $A[i, j]$ quantifies how much token j contributes to the representation of token i . One can think of this as the row-major matmul (aka computation point of view), thus it shall be lower triangle, as later tokens contribute nothing to previous tokens.
- $A[i, j]$ reflects the dependence of token i on token j in the context of the sequence. Thus it shall be a lower triangle. As early tokens have no dependence on later tokens.
- $A[i, j]$ represents the attention that token i gives to token j . Thus it shall be lower triangle. As early tokens give no attention to the later tokens.

In summary, $A[i, j] \equiv i$'s attention to $j \equiv i$'s dependence on $j \equiv j$'s contribution to i . The rows are girls and the columns are boys (aka girls pay attention to boys and boys contribute to girls). The attention matrix is **lower triangular**.

8.2.3 Fourier Series, Fourier Transform, Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT)

You're absolutely right to highlight this distinction! Let me clarify why the Fourier Transform often appears in terms of complex exponentials ($e^{-j\omega t}$) instead of explicitly using sines and cosines like the Fourier Series does, and how they are connected.

Fourier Series Representation The Fourier Series decomposes a periodic, continuous-time signal $f(x)$ into a sum of sines and cosines. For a function with period 2π , the Fourier Series is given by:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} [a_n \cos(nx) + b_n \sin(nx)],$$

where a_n and b_n are real-valued coefficients computed by integrating the function multiplied by $\cos(nx)$ and $\sin(nx)$, respectively. This is a purely real-valued decomposition and is particularly intuitive for periodic functions.

To compute the coefficients a_0 , a_n , and b_n in the Fourier Series for a periodic function $f(x)$ with period 2π , use the following formulas:

$$a_0 = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) dx, \quad (479)$$

$$a_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(nx) dx, \quad n = 1, 2, 3, \dots, \quad (480)$$

$$b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(nx) dx, \quad n = 1, 2, 3, \dots \quad (481)$$

Notice this is what typical calculus course would cover.

Fourier Transform Representation The Fourier Transform generalizes the Fourier Series to handle non-periodic functions. Instead of using discrete frequencies, it represents the signal $f(t)$ as a continuous spectrum of frequencies $F(\omega)$. The standard definition of the Fourier Transform is:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt,$$

where $e^{-j\omega t}$ is a complex exponential. While this form doesn't explicitly involve sines and cosines, it is equivalent to combining these components due to Euler's formula:

$$e^{-j\omega t} = \cos(\omega t) - j \sin(\omega t).$$

Decomposing into Sines and Cosines Substituting Euler's formula into the Fourier Transform gives:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) \cos(\omega t) dt - j \int_{-\infty}^{\infty} f(t) \sin(\omega t) dt.$$

Here, the real part of $F(\omega)$ corresponds to the cosine components, while the imaginary part corresponds to the sine components. Thus, the Fourier Transform inherently includes sines and cosines, but combines them compactly into a single expression using complex exponentials.

Advantages of Complex Exponentials The use of complex exponentials provides several advantages. First, it offers a compact representation, eliminating the need to handle separate integrals for sines and cosines. Second, it encodes both the amplitude and phase of each frequency component in a single term: the magnitude $|F(\omega)|$ gives the amplitude, and the argument $\arg(F(\omega))$ gives the phase. Finally, it aligns with broader mathematical tools, such as linear algebra and complex analysis, making it more versatile for applications in physics and engineering.

Notice two things,

- Fourier series deal with **real-valued** and **periodic** functions only, thus taught at freshman level calculus class.

- Fourier transform works on **complex-valued** function and it doesn't have to be periodic. Most real signal functions in the real world have a Fourier transform. This is typically covered in a class like DSP, ODE / PDE, which I need to take later.

Discrete Fourier Transform (DFT) is a mathematical technique used to analyze and represent discrete signals in the frequency domain. It transforms a finite sequence of equally spaced samples, typically taken from a continuous signal, into a discrete set of complex numbers that represent the amplitudes and phases of sinusoidal components at specific frequencies. The DFT is defined by the formula:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}, \quad k = 0, 1, \dots, N-1 \quad (482)$$

where $x[n]$ is the input signal, $X[k]$ is the k -th frequency component, N is the number of samples, and j is the imaginary unit. The inverse DFT (IDFT) reconstructs the original signal from the frequency-domain representation. The DFT is widely used in signal processing, audio and image compression, and spectral analysis due to its ability to reveal the frequency content of discrete signals. While powerful, its computational cost scales quadratically with the number of samples, $O(N^2)$, which led to the development of the more efficient **Fast Fourier Transform (FFT)** algorithm, reducing the complexity to $O(N \log N)$.

Fast Fourier Transform (FFT) The FFT relies heavily on the mathematical properties of the **roots of unity**. A root of unity is a complex number that satisfies the equation:

$$W_N^N = 1,$$

where $W_N = e^{-j \frac{2\pi}{N}}$ is the fundamental N -th root of unity. The key property of these roots is their **cyclic periodicity**, meaning:

$$W_N^{k+N} = W_N^k.$$

This periodicity simplifies calculations in the FFT by allowing repeated terms to be reused. Additionally, the roots of unity exhibit **symmetry**, including:

$$\begin{aligned} W_N^{N-k} &= \overline{W_N^k}, & (\text{conjugate symmetry}), \\ W_N^{2k} &= W_{N/2}^k, & (\text{even-odd symmetry}). \end{aligned}$$

These properties enable the FFT to decompose a large N -point DFT into smaller, more manageable sub-problems, reducing the computational complexity significantly.

In summary, **Fourier Series** works in real domain and handles only periodic function, **Fourier Transform** expands the idea of Fourier series to the complex field so that the representation is compact and contains both amplitude and phase info and it can handle non-periodic functions as well. **DFT** is the discretized version of Fourier transform. **FFT** is a fast algorithm that implements **DFT**. In the original transformer paper, it uses i -th dimension in hidden vector as the different frequency as in **Fourier Series**.

8.2.4 Convolution and FFT Optimization

Definition of Convolution The convolution of two sequences $x[n]$ (input signal) and $h[n]$ (filter or kernel) is defined as:

$$y[k] = \sum_{m=0}^{n-1} x[m]h[k-m],$$

where:

- $x[m]$ is the input sequence of length n ,
- $h[m]$ is the kernel or filter of length n ,
- $y[k]$ is the output sequence of length n .

Each output value $y[k]$ is computed by shifting $h[m]$, multiplying it with the corresponding values of $x[m]$, and summing the results.

Computational Complexity of Convolution In the naive implementation of convolution:

- For each output index k (total n output points), the kernel $h[m]$ is shifted and multiplied with a segment of the input $x[m]$.
- Each computation involves n multiplications and additions.
- Since this is done for all n output points, the total number of operations is:

$$O(n \cdot n) = O(n^2).$$

Example Let $x = [1, 2, 3]$ and $h = [1, 0, -1]$. The convolution $y[k]$ is computed as:

$$\begin{aligned} y[0] &= 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1), \\ y[1] &= 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 0, \\ y[2] &= 1 \cdot (-1) + 2 \cdot 0 + 3 \cdot 1. \end{aligned}$$

For each $y[k]$, there are $n = 3$ multiplications and additions. With $n = 3$ output points, the naive approach involves $n^2 = 9$ operations.

Optimizing Convolution with FFT The convolution theorem states that convolution in the time domain corresponds to element-wise multiplication in the frequency domain. This property allows convolution to be computed more efficiently using the FFT:

1. Transform both $x[m]$ and $h[m]$ to the frequency domain using the FFT:

$$X[f] = \text{FFT}(x[m]), \quad H[f] = \text{FFT}(h[m]).$$

This step has complexity $O(n \log n)$.

2. Multiply the transformed signals element-wise in the frequency domain:

$$Y[f] = X[f] \cdot H[f].$$

This step has complexity $O(n)$.

3. Apply the inverse FFT to $Y[f]$ to obtain the convolved result $y[k]$ in the time domain:

$$y[k] = \text{IFFT}(Y[f]).$$

This step also has complexity $O(n \log n)$.

Total Complexity of FFT-Based Convolution The FFT-based convolution reduces the overall complexity to:

$$O(n \log n),$$

which is much faster than the naive $O(n^2)$ approach for large n . **Notice that even FFT could convert something real to complex, in the convolution case, certain properties guarantee the ifft result will be always in real. Also notice there seems to be a flipping kernel involved, we need to revisit this after i finish ODE/PDE and DSP course.**

8.2.5 Positional encoding (embedding)

Math formula PE is a `seq_len` x `hidden_size` matrix. In the original paper, both seq len and hidden size are 512. I think in modern arch, seq len is a lot larger

The sinusoidal positional encoding is defined as follows:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \quad (483)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \quad (484)$$

- pos : The position index of a token in the input sequence (e.g., 0, 1, 2, ...).
- $2i$: The even dimensions of the positional encoding vector.

- $2i + 1$: The odd dimensions of the positional encoding vector.
- d_{model} : The dimensionality of the model embeddings (hidden size).
- $10000^{\frac{2i}{d_{\text{model}}}}$: A scaling factor that determines the frequency of the sine and cosine functions for each dimension.

This encoding scheme provides a way to represent the positional information of tokens in a sequence. By using sine and cosine functions with different frequencies, the encoding captures both fine-grained and global positional information, enabling the model to reason about token positions effectively.

PE and Fourier series We discuss how is this PE scheme related to Fourier series.

First, notice that $\frac{1}{10000^{\frac{2i}{d_{\text{model}}}}}$ is the frequency for the sinusoid function that takes pos as input variable.

Second recall the three important integrals that demonstrate the orthogonality of fourier series.

$$\int_{-\pi}^{\pi} \sin(mx) \cos(nx) dx = 0, \quad \text{for all integers } m \text{ and } n. \quad (485)$$

$$\int_{-\pi}^{\pi} \sin(mx) \sin(nx) dx = \begin{cases} 0, & \text{if } m \neq n, \\ \pi, & \text{if } m = n. \end{cases} \quad (486)$$

$$\int_{-\pi}^{\pi} \cos(mx) \cos(nx) dx = \begin{cases} 0, & \text{if } m \neq n, \\ \pi, & \text{if } m = n. \end{cases} \quad (487)$$

Notice, per chatgpt, the orthogonality of Fourier series could extend to real numbers for m and n (i.e., not necessarily always integer). This means if we treat pos as a function variable, then for each dimension of `hidden_size` we achieve orthogonality because either they are of different frequencies, or they are of same frequency but using different sinusoid functions (e.g., sin vs cos). In another word, across the **columns** of PE matrix are orthogonal!⁴⁰. Such orthogonal columns mean the embedding vectors use an orthogonal basis as basis, which is a nice thing!

What PE is really trying to achieve ? Position Encoding really tries to encode (or embed, if we have learnable parameters) **each position**, as if it were each **token**, to the hidden space; so that the encoding of that position can be added on top of the embedding of that token. Here, we would like to have **orthogonal features** (e.g., think about FFT and PCA). That is to say we can apply **Gram method** or **Covariance method** to test if the features of PE are orthogonal – simply put, stack each position’s encoding as a row vector to a matrix X (a sample matrix used for testing feature orthogonality) and then

⁴⁰Notice, discretized integral is just inner product

test if each **column** of X is orthogonal to the other columns, aka test if XX^T is a **diagonal** matrix – in the last two last sentences of the last paragraph we made the point explicitly: In another word, across the **columns** of PE matrix are orthogonal!

Linearity w.r.t k PE(pos+ k) is a linear function of PE(pos), due to trig identity Supposedly easy to use linear layer to capture. ChatGPT mentioned it could use

$$\sin(a + b) = \sin(a) \cos(b) + \cos(a) \sin(b) \quad (488)$$

$$\cos(a + b) = \cos(a) \cos(b) - \sin(a) \sin(b) \quad (489)$$

$$PE(pos + k) = A(k)PE(pos) \quad (490)$$

$$A(k) = \begin{bmatrix} \mathbf{R}(f_0 \cdot k) & 0 & 0 & \cdots & 0 \\ 0 & \mathbf{R}(f_1 \cdot k) & 0 & \cdots & 0 \\ 0 & 0 & \mathbf{R}(f_2 \cdot k) & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \mathbf{R}(f_m \cdot k) \end{bmatrix} \quad (491)$$

$$\mathbf{R}(f_i \cdot k) = \begin{bmatrix} \cos(f_i \cdot k) & \sin(f_i \cdot k) \\ -\sin(f_i \cdot k) & \cos(f_i \cdot k) \end{bmatrix}. \quad (492)$$

$R(f_i \cdot k)$ is a rotation matrix, $A(k)$ is a block-diagonal matrix that happens to be orthogonal. As a result from vector PE(pos) to a vector PE(pos+ k) is just a linear operator. (weiz 2024-12-11 I need to verify this as it is only spilled by ChatGPT).

As hidden dimension index increases, frequency decreases a lot faster
TODO:include that PE sin plot from annotated transformer

Higher frequency i.e., lower hidden index captures local info, lower frequency i.e., higher hidden index captures global info This is just a nice conceptual summary.

8.2.6 Theoretical Justification for Dividing by $\sqrt{d}$ in Scaled Dot-Product Attention

Dot Product Between Query and Key Vectors Consider the dot product between two d -dimensional vectors $\mathbf{q}$ and $\mathbf{k}$:

$$s = \mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^d q_i k_i$$

where q_i and k_i are independent random variables with:

$$\mathbb{E}[q_i] = \mathbb{E}[k_i] = 0, \quad \text{Var}(q_i) = \text{Var}(k_i) = 1.$$

Expectation of the Dot Product Using the properties of expectation:

$$\mathbb{E}[s] = \mathbb{E}\left[\sum_{i=1}^d q_i k_i\right] = \sum_{i=1}^d \mathbb{E}[q_i k_i].$$

Since q_i and k_i are independent and zero-mean:

$$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0.$$

Thus:

$$\mathbb{E}[s] = 0.$$

Variance of the Dot Product The variance of s is:

$$\text{Var}(s) = \text{Var}\left(\sum_{i=1}^d q_i k_i\right).$$

For independent random variables:

$$\text{Var}\left(\sum_{i=1}^d X_i\right) = \sum_{i=1}^d \text{Var}(X_i).$$

Here, each term $q_i k_i$ has variance:

$$\text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - \mathbb{E}[q_i k_i]^2.$$

Since $\mathbb{E}[q_i k_i] = 0$, we have:

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1 \cdot 1 = 1.$$

Therefore:

$$\text{Var}(s) = \sum_{i=1}^d \text{Var}(q_i k_i) = d.$$

Implication The dot product $s = \mathbf{q} \cdot \mathbf{k}$ has:

$$\mathbb{E}[s] = 0, \quad \text{Var}(s) = d.$$

As d increases, the variance of s grows linearly with d . This means the values of s will be spread across a wider range, potentially leading to very large values for high-dimensional vectors.

Effect on Softmax The softmax function is sensitive to the scale of its inputs:

- For large input values, softmax outputs become very sharp, assigning almost all weight to the largest value.
- For small input values, softmax outputs are more balanced.

If we do not scale s by $\sqrt{d}$, the large variance of s for high-dimensional vectors will result in excessively sharp attention distributions, which can:

1. Lead to poor learning dynamics (e.g., overconfident attention).
2. Cause vanishing gradients for most entries in the attention matrix.

Scaling by $\sqrt{d}$ To address this, we divide s by $\sqrt{d}$, yielding:

$$s' = \frac{s}{\sqrt{d}} = \frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d}}.$$

The new scaled dot product s' has:

$$\mathbb{E}[s'] = 0, \quad \text{Var}(s') = \frac{\text{Var}(s)}{d} = \frac{d}{d} = 1.$$

This normalization ensures that the variance of the scaled dot product is independent of the dimensionality d , preventing large input values and stabilizing the softmax.

Conclusion Dividing by $\sqrt{d}$ normalizes the variance of the dot product, ensuring that the scaled attention mechanism is stable and well-behaved. This step prevents overly sharp attention distributions and improves the model's ability to learn effectively.

Notice $\text{Var}(aX) = a^2 \text{Var}(x)$

8.3 Two auxiliary implementations professors have provided

8.3.1 Softmax, to track max or not to track in gradients' calculation ?

softmax, logsumexp and cross-entropy

$$z \in R^n \text{ } z \text{ is logits} \quad (493)$$

$$p \in R^n \text{ } p \text{ is the one-hot encoding} \quad (494)$$

$$q \in R^n \text{ } q \text{ is the model softmax output} \quad (495)$$

$$.- \text{ means dot subtract, i.e., element-wise subtract} \quad (496)$$

$$\text{softmax}(z) \in R^n \quad (497)$$

$$\text{logsumexp}(z) \in R \quad (498)$$

$$\text{cross-entropy}(p, q) \in R \quad (499)$$

$$\text{softmax}(z) = \frac{e^z}{\sum_{i=1}^{i=n} e^{z_i}} \quad (500)$$

$$= \frac{e^z .- \max(z)}{\sum_{i=1}^{i=n} e^{z_i - \max(z)}} \quad (501)$$

$$\text{logsumexp}(z) = \log\left(\sum_{i=1}^{i=n} e^{z_i}\right) \quad (502)$$

$$= \log\left(\sum_{i=1}^{i=n} e^{z_i - \max(z) + \max(z)}\right) \quad (503)$$

$$= \log\left(\sum_{i=1}^{i=n} e^{z_i - \max(z)} e^{\max(z)}\right)$$

$$= \log\left(e^{\max(z)} \sum_{i=1}^{i=n} e^{z_i - \max(z)}\right)$$

$$= \log(e^{\max(z)}) + \log\left(\sum_{i=1}^{i=n} e^{z_i - \max(z)}\right) \quad (504)$$

$$= \log\left(\sum_{i=1}^{i=n} e^{z_i - \max(z)}\right) + \max(z) \quad (505)$$

$$\text{cross-entropy}(p, q) = - \langle p, \log(q) \rangle \quad (506)$$

$$= - \langle p, \log(\text{softmax}(z)) \rangle \quad (507)$$

$$= - \langle p, \log\left(\frac{e^z}{\sum_{i=1}^{i=n} e^{z_i}}\right) \rangle \quad (508)$$

$$= - \langle p, \log(e^z) - \log\left(\sum_{i=1}^{i=n} e^{z_i}\right) \rangle \quad (509)$$

$$= - \langle p, z - \text{logsumexp}(z) \rangle \quad (510)$$

$$= \langle p, \text{logsumexp}(z) \rangle - \langle p, z \rangle \quad (511)$$

$$\text{lse}(z) \in R, p \text{ is one-hot} \stackrel{=}{=} \text{logsumexp}(z) - \langle p, z \rangle \quad (512)$$

As we see the the $z - \max(z)$ is just a numerical stabilizer for both `softmax` and `logsumexp`. In theory, we could subtract any number not only $\max(z)$, as a result when we calculate gradients, we can treat $\max(z)$ as a constant number chosen out of convenience and should not track in the gradients calculation, see the last paragraph of this section. **Notice that at the beginning, I had forgotten about the $\log(q)$ part and mistakenly thought cross entropy is $-\langle p, \text{softmax}(z) \rangle$, but the $\log$ part is very important, and why `logsumexp` came into the picture. Also, later in stats class we are going to learn log-likelihood and max likelihood estimation!**

Why initial cross entropy loss is $\log(n)$, where n is number number of classes ? There are two ways of seeing it. **Alternative 1:** cross entropy of p and q is just $-\langle p, \log(q) \rangle$. At the beginning of the training, q is about a uniform distribution, $q_i = \frac{1}{n}$. Since p is one-hot encoding, $-\langle p, \log(q) \rangle = -\log(q_i) = \log(n)$. **Alternative 2:** Per Equation (512), we have $ce - loss(p, q) = lse(z) - \langle p, z \rangle$: at the beginning of training, z is initialized vector of mean 0 very small floating numbers, thus $\langle p, z \rangle \equiv z_i \approx 0$, $lse(z) \equiv \log(\sum_{i=1}^{i=n} e^{z_i}) \approx \log(\sum_{i=1}^{i=n} e^0) = \log(n)$. Notice that Alternative 1 is from the physical meaning explanation perspective while Alternative 2 is from a computation point of view.

max gradients, track or not track ? In the professors' implementation `softmax`, I find they didn't track `max` operation's gradients. Notice I will need to leverage the cross-entropy loss in $\frac{\partial(L)}{\partial(logits)} = (\text{softmax(logits)} - y)^T$, where y is the target one-hot encoding. Actually no, we are really not trying to bake the selection of `max` into the loss function, it is merely a smart way to choose a good value to make the calculation more stable. Fascinating! I should write a few pytorch code to verify this, actually if I do track the gradients for `max`, that will change the definition of `softmax` and even though the forward calculation is correct, the backward calculation is probably going to be messed up (as it turns out, very luckily, it is not). So fascinating, I should have documented that to prove that tracking or not tracking is equivalent, this cannot be proved in the general case but it can be proved in the cross-entropy loss case. But it is not the point of the prof's point, he is merely trying to find a stable way to calculate the loss **and** gradients w.r.t logits. If we do track the `max` function, that would actually be wrong for general purpose loss gradients calculation but won't be too bad for cross-entropy loss!! Apparently, we should not track gradients of `max` for general numerical-stable `submax` operation. Next, I give both the theoretical proof and coding proof to prove that (1) in general, tracking `max` and not tracking `max` lead to different gradients calculation, but (2) in cross entropy loss, tracking `max` and not tracking lead to the same gradients w.r.t logits. Still, we shall not track `max` in cross-entropy loss, as `max` is not a must in the equation but only a mere convenience to stabilize the numerical computation.

Assuming $x \in R^n$, we have

$$\begin{aligned} \text{argmax}(x) &= k \\ \frac{\partial \text{max}(x)}{\partial x} &= \begin{bmatrix} 0 \dots & 1 & 0 \dots \\ 0 \dots & 1 & 0 \dots \\ 0 \dots & \underbrace{\dots}_{1} & 0 \dots \\ & k\text{-th col} & \end{bmatrix} \end{aligned} \quad (513)$$

$$z = \text{submax}(x) = x - \text{max}(x) \quad (514)$$

$$\frac{\partial z}{\partial x} = \begin{bmatrix} 1 & & -1 & \\ & \ddots & -1 & \\ & & 0 & \\ & & \vdots & \\ & & \underbrace{-1}_{k\text{-th col}} & 1 \end{bmatrix} \quad (515)$$

Case I (a simple sum):

$$z = \text{submax}(x) = x - \text{max}(x) \quad (516)$$

$$L = \text{sum}(z) \quad (517)$$

when not tracking max

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} \quad (518)$$

$$\text{max}(x) \text{ is just constant} \underline{\underline{=}} \underbrace{[1, \dots, 1]}_n \text{eye}(n) \quad (519)$$

$$= \underbrace{[1, \dots, 1]}_n \quad (520)$$

when tracking max

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} = [1, \dots, 1] \begin{bmatrix} 1 & & -1 & \\ & \ddots & -1 & \\ & & 0 & \\ & & \vdots & \\ & & \underbrace{-1}_{k\text{-th col}} & 1 \end{bmatrix} \quad (521)$$

$$= [1 \dots, \quad 1 - n \quad \dots, 1] \quad (522)$$

Apparently, Equation (520) and Equation (522) are not the same even for this simple case

Case II (cross-entropy):

$$\begin{aligned} z &= x - \max(x) \\ a &= \text{softmax}(z) \end{aligned} \quad (523)$$

$$L = \text{cross-entropy}(p, a) \quad (524)$$

$$\frac{\partial L}{\partial z} \stackrel{\text{Equation (158)}}{=} (a - p)^T \quad (525)$$

when not tracking matrix

$$\begin{aligned} \frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z} \frac{\partial z}{\partial x} \\ &\stackrel{\max(x) \text{ is treated as constant}}{=} \frac{\partial L}{\partial z} \text{eye}(n) \\ &= \frac{\partial L}{\partial z} \\ &= (a - p)^T \end{aligned} \quad (526)$$

when tracking max

$$\begin{aligned} k &= \text{argmax}(x) \\ \frac{\partial L}{\partial x} &= (a - p)^T \begin{bmatrix} 1 & & -1 \\ & \ddots & -1 \\ & & 0 \\ & & \vdots \\ & & -1 \\ & \underbrace{\hspace{1cm}}_{k\text{-th col}} & 1 \end{bmatrix} \\ &= a^T \begin{bmatrix} 1 & & -1 \\ & \ddots & -1 \\ & & 0 \\ & & \vdots \\ & & -1 \\ & \underbrace{\hspace{1cm}}_{k\text{-th col}} & 1 \end{bmatrix} - p^T \begin{bmatrix} 1 & & -1 \\ & \ddots & -1 \\ & & 0 \\ & & \vdots \\ & & -1 \\ & \underbrace{\hspace{1cm}}_{k\text{-th col}} & 1 \end{bmatrix} \\ &\stackrel{\sum_{i=1}^{i=n} a_i = 1, \sum_{i=1}^{i=n} p_i = 1}{=} [a_1 \dots, \quad a_k - 1 \quad, \dots a_n] - [p_1 \dots, \quad p_k - 1 \quad, \dots p_n] \\ &= [a_1 - p_1 \dots, \quad a_k - p_k \quad, \dots a_n - p_n] \\ &= (a - p)^T \end{aligned} \quad (527)$$

$$(528)$$

Apparently, Equation (528) and Equation (526) are **same** for cross-entropy loss gradients calculation, with or without tracking max.

In summary, tracking or not tracking max, when **submax** is applied, will be different for general gradients calculation. But cross-entropy loss has nice properties that tracking or not tracking is the same for the gradients w.r.t logits.

Still, we need to decide shall we track max, depending on the meaning of the equation. Even in the cross-entropy loss case, we **shall not** track max as max is merely there for stabilizing numerical calculation. Fascinating. I also write a program that verified this https://github.com/weidotwisc/10714/blob/1753c1db337d59a0fe785525db979a9d109b4db1/hw_code/hw4_extra/apps/bikinie.py#L109

8.3.2 Batch matmul

Using element-wise multiply and summation over the last axis to implement 2D matmul

$$\begin{aligned} A &\in R^{m \times n}, B \in R^{n \times k}, B^T \in R^{k \times n} \\ A_{\text{reshape}} &\in R^{m \times 1 \times n}, B_{\text{reshape}}^T \in R^{1 \times k \times n} \\ A_{\text{reshape|bcast}} &\in R^{m \times k \times n}, B_{\text{reshape|bcast}}^T \in R^{m \times k \times n} \end{aligned} \quad (529)$$

We need to prove

$$A @ B = (A_{\text{reshape|bcast}} * B_{\text{reshape|bcast}}^T).sum(axis = -1) \quad (530)$$

proof: First Rename $A_{\text{reshape|bcast}}$ as $C \in R^{m \times k \times n}$, $B_{\text{reshape|bcast}}^T$ as $D \in R^{m \times k \times n}$. Then we just need to prove that

$$\sum_{p=1}^{p=n} C[i, j, p] * D[i, j, p] = AB[i, j] \quad (531)$$

$$C[i, j, p] \stackrel{\substack{\text{j's stride is 0} \\ \text{bcast def}}}{=} A[i, p] \quad (532)$$

$$D[i, j, p] \stackrel{\substack{\text{i's stride is 0} \\ \text{bcast def}}}{=} B^T[j, p] = B[p, j] \quad (533)$$

$$\sum_{p=1}^{p=n} C[i, j, p] * D[i, j, p] \stackrel{\substack{\text{Equation (532)} \\ \text{Equation (533)}}}{=} \sum_{p=1}^{p=n} A[i, p] * B[p, j] \stackrel{\substack{\text{Strang} \\ \text{1st matmul}}}{=} AB[i, j] \quad \square \quad (534)$$

Extend to multi-dim batch matmul Assume

$$A \in R^{\dots \times m \times n}, B \in R^{\dots \times n \times k}, B^T \in R^{\dots \times k \times n} \dots \text{ means matching dimensions after bcast } C = A_{\text{reshape|bcast}} \text{ in } R^{\dots \times m \times k \times n} \quad (535)$$

To prove

$$A @ B = (C * D).sum(axis = -1) \quad (536)$$

aka,

$$A @ B[\dots, i, j] = (C * D).sum(axis = -1)[\dots, i, j] \quad (537)$$

Proof: For **any** realization $x \in \dots$ (e.g., if $\dots$ is of shape (2,3), x could be [0,0] or [0,1]), from Equation (534), we can trivially get

$$A @ B[x, i, j] = (C * D).sum(axis = -1)[x, i, j] \quad \square \quad (538)$$

8.4 Part 1: Implementing the Multi-Head Attention Activation Layer

The `Module` to implement is `MultiHeadAttention`. The **input** is $Q, K, V \in R^{B \times H \times T \times D}$, the **output** is $V \in R^{B \times H \times T \times D}$, $Attn \in R^{B \times H \times T \times T}$.

This section just deals with attention calculation, assuming Q, K, V are already calculated. Therefore, there are no trainable parameters.

8.4.1 The attention calculation formula

B batch size, H # of heads, T seq len, D dim per head

queries $Q \in R^{B \times H \times T \times D}$, keys $K \in R^{B \times H \times T \times D}$, values $V \in R^{B \times H \times T \times D}$

$$Attn = \text{dropout}(\text{softmax}(\frac{QK^T}{\sqrt{D}})) \quad (539)$$

$$X = Attn @ V \quad (540)$$

$$X \in R^{B \times H \times T \times D}, Attn \in R^{B \times H \times T \times T}$$

$$\text{shape}(Q) \equiv \text{shape}(K) \equiv \text{shape}(V) \equiv \text{shape}(X)^{41}$$

Notice that (1) the `matmul` used in Equation (540) are `batched-matmul`. (2) `dropout` is applied to the probabilities calculated via `softmax`, since `dropout` scales up the remaining values, the result is still a valid probability distribution.

8.4.2 MultiHeadAttention (Module) illustration

Figure 17 is the illustration of `MultiHeadAttention` module.

8.4.3 Practical issues

1. `triu` in `numpy`, `i` is the query length, `j` is the key length
2. Scaling QK^T by $1/\sqrt{dim}$ first, then subtract mask, instead of subtracting mask then scaling.
3. Dropout module always scales up the remaining values, so `dropout(softmax())` is still a valid probability distribution.
4. The `matmul` impl takes 2nd matrix in transpose format, so we can directly call `matmul(Q, K)`, instead of `(q, KT)`
5. Remember to `bcast` the mask (aka the matrix with diagonal shifted up one position of large values) to subtract. Also notice the resulting attention probability matrix is a lower-triangular with non-zero diagonals. The current token position n is attended as we are trying to predict the next token at position $n + 1$, so there is no shame to attend the current position :)

weiz 2024-12-29 finished with almost only one try, on mom's birthday :)

⁴¹as long as the dim per head for each tensor is the same

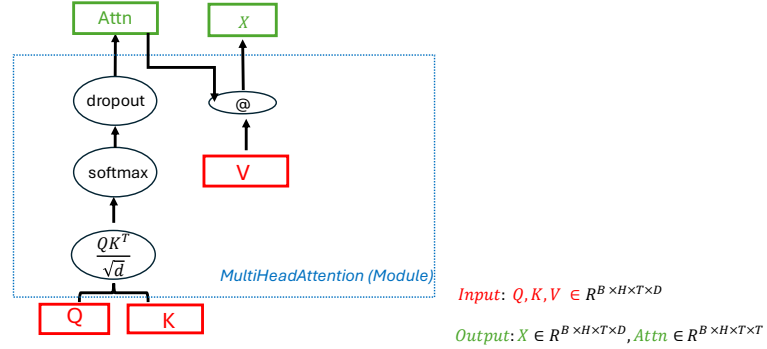


Figure 17: MultiHeadAttention Module

8.5 Part 2 Implementing the Self-Attention Layer with trainable parameters

The Module to implement is **AttentionLayer**. Then input is $Q, K, V \in \mathbb{R}^{B \times T \times D'_{Q|K|V}}$

⁴² The **output** is $X \in \mathbb{R}^{B \times T \times D'_Q}$. The learnable parameters are linear layers $W_q, W_k, W_v \in \mathbb{R}^{D'_{Q|K|V} \times (HD)}$ and the output projection linear layer $W_o \in \mathbb{R}^{(HD) \times D'_Q}$.

8.5.1 Math formulae

B is the batch size, T is the sequence length, D'_Q, D'_K, D'_V are the embedding dimensions (or the input dimensions of the **AttentionLayer**), which are usually the same and could be simplified as D' . H is the number of heads, D is the per head dimension for the Q, K, V vector-space, (HD) is the dimension of Q, K, V

⁴²In this homework, D' could be different for Q, K, V . Though in decoder-only architecture, they are always the same.

vector-space.

$$Q, K, V \in R^{B \times T \times D'_{Q|K|V}}, W_q, W_k, W_v \in R^{D'_{Q|K|V} \times (HD)}$$

Step1: layernorm and project to Q,K,V space

$$Q' = \text{LayerNorm}_q(Q)W_q, Q' \in R^{B \times T \times (HD)} \quad (541)$$

$$K' = \text{LayerNorm}_k(K)W_k, K' \in R^{B \times T \times (HD)} \quad (542)$$

$$V' = \text{LayerNorm}_V(V)W_v, V' \in R^{B \times T \times (HD)} \quad (543)$$

Step2: reshape and permute to prepare calls to Attention module

$$Q'' = (Q'.\text{reshape}((B, T, H, D))).\text{permute}((0, 2, 1, 3)), Q'' \in R^{B \times H \times T \times D} \quad (544)$$

$$K'' = (K'.\text{reshape}((B, T, H, D))).\text{permute}((0, 2, 1, 3)), K'' \in R^{B \times H \times T \times D} \quad (545)$$

$$V'' = (V'.\text{reshape}((B, T, H, D))).\text{permute}((0, 2, 1, 3)), V'' \in R^{B \times H \times T \times D} \quad (546)$$

Step3: call Attention Module to get X''

$$X'' = \text{MultiHeadAttention}(Q'', K'', V'')^{43} \quad (547)$$

Step4: reshape and permute to prepare calls to project back to input space

$$X' = (X''.\text{permute}((0, 2, 1, 3))).\text{reshape}(B, T, (HD)), X' \in R^{B \times T \times (HD)} \quad (548)$$

Step5: project back to input space

$$X = X' @ W_o \quad (549)$$

8.5.2 AttentionLayer (Module) illustration

Figure 18 is the illustration of **AttentionLayer** module. Notice that I think this is the most import module in this homework. And realize that there are 4 sets of trainable parameters: $Q_{proj}, K_{proj}, V_{proj}$ and W_{output} , in a more popular/mundane architecture (e.g., PyTorch's **MultiHeadAttention** module), all of such matrices of of $embed_{dim} \times embed_{dim}$. The multi-head split and re-group/concatenation were achieved in the 1st **reshape&permute** op (aka split to multi-heads after $Q_{proj}, K_{proj}, V_{proj}$) and in the 2nd **reshape&permute** op (aka regroup / concatenate after the needle's **MultiHeadAttention** op). In our homework, the dimension of $Q_{proj}, K_{proj}, V_{proj}$ however can be configured as **q_features x inner_dim, k_features x inner_dim** and **v_features x inner_dim**; W_{output} is of **inner_dim x out_features**. Again, in PyTorch's **MultiHeadAttention**, **q_features** $\equiv$ **k_features** $\equiv$ **v_features** $\equiv$ **inner_dim** $\equiv$ **out_features**. This is also what modern GPT like model does, so in the input and output of **AttentionLayer** are of exactly the same shape **bs x seq_len x embed_dim**. Notice that **inner_dim = num_heads x head_dim**

Also notice, there is no **Dropout** layer anywhere in **AttentionLayer**, except for the attention matrix computed in the **MultiHeadAttention** module.

⁴³MultiHeadAttention module is implemented in Section 8.4

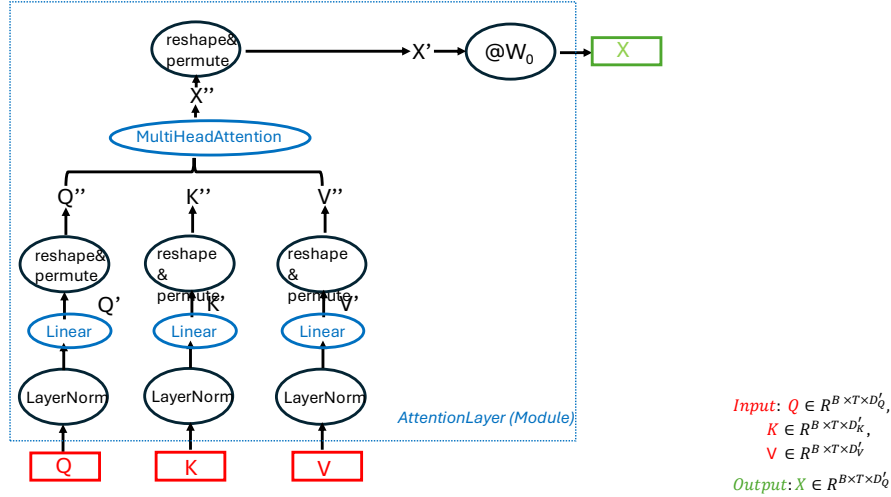


Figure 18: AttentionLayer Module. Notice needle impl first conducts linear projection that have trainable parameters, after which permute and reshape to apply MultiHeadAttention. Finally it will project to the same space via W_0 trainable parameters. Notice that in Needle we don't constrain that input dimension and output dimension are the same. In contrast, **torch.nn.MultiHeadAttention** constrains that input and output dimension must be the same.

8.5.3 LayerNorm

What we implemented was LayerNorm1D, which only works with 2D tensor (bs by dim). In this module, we need to make it work with $\mathbb{R}^{B \times T \times D'}$ shape, we just need to reshape the input tensor to $\mathbb{R}^{(BT) \times D'}$ shape first.

8.5.4 Prenorm vs Postnorm

This Prenorm vs Postnorm section is taken entirely from ChatGPT. While in original Transformer paper, Post-norm was used. In this homework, Prenorm is used.

Key Difference: Gradient Flow and Residual Path In **postnorm**, the residual connection passes the raw outputs of the sublayer directly to the next layer:

$$\text{Output}_{\text{Postnorm}} = \text{LayerNorm}(X + \text{Sublayer}(X))$$

This means that the normalization step happens **after** combining the input X with the sublayer's output.

In **prenorm**, the normalization is applied **before** the sublayer, so the residual

path always contains the input X directly:

$$\text{Output}_{\text{prenorm}} = X + \text{Sublayer}(\text{LayerNorm}(X))$$

This difference leads to **distinct gradient dynamics**:

- In **prenorm**, the residual connection X is untouched during the forward pass, allowing gradients to flow more directly back through the residual path. This mitigates the risk of vanishing or exploding gradients.
- In **postnorm**, gradients may need to propagate through both the sublayer and LayerNorm, which can introduce instability in deep models.

Why It's Not Just About the First or Last Layer While it may seem that the difference only affects the **first layer's input** or the **last layer's output**, this isn't the case:

- **Internal Interactions:** LayerNorm reshapes the inputs to each sublayer, so whether this happens **before** or **after** the sublayer affects how information is processed and passed forward.
- **Residual Signal:** In **prenorm**, the residual X is untouched, providing a clearer signal for the next layer. In **postnorm**, the signal passed forward has already been altered by LayerNorm.

The Effect on Depth

- For **shallow Transformers** (e.g., 6 layers), the differences between prenorm and postnorm are minimal because gradient degradation is less of an issue.
- For **deep Transformers** (e.g., 24+ layers), prenorm significantly improves gradient flow, allowing much deeper architectures to train effectively without specialized tricks (e.g., learning rate warmup or gradient clipping).

Summary The main distinction isn't just at the first or last layer but in how **residual connections** and **LayerNorm** interact **within each layer**. Prenorm simplifies the residual path, improving gradient flow and training stability, particularly in very deep Transformers.

8.5.5 Implementation

Notice that

1. **AttentionLayer** transforms input tensor to the output tensor of the same shape
2. Q,K,V projections don't have bias terms, though normally it should have – guess profs want to design test cases that are easier on them

3. **MultiHeadadAttention** implemented in part 1 will return not only the outputs but also the softmax probabilities, so just make sure use a tuple to capture the returned results.
4. Finally, Q must be passed in, if K, V are not passed in as inputs, then they default to Q thus becoming a self-attention model, which is the backbone of the decoder-only transformer architecture and the the focus of the rest of the homework.

8.6 Part 3 Implementing a prenorm residual Transformer Layer

The module to implement is **TransformerLayer**. The **input** is $X \in R^{B \times T \times D'}$, the **output** is $X \in R^{B \times T \times D'}$, aka **TransformerLayer** keeps the input and output shape consistent, which is the key to enable **residual** connections and deep architectures ⁴⁴. D' is the embedding dimension. The trainable parameters are the ones in the **AttentionLayer** and two Linear layers that scales up, introduce some non-linearity and then scales down to the embedding dimension space.

8.6.1 Math formulae

$$x = x + \text{Dropout}(\text{AttentionLayer}(q = x, k = x, v = x))$$

$$x = x + \text{Dropout}(\text{Linear}_2(\text{Dropout}(\text{ReLU}(\text{Linear}_1(\text{LayerNorm}(x))))))$$

Linear_1 is a $D' \times \text{hidden_dim}$ layer, Linear_2 is a $\text{hidden_dim} \times D'$ layer. We only need one activation function layer, aka the output of Linear_2 doesn't need to go through an activation layer, similar to CNN models. hidden_dim typically is larger than D' .

8.6.2 Illustration of Transformer Layer

Figure 19 is the illustration of **TransformerLayer** module.

8.6.3 Implementation Details

Several things to notice,

1. This is a self-attention model, aka $Q = X, K = X, V = X$.
2. This is a prenorm's implementation (i.e., apply layer norm directly to input instead of output)
3. **TransformerLayer** transforms input tensor of shape $R^{B \times T \times \text{emb_dim}}$ to the output tensor of the same shape of $R^{B \times T \times \text{emb_dim}}$, in doing so, we can use **residual** connection to enable deep architecture, and this is a

⁴⁴This is **different** from **RNNCell** where embedding dimension and hidden dimension don't necessarily match.

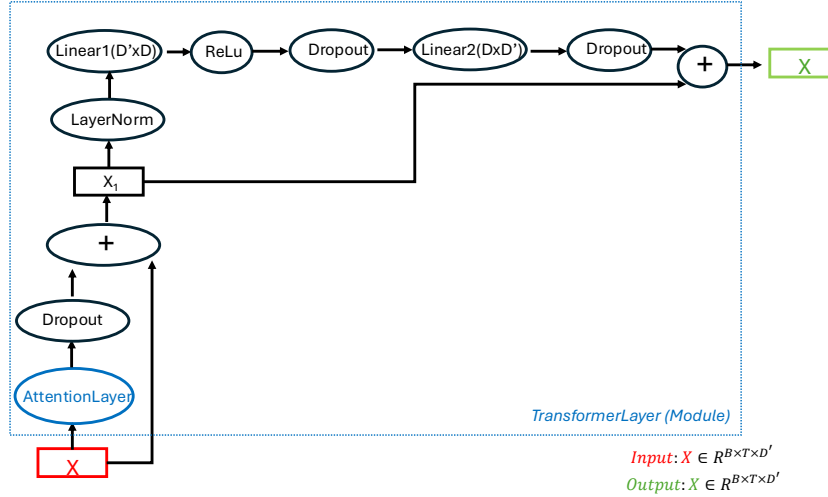


Figure 19: Transformer Layer Module

key difference between Transformer and RNN, where the input and output dimension don't necessarily match and don't use residual connections.

4. **AttentionLayer** has no bias terms for Q,K,V (though normal transformer should have the bias terms for them, i think it is because the profs want the test cases to be easier on them), MLP layers (aka Linear1 and Linear2 layers) do have bias terms.

8.7 Part 4 Implementing the Transformer model

The input to **Transformer** model is of shape $X \in \mathbb{R}^{B \times T \times D'}$, the output is $X \in \mathbb{R}^{B \times T \times D'}$. D' is the embedding dimension (aka **embedding_size** or **q_features**). The positional embedding is using a $T \times D'$ **Embedding** layer (T is the sequence length and can be thought as the "vocab size").

8.7.1 Math Formulae

$$x = x + pos_embedding(x) \quad (550)$$

$$x = TransformerLayer_i(x) \quad i = 1..N \quad (551)$$

8.7.2 Illustration of Transformer Module

Figure 20 is the illustration of **Transformer** module.

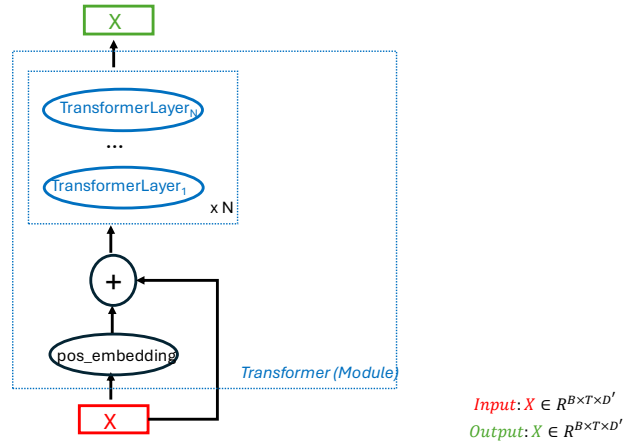


Figure 20: Transformer Module

8.7.3 Implementation details

1. Notice that the `Embedding` takes `(seq_len, bs)` as input shape (to be compatible with pytorch's `Embedding`, which originally tackled RNN and LSTM), we need to permute it to work in the `(bs, seq_len)` shape to work with transformer modules.
2. Finally, I had to change the `rto1` from `1e-5` to `1e-4` to make all tests passed, otherwise 1 of the 32 test cases will fail on 1 of the 1080 elements. The problematic test case was:

```
test_transformer_model(
    batch_size=8, seq_len=5, input_dim=27,
    hidden_size=64, num_layers=4,
    num_head=8, dim_head=32,
    causal=False, dropout=0.1, device=ndl.cpu())
```

8.8 Transformer Language model on the Penn Tree bank dataset

8.8.1 Key different between RNN and Transformer

1. seq len is baked in transformer, but in RNN it is deduced from input tensor and can be arbitrary number via dynamically unrolling RNN cells
2. The input tensor shape for RNN is `seq x bs x embedding_dim`, the input tensor shape for Transformer is `bs x seq x embedding_dim`

3. in LM, seq length has nothing to do with RNN/LSTM trainable parameters; but it could be related to transformer based trainable parameters particularly for positional embedding. However, if the positional embedding / encoding uses some thing like RoPE, then seq length has nothing to do with transformer trainable parameters either. Notice, the Q,K,V projection matrix is dimension is `q_features` by `out_features`, where most of them time they are just hidden dimensions.

8.8.2 Pratical issues

1. notice still have pretty bad memory issue (OOM on 16GB v100, but okay on 80GB A100, why I don't have such isses in RNN and LSTM ? maybe because my parameters size are not that large in RNN and LSTM TODO: change the python file name to something more like `LM_test.py` After 10 epochs (running on A100 80GB gpu), I get `eval_loss = [5.33069158]`, `eval_acc = 0.18607346339779005`

8.9 Parity check between PyTorch and Needle Transformer Implementation

8.9.1 MultiHeadAttention (pyt) vs AttentionLayer (ndl)

1. pyt: `in_proj_weight`: projection weights for q, k and v, packed into a single tensor. Weights are packed along dimension 0, in q, k, v order. ndl: `q_proj`, `k_proj`, `v_proj` are separated matrices.
2. pyt: has bias terms. ndl: none of the linear layers have bias terms
3. pyt: `MultiHeadAttention` doesn't apply `LayerNorm`. ndl: apply `LayerNorm` to input query, key, value (aka pre-norm)
4. pyt: the default input shape is `seq_len x bs x dim` (unless set `batch_first=False`. ndl: the default input shape is `bs x seq_len x dim`
5. pyt: `MultiHeadAttention` returns logits and attention matrix. ndl: `AttentionLayer` only returns logits. attention matrix is returned in the `MultiHeadAttention` module in needle.

I have used https://github.com/weidotwisc/10714/blob/f24060758eb46439709b4c10268337b81dee92fc/hw_code/hw4_extra/apps/utils.py#L435 to establish parity between `torch.nn.MultiHeadAttention` and `needle.nn.AttentionLayer`.

8.9.2 TransformerEncoderLayer (pyt) vs TransformerLayer(ndl)

Notice that if we turn on `norm_first` for `TransformerEncoderLayer` in pyt as in https://pytorch.org/docs/stable/_modules/torch/nn/modules/transformer.html#TransformerEncoderLayer.forward, we might get

the same implementation as in `TransformerLayer` in `ndl` (also notice paper <https://arxiv.org/pdf/2002.04745v1>). To check: write a simple numpy code to check for the forward part.

The implementation of `torch.nn.TransformerEncoderLayer` uses post-norm, whereas in `needle.nn.TransformerLayer` uses pre-norm. Figure 21 illustrates the difference between Postnorm and Prenorm. So the tricky part is that I need to understand by turning on the `norm_first=True`, can I get the same implementation as in `needle`. For that purpose, I implemented three things in https://github.com/weidotwisc/10714/blob/main/hw_code/hw4_extra/apps/explore_pyt_transformer_decoder.py:

1. `parity_transform_post_norm_numpy()`, a copy from lecture just to make sure that we have a numpy post-norm implementation to gauge against `torch.nn.TransformerDecoderLayer`'s default (aka post-norm)
 2. `parity_transform_pre_norm_numpy()`, based on [11] and documents in pytorch, gauge against `torch.nn.TransformerDecoderLayer`'s `norm_first=True`
 3. Once above is made sure, we can compare the needle's prenorm implementation against `torch.nn.TransformerDecoderLayer`'s `norm_first=True` in `parity_transform_pre_norm_needle()`
1. `TransformerDecoderLayer` (in `pyt`) asks the input `embed_dim` must be equal to `num_heads × head_dim`. Whereas in `TransformerLayer` (in `ndl`), we don't have such constraints.

See `explore_pyt_transformer_decoder.py`

8.9.3 PyTTransformerLanguageModel (pyt) vs LanguageModel (ndl)

In https://github.com/weidotwisc/10714/blob/main/hw_code/hw4_extra/apps/pyt_lm_transformer.py I implemented PyT based transformer LM. In https://github.com/weidotwisc/10714/blob/main/hw_code/hw4_extra/apps/ndl_lm_transformer.py, I implemented Needle based transformer LM.

Caveats in PyT impl

1. The dataset we use is needle based, as a result, the input tensor is RNN-like input tensor in the format of `seq_len × bs`, instead of `bs × seq_len`. So we need to transpose them. **This is the most important piece that I missed.** See https://github.com/weidotwisc/10714/blob/957f82798515b69f8dfa2bb06185ccd732db7a55/hw_code/hw4_extra/apps/pyt_lm_transformer.py#L51
2. Remember we need to provide causal mask `M` when calling into `TransformerDecoderLayer`!
3. Remember we need to implement positional embedding, and the 'vocab size' is really the sequence length in our case.

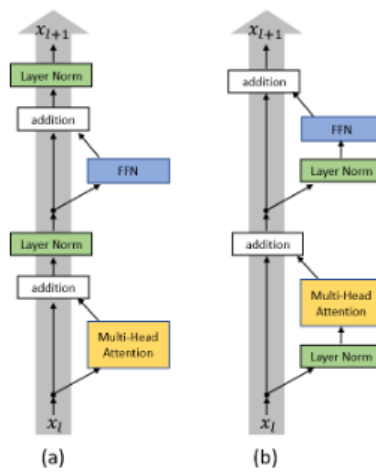


Figure 1. (a) Post-LN Transformer layer; (b) Pre-LN Transformer layer.

Figure 21: Postnorm (default PyTorch TransformerEncoderLayer) vs Prenorm (turned on `norm_first = True`). The figure is taken from [11]

4. Remember we should turn on the `train` mode and `eval` mode when proper (so that things like Dropout and LayerNorm can work properly during training and eval, though this is less important to affect model performance).

5. We need to turn on `torch.no_grad()`: during inference so that it can run faster (i.e., w/o affecting compute graph), this is not a model performance issue.
6. Remember for Modules like `Residual` and `Dropout`, we don't need to set devices, as they don't have trainable parameters. But other layers, we need to set device properly
7. In `TransformerDecoderLayer`, the embedding dimension is always the same as output dimension.

Caveats in Needle impl

1. Notice that the `LanguageModel` in needle assumes the input tensor is RNN-friendly `seq_len x bs`, we need to handle it properly (i.e., transpose to `seq_len`. Also `Embedding` layer in needle assumes `seq_len x bs`, but `Embedding` layer in pytorch can work with whatever shape.
2. Notice that `Linear` layer in needle only handles 2D input matrix, so we need to make sure reshape the `(bs, seq_len, embed_dim)` into `(bs x seq_len, embed_dim)`.

Eventually, we get pretty close performance (even though we cannot get completely same performance, because in `AttentionLayer`'s linear layers, we don't have bias terms in needle). However, it seems that needle based implementation has slightly better performance than the pytorch one.

```
python apps/ndl_lm_transformer.py
(1 epoch)
eval_loss = [6.07330316], eval_acc = 0.12936269854972376
(10 epoch)
eval_loss = [4.70171681], eval_acc = 0.22103655904696132
```

```
python apps/pyt_lm_transformer.py
Eval Loss: 5.8096 Eval correctness: 0.1304 ( 1 epoch )
Eval Loss: 4.8413 Eval correctness: 0.1997 (10 epoch)
```

weiz 2025-01-19 On this day, I have finally finished my QueSang!!! :)

9 HW5 Annotated Transformer

In this Section, Wei is creating a new homework 5 in needle to do a parity implementation of Annotated transformer in needle.

9.1 High-level missed points

1. Encoder's output from the **last** layer is going to be used as parts of input to **every** Decoder layer.

2. **LayerNorm** trainable parameters scaling γ and shift β are of the same size of output features (i.e., each Transformer layer's embedding dimension)
3. **Cross-Attention** in Decoder:
 - **Query (Q)**: Comes from the decoder (target language embeddings).
 - **Key (K)** and **Value (V)**: Come from the encoder (source language representations).

This allows the decoder to attend to relevant parts of the source sentence while generating the next token.

9.2 PyTorch and Needle Parity Implementation

TODOs!!!

10 TODOs left

1. Garbage collector so that we can train w/o too much memory. Seems my RNN-based LM doesn't have memory leak issue though ;_P
2. `filteridlate's gradients()`
3. Maybe implement `__getitem__` and `__setitem__` for `Tensor` class so that we don't have to use `stack` and `split` (though they are not too bad to use)
4. Maybe support batch matmul (simply call underlying batch gemm matmul will be probably better). Though current reshape then call 2D matmul is not too bad (yet)
5. Support `named_parameter` (like what done in PyTorch) for better
6. Write my own testing framework.
 - Study what did Chen and Zico did for testing!
 - Use numerical difference (e.g., Stefan matlab code, cmu dlssys code)
 - Use some Gradient checking from scipy or numpy ?
 - Write pytorch-based "alignment" converters (from/to PyT and Needle), just like what I did for RNN. This is the ultimate test!
7. Parity testing between pytorch optimizers and needle optimizers
8. In Pytorch, non leaf node's gradients are not kept around unless you use `retain()`. In my needle's implementation, I have kept around non-leaf nodes' gradients. Maybe I should discard those, and might pass the memory test in one of the tests that complain too many tensors are used
9. Use tools like `GraphViz` to visualize AD graph.

11 Towards Deep Learning Framework verification

This section describes our effort towards verification of the deep learning framework.

11.1 How to build and run the code

11.1.1 How to build the code

11.2 How to run the code

There are three backends `nd`, `nd_cuda`, `np`, defined by environment variable `BACKEND`

11.2.1 Run the transformer

Run transformer with

```
export NEEDLE_BACKEND=nd_cuda
time python apps/ndl_lm_transformer.py
real    37m46.392s
user    12m58.083s
sys     21m18.504s
```

```
export NEEDLE_BACKEND=nd
time python apps/ndl_lm_transformer.py
real    37m20.196s
user    12m58.470s
sys     21m5.475s
```

11.3 Overall Software Architecture

11.4 Artifacts of the code base

LoC, number of test cases

11.5 Verification by component

11.5.1 Verification of the Autodiff algorithm

11.5.2 Verification

References

- [1] Sheldon Jay Axler. *Linear Algebra Done Right*. Undergraduate Texts in Mathematics. Springer, New York, 1997.

- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [3] Randal J. Barnes. Matrix differentiation (and some other stuff). 2009.
- [4] Tianqi Chen and Zico Kolter. dlsyscourse.org, 2023.
- [5] Stefan Harmeling. Blue note 142 (matrix differential calculus cheatsheet). **PersonalCommunication**.
- [6] Stefan Harmeling. Probabilistic reasoning and machine learning. https://www.youtube.com/watch?v=PmhJCl_DWH8&list=PLzrCX1f6ypbzDYKDchKfM-I9s20mFCL0q.
- [7] Wen-mei W. Hwu, David B. Kirk, and Hajj Izzat. *Programming Massively Parallel Processors, Fourth Edition: A Hands-on Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 4th edition, 2022.
- [8] Jan R. Magnus and Heinz Neudecker. *Matrix Differential Calculus with Applications in Statistics and Econometrics*. John Wiley, third edition, 2019.
- [9] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale, 2022.
- [10] Michael A. Nielsen. Neural networks and deep learning, 2018.
- [11] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *CoRR*, abs/2002.04745, 2020.
- [12] Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. Understanding and improving layer normalization, 2019.